Public contact-email-redacted derivative | arXiv:2611.17334v1

arXiv:2611.17334v1 [cs.CV] 12 Nov 2026

# Unbundling Multimodal Vision Encoding

**Ellis Brown**[*∞♆], **Chris Hoang**[*∞♆],
**Shengyi Qian**[∞], **Shang-Wen Li**[∞], **Mike Lewis**[∞], **Marjan Ghazvininejad**[∞], **Luke Zettlemoyer**[∞], **Lucas Beyer**[∞],
**Mengye Ren**[†♆], **Jakob Verbeek**[†∞], **Saining Xie**[†♆], **Hu Xu**[†∞]

[∞]Meta FAIR, [♆]New York University
[*]equal contribution, ordered randomly, [†]equal advising

How multimodal language models should encode visual input remains up for debate. Most models use a pretrained vision encoder, while some recent recipes train the encoder from scratch with the language model or remove it entirely. Vision encoders bundle together three design choices: a *visual prior* from upstream pretraining, *dedicated vision capacity*, and *pre-fusion placement* of that capacity (i.e., before the multimodal backbone). Because published models also differ in data, architecture, and compute, comparing them cannot isolate the effect of each choice. We disentangle these choices via a controlled framework where we pretrain all models from scratch on common data and compare them on visual question answering and language benchmarks at matched *lifetime compute*. We find that the visual prior is not necessary; an encoder trained jointly from scratch is on par with a pretrained one, without a separate training stage. What matters instead is dedicated vision capacity: an encoder-free model trails on visual understanding even when its backbone is enlarged by the encoder's parameter count. Yet that capacity need not precede multimodal fusion, as vision-only modules added *in-fusion* to early backbone layers perform comparably to a pre-fusion encoder and can lower time to first token. These results also hold across backbone sizes, when encoder pretraining is treated as free, and when starting from a pretrained language model. Overall, our findings favor dedicated vision capacity trained jointly with the backbone and placed early within it.

**Date:** November 12, 2026
**Blog:** https://unbundling.github.io

Meta NYU

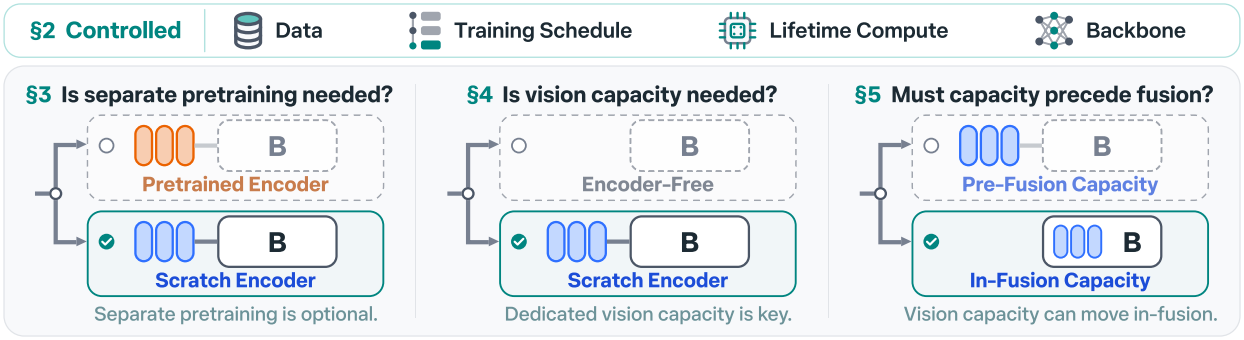



**Figure 1 A controlled study of multimodal vision encoding.** (Top) Every recipe shares one backbone architecture ("B") and trains all modules from scratch on common data at matched lifetime compute (§2). (Bottom) Each panel unbundles one vision encoder design choice, marks the favored recipe (✓), and states its takeaway; **orange** marks a pretrained prior and **blue** dedicated vision capacity. §6 goes beyond lifetime compute by treating encoder pretraining as free, adding vision to a pretrained language model, and measuring training and inference speed.

## Contents


**1 Introduction** **3**

**2 A controlled protocol for comparing multimodal recipes** **4**
2.1 Recipes and the fusion boundary . . . . . 4
2.2 Common data . . . . . 4
2.3 Lifetime compute . . . . . 4
2.4 Training schedule . . . . . 5
2.5 Evaluation . . . . . 5

**3 Does the vision encoder need separate pretraining?** **5**

**4 Does vision encoding need dedicated capacity?** **6**

**5 Does dedicated capacity need to precede multimodal fusion?** **8**
5.1 Moving the encoder's capacity inside the backbone . . . . . 8
5.2 Where and how to add in-fusion capacity . . . . . 9
5.3 Can in-fusion capacity match a pre-fusion encoder? . . . . . 9
5.4 Results hold with scale . . . . . 10

**6 Vision encoding beyond lifetime compute** **11**
6.1 Zero-cost pretrained encoders . . . . . 11
6.2 Midtraining from pretrained language models . . . . . 11
6.3 Training and inference speed . . . . . 12

**7 Related work** **12**

**8 Discussion** **13**

**References** **14**

**Appendix** **21**

**A Controlled protocol** **21**

**B Evaluation** **26**

**C Separate encoder pretraining** **29**

**D Dedicated vision capacity** **32**

**E In-fusion vision capacity** **36**

**F Decay, learning rate, and caption loss** **38**

**G Scaling ladder** **40**

**H Beyond lifetime compute: midtraining and inference** **43**

**I Per-benchmark results** **47**

# 1 Introduction

Most multimodal models acquire visual understanding from a vision encoder pretrained for a separate objective, such as image recognition or image–text alignment, rather than from being trained directly with the multimodal backbone that uses its features (He et al., 2016; Radford et al., 2021; Tschannen et al., 2025; Siméoni et al., 2026; Tschannen et al., 2023). Multimodal training has progressed from freezing this encoder (Alayrac et al., 2022; Li et al., 2023b; Liu et al., 2023; Muse Glimmer Team, 2026) to updating it (Beyer et al., 2024; Bai et al., 2025a), but encoder pretraining has remained a separate stage. Recent recipes bring vision and language closer by discarding parts of the pretrained encoder setup: Kimi K3 trains a vision encoder from scratch jointly with the multimodal backbone (Kimi Team, 2026b), and encoder-free models directly process pixel patches, forcing visual encoding to be learned entirely within the backbone (Adept AI, 2023; Diao et al., 2024; Liu et al., 2026; Gemma Team, 2026; Thinking Machines Lab, 2026a). Each recipe implies a different answer to what the vision encoder contributes, if anything at all. We ask which of its inherited design choices benefit multimodal models trained entirely from scratch.

The predominant multimodal vision encoding recipes bundle together three design choices: (i) a *visual prior* from upstream pretraining, (ii) *dedicated vision capacity* (i.e., parameters that only vision tokens use), and (iii) *pre-fusion placement* of that capacity in a separate encoder that operates before vision and text tokens interact in the backbone—rather than operating *in-fusion*, within the shared backbone. A pretrained encoder keeps all three, a from-scratch encoder gives up only the prior, and an encoder-free model gives up all three.

Existing comparisons cannot isolate these choices, for two reasons. First, they entangle the choices with each other and the rest of the recipe. Published models differ in data, training stages, compute, and architecture. While controlled studies hold multimodal training data and compute fixed, their comparisons among pretrained encoders (McKinzie et al., 2024; Tong et al., 2024a; Fan et al., 2025; Li et al., 2025) vary *which visual prior* an encoder imports rather than *whether it needs one* in the first place. Studies of encoder-based vs. encoder-free models either change all three choices (Tong et al., 2026; Liu et al., 2026) or change capacity and placement together (Shukor et al., 2025). Second, comparisons with pretrained encoders control only multimodal training, treating encoder pretraining as free even though it adds substantial compute and data of its own (e.g., SigLIP 2 trains on 40B image–text samples; Tschannen et al., 2025), and several studies attribute the encoder's advantage to its pretraining (Lei et al., 2025; Diao et al., 2025).

In this work, we conduct a controlled study that unbundles the design choices of multimodal vision encoding one at a time (Figure 1): (i) *does the vision encoder need separate pretraining?* (ii) *does vision encoding need dedicated capacity?* and (iii) *does that capacity need to precede multimodal fusion?* We train every model entirely from scratch, language backbone included, on a common pool of data with one encoder architecture, one dense backbone architecture, and one optimization recipe. We evaluate models on downstream visual question-answering (VQA) and language benchmarks at backbone sizes from 0.6B to 6.4B. Because recipes spend compute in different stages, we compare them at matched *lifetime compute*: the total compute of all training stages, including any pretraining of the encoder or language model.

Our experiments answer these questions and test whether the answers hold beyond lifetime compute:

1. **Separate pretraining is optional** (§3). At matched lifetime compute, a from-scratch encoder is on par with encoders pretrained on the same data. Even if we *ignore upstream compute* (§6.1), more encoder pretraining improves recognition but not VQA.
2. **Dedicated vision capacity is key** (§4). An encoder-free model trails an encoder-based model on visual understanding, even when its backbone is enlarged by the encoder's parameter count. The encoder's advantage thus lies in capacity dedicated to vision rather than in total parameter count.
3. **Vision capacity can move in-fusion** (§5). Adding vision-only capacity to the backbone's earliest layers is comparable to adding a from-scratch pre-fusion encoder. Scaling laws show in-fusion tracking the encoder, whereas encoder-free trails it at every size (§5.4).
4. **The findings hold beyond lifetime compute** (§6). Starting from a pretrained 1.7B language model, in-fusion stays comparable to the from-scratch encoder while encoder-free falls further behind. In-fusion can also lower time to first token, since no separate encoder must run before the backbone.

Together, these findings show that multimodal models benefit from dedicated vision capacity, which however needs neither separate pretraining nor pre-fusion placement.

## 2 A controlled protocol for comparing multimodal recipes

We propose a protocol that varies the three design choices a vision encoder bundles, while holding fixed the data sources, optimization settings, compute, and backbone architecture. This isolates their effects when comparing current multimodal model recipes. Conclusions about these design choices cannot be drawn from comparing published models, because they differ in several factors at once—e.g., they build on different pretrained language models and vision encoders that each vary in training data and compute. We therefore use no external pretrained models: we train every component from scratch, with common data sources and training setups, and compare recipes at matched *lifetime compute*, the total compute of all training stages.

### 2.1 Recipes and the fusion boundary

All recipes differ only in how they encode visual input.

**Table 1 Unbundling the encoder's design choices.** A pretrained encoder bundles all three; in-fusion keeps only dedicated vision capacity.

| Recipe | Prior | Capacity | Pre-fusion |
|---|---|---|---|
| ◆ Pretrained encoder | ✓ | ✓ | ✓ |
| ● Scratch encoder | × | ✓ | ✓ |
| ■ Encoder-free | × | × | × |
| ▼ In-fusion | × | ✓ | × |

**Backbone.** Our backbones use an architecture modeled after Qwen3 (Yang et al., 2025): a dense, decoder-only, pre-norm Transformer (Vaswani et al., 2017) with RMSNorm (Zhang and Sennrich, 2019), RoPE (Su et al., 2024), SwiGLU feed-forward layers (Shazeer, 2020), and grouped-query attention (Ainslie et al., 2023) with query–key normalization (Dehghani et al., 2023) (Appendix A.1). Our main comparisons (Sections 3 to 5) use the Qwen3 1.7B and 4B configurations with dedicated vision capacity fixed at 0.4B parameters, and our scaling study (Section 5.4) uses a 0.6B–6.4B ladder that grows vision capacity in proportion to the backbone. Training uses autoregressive cross-entropy on text tokens; vision tokens precede the text they condition and have no loss. Each image's vision tokens attend to each other bidirectionally and text tokens attend causally to all earlier tokens, including vision tokens (Appendix A.4).

**Fusion boundary.** We describe how image patches reach the backbone relative to the *multimodal fusion boundary*, the first layer in which vision and text tokens can interact: vision-only computation before it is *pre-fusion* and computation inside the shared backbone is *in-fusion* (Figure 1 and Table 1). Scratch encoders (●) process image patches pre-fusion with a randomly initialized 27-layer, 0.4B-parameter Vision Transformer (Dosovitskiy et al., 2021; Alabdulmohsin et al., 2023) modeled after MoonViT-V2 (Kimi K3's encoder; Kimi Team, 2026b), whose output is linearly projected into the backbone. Pretrained encoders (◆) share this architecture, but we first pretrain them ourselves on our common data (Section 2.2) rather than taking external checkpoints. Encoder-free models (■) map patches directly into the backbone with a linear patch embedding (Adept AI, 2023; Gemma Team, 2026), followed by RMSNorm and a learned scalar gate (Qi et al., 2026; Li et al., 2026). In-fusion models (▼) instead add vision-only modules inside the backbone (Section 5). Every recipe trains on $224 \times 224$ images, read as 256 vision tokens using $14 \times 14$ patches. Instruction tuning for evaluation raises the resolution to $448 \times 448$ (1,024 vision tokens), as in PaliGemma's short high-resolution stage (Beyer et al., 2024), since OCR and chart reading needs fine detail (Tong et al., 2024a; Beyer, 2024).

### 2.2 Common data

Text-only sequences come from Nemotron-CC v2/v2.1 (NVIDIA, 2025b,a), and image–text sequences use MetaCLIP 1.2 pairs (English subset) (Xu et al., 2024). Encoder pretraining uses these pairs with their original alt-text, as in each objective's original setup (Section 3; e.g., CapPa, Tschannen et al., 2023), while multimodal training uses captions regenerated by Qwen2.5-VL-32B (Bai et al., 2025b) and shared by every recipe. Multimodal training mixes text-only and image–text sequences 70:30, so recipes with a pretrained encoder spend more lifetime compute on image–text data. This matches the language share Han et al. (2026) found optimal and gave the best visual understanding in our sweep at a small cost to text (Appendix A.4).

### 2.3 Lifetime compute

Multimodal models often build on separately pretrained components whose training can even exceed the multimodal stage. For example, SigLIP 2, a commonly used encoder, is trained on 40B image–text sam-

ples (Tschannen et al., 2025), which translates to 10 trillion image tokens, while language models are typically also trained on trillions of text tokens (Touvron et al., 2023; Yang et al., 2025). Counting only multimodal training therefore favors recipes that spend more compute in these earlier stages. To make such comparisons fair, we propose *lifetime compute*, the total training compute of every stage that produces the final model,

$$C_{\text{lifetime}} = \sum_{s \in \mathcal{S}} C_s, \tag{1}$$

where $\mathcal{S}$ is the set of training stages and $C_s$ is the training compute of stage $s$. Here, $\mathcal{S}$ contains multimodal training, any text-only training before it and, for pretrained encoders, encoder pretraining, but not instruction tuning. We report compute in FLOPs; because recipes differ in cost per token, matched compute means that they process different numbers of tokens. Appendix A.5 describes the compute accounting and converts each budget into token-equivalents, the number of tokens a text-only model of the same size would process with the same compute. Lifetime compute models a project that allocates one budget across every training stage, which lets us separate the contribution of each design choice. In practice, pretrained encoders are often available for free, so we also compare encoders with their pretraining compute not counted (Section 6.1).

### 2.4 Training schedule

To compare recipes at many training lengths from one run (Zhai et al., 2022), we use a warmup–stable–decay schedule (Hu et al., 2024; Wen et al., 2025; Hägele et al., 2024), where the learning rate stays constant after warmup and any checkpoint of that run can be decayed to 5% of its peak over the final 10% of training (Appendix A.3). Our main comparisons span compute budgets from $6.5 \times 10^{20}$ to $2.9 \times 10^{22}$ FLOPs, or 56B to 1.11T token-equivalents (Table 12). We index budgets by tokens per parameter (TPP): token-equivalents divided by the backbone's non-embedding parameters, i.e., all parameters except the tied token embedding (1.4B at 1.7B and 3.6B at 4B; backbones are named by their total parameters). Budgets reach 788 TPP at 1.7B and 306 TPP at 4B. All recipes share one peak learning rate per backbone size, scaled with the inverse square root of its total parameter count ($3 \times 10^{-4}$ at 1.7B and $2 \times 10^{-4}$ at 4B). At 1.7B, the encoder's lead over encoder-free and parity with in-fusion hold from half to twice that learning rate (Appendix F.2).

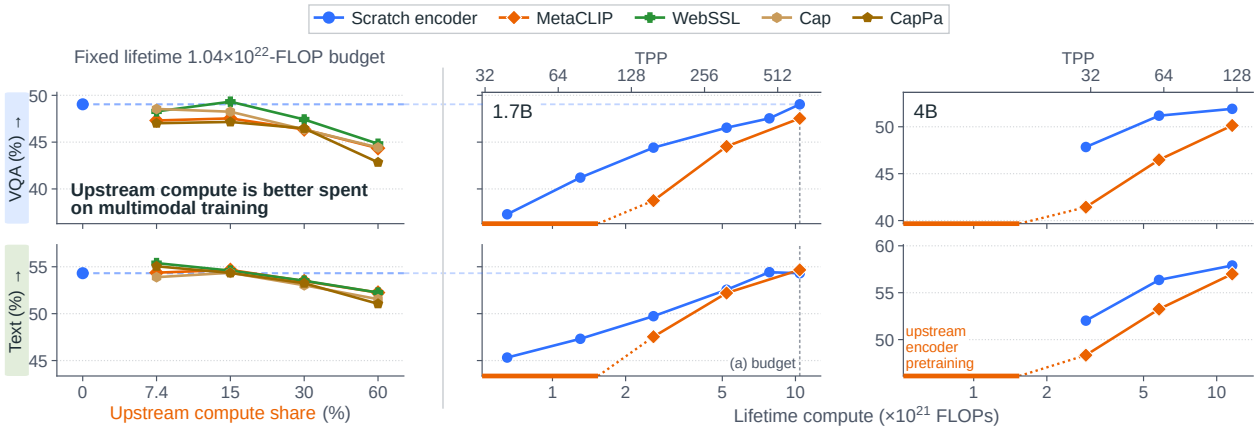
### 2.5 Evaluation

We evaluate recipes on text and visual understanding benchmarks and report the following metrics. **Avg. Text** averages four category means over 12 language benchmarks: commonsense reasoning (COPA, HellaSwag, PIQA, Social IQa, and WinoGrande (Roemmele et al., 2011; Zellers et al., 2019; Bisk et al., 2020; Sap et al., 2019; Sakaguchi et al., 2020)), science question answering (ARC-Easy, ARC-Challenge, and OpenBookQA (Clark et al., 2018; Mihaylov et al., 2018)), broad knowledge (MMLU (Hendrycks et al., 2021)), and reading comprehension (RACE-middle, RACE-high, and BoolQ (Lai et al., 2017; Clark et al., 2019)). **Avg. VQA** averages the general, knowledge, OCR and chart, and vision-centric category means of the 16 Cambrian visual question-answering benchmarks (Tong et al., 2024a), after instruction tuning each checkpoint for three epochs on Cambrian-7M with $448 \times 448$ images. A text-only model at the same lifetime compute serves as a reference for Avg. Text (Appendix B.3). Sections 4 and 5.4 also compare held-out caption loss on images never seen in training (Appendix B.4). Appendix B details the benchmarks, and Appendix I gives every benchmark score.

## 3 Does the vision encoder need separate pretraining?

The first design choice a vision encoder bundles is a visual prior imported through separate pretraining, which recent models (Kimi Team, 2026b) forgo by training the encoder from scratch with the language model. Under a fixed lifetime compute budget, encoder pretraining directly competes with multimodal training for FLOPs. Whether the prior is needed also shapes the remainder of our study, because encoder-free architectures cannot easily import one. We test whether the prior helps by comparing encoders pretrained with different objectives against the same encoder trained from scratch with the backbone.

**Setup.** We pretrain a separate encoder with each of four objectives and the same compute: contrastive image–text alignment (MetaCLIP; Xu et al., 2024), self-supervised learning (WebSSL; Fan et al., 2025), and captioning (Cap and CapPa; Tschannen et al., 2023). Pretrained encoders are updated during multimodal




**(a)** 1.7B backbone **(b)** From-scratch versus MetaCLIP pretrained

**Figure 2 Encoder pretraining is not beneficial at matched lifetime compute.** **(a)** At 1.7B and $1.04 \times 10^{22}$-FLOP lifetime budget, spending a larger share of compute on encoder pretraining is comparable to the from-scratch encoder (horizontal **dashed blue**) up to 15% and lowers performance beyond it. **(b)** MetaCLIP pretraining uses $1.54 \times 10^{21}$ FLOPs at every backbone size (**orange floor**); the from-scratch encoder matches or exceeds MetaCLIP at every budget. The vertical **dashed gray** line at 1.7B marks the budget of **(a)** and the top axis gives the from-scratch encoder's TPP.

training and compared with an encoder trained from scratch at matched lifetime compute; we also test a frozen MetaCLIP encoder. MetaCLIP, an instance of widely used CLIP-style pretraining (Radford et al., 2021; Zhai et al., 2023; Tschannen et al., 2025), serves as the pretrained reference in the remaining experiments. Appendix C.1 details the pretraining objectives and Appendix A.1 the encoder architecture.

**A from-scratch encoder is on par with pretrained ones.** With a 1.7B backbone and a $1.04 \times 10^{22}$ FLOP budget, all four pretrained encoders are comparable to the from-scratch encoder when pretraining takes 7.4% or 15% of the budget (e.g., Cap sees 1B or 2B image–text examples). Larger upstream shares hurt multimodal performance: at a 60% share, both averages fall clearly below the from-scratch encoder (Figure 2a). Across backbone sizes, the from-scratch encoder matches or exceeds MetaCLIP at every budget (Figure 2b). Appendix C gives the per-objective results, including frozen encoders, the full pretraining sweep, the results at 4B, and compositional image–text matching on SugarCrepe (Hsieh et al., 2023).

Encoder pretraining does not help even when its compute is free (Section 6.1). More pretraining raises ImageNet accuracy but not Avg. VQA, and pretrained encoders are only comparable to the from-scratch encoder at matched multimodal compute. Delaying vision also hurts, as training on text alone for the first 25–75% of training, with the total number of image–text examples held fixed, clearly lowers Avg. VQA (Table 11), consistent with the vision laziness that Han et al. (2026) observe after text-only training.

**Takeaway 1: Encoder pretraining offers little advantage.** At matched lifetime compute, an encoder trained from scratch together with the backbone is on par with encoders pretrained on the same data.

## 4 Does vision encoding need dedicated capacity?

The from-scratch encoder of Section 3 forgoes the imported prior but keeps the second design choice: dedicated vision capacity. *Encoder-free* models drop this capacity entirely and project image patches directly into the shared backbone (Adept AI, 2023; Diao et al., 2024; Liu et al., 2026; Gemma Team, 2026; Thinking Machines Lab, 2026a). Dropping the encoder, however, also removes a substantial number of parameters and changes how raw patches enter the backbone, so a gap between the two recipes could reflect fewer total parameters, a weak patch embedding, or missing dedicated capacity. In this section, we test whether visual understanding needs this dedicated capacity, leaving the study of its precise placement to Section 5.

**Setup.** We compare the from-scratch encoder (Section 3) with the encoder-free recipe (Section 2.1), which maps each patch into the backbone with a linear patch embedding followed by RMSNorm and a learned gate. As in every recipe, vision tokens attend to one another bidirectionally in the backbone (Section 2.1). Both recipes share common backbones, data, and lifetime compute budgets, and we compare them at 1.7B and 4B.

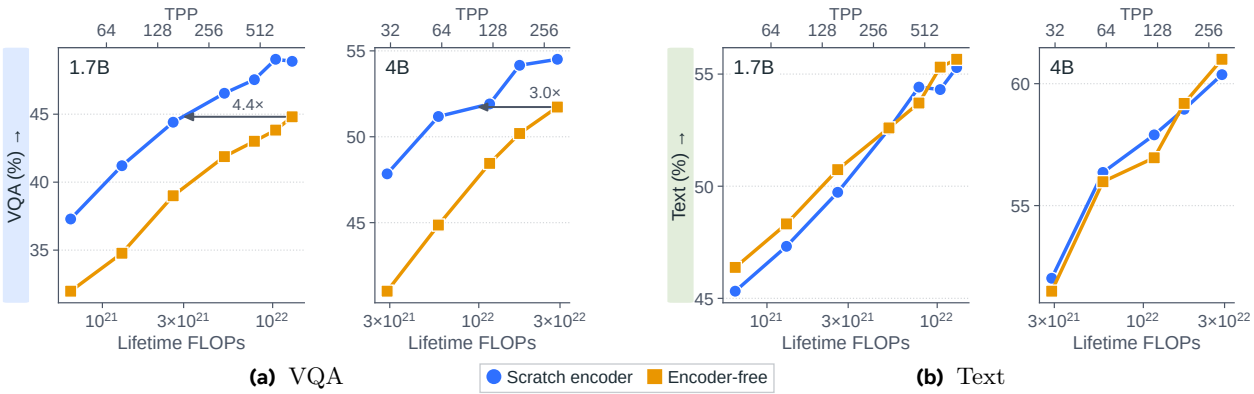



**Figure 3 Encoder-free trails the encoder on Avg. VQA, while Avg. Text stays comparable.** Each checkpoint is decayed, and arrows mark the compute factor encoder-free needs to match the encoder. Appendix D.1 gives per-category results.

**Encoder-free trails encoder on visual understanding.** Figure 3 shows that Avg. VQA clearly suffers from a missing encoder, across scales and budgets, while Avg. Text stays comparable. This also holds when scaling the encoder's size (Section 5.4). This result differs from Shukor et al. (2025), who find that encoder-free models match or slightly outperform encoder-based ones, mainly at smaller sizes. We identify two choices in their setup that work against the encoder (Appendix D.4). First, their encoder's learning rate may have been too small. Second, they fit their scaling laws to an average of text, interleaved, and caption losses, which drowns out the signal from vision. In-distribution held-out caption loss instead shows the encoder-free deficit and tracks Avg. VQA across recipes, sizes, and budgets (Figure 18), whereas their averaged loss hides it.

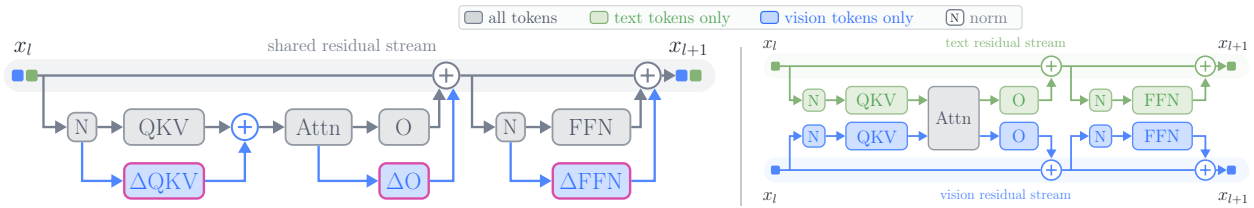
**Matching total parameters does not overcome encoder-free's deficit.** We next widen or deepen the encoder-free backbone by 0.4B parameters to match the vision parameters added by the encoder (Table 2; 4B in Table 19). None of the variants comes close to closing the gap on Avg. VQA, though all improve Avg. Text, since text tokens also use the added parameters. The remaining gap in visual understanding is largest on OCR and chart benchmarks. The same parameters are thus more useful as a vision encoder. As none of these variants improves Avg. VQA, encoder-free hereafter keeps the plain backbone shared across all recipes.

**Table 2 Matching total parameters does not close the encoder-free gap.** All rows use a 1.7B backbone and $2.6\times10^{21}$ lifetime FLOPs. Indented rows (+) add the encoder's 0.4B parameters to the backbone. Bold: within one point of the best.

| Architecture | Added params (B) | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Variant | Vision-only | Shared | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| • Scratch encoder | 0.4 | – | **44.4** | **57.2** | **35.7** | **38.2** | **46.5** | 49.7 | **63.9** | 45.0 | 36.7 | 53.4 |
| ▪ Encoder-free | – | – | 39.0 | 50.7 | **35.0** | 25.2 | 45.0 | **50.7** | **64.1** | **45.8** | 38.2 | 54.8 |
| + wider | – | 0.4 | 38.6 | 48.5 | 34.2 | 29.3 | 42.2 | **51.3** | **64.0** | **46.5** | 38.0 | **56.6** |
| + deeper | – | 0.4 | 38.7 | 50.0 | 33.2 | 26.3 | 45.4 | **51.2** | **64.2** | **45.8** | 39.1 | **55.9** |
| + wider and deeper | – | 0.4 | 38.8 | 51.6 | **34.8** | 25.1 | 43.7 | **51.4** | **64.0** | **45.7** | **40.3** | **55.9** |
| *Richer patch embedding* | | | | | | | | | | | | |
| Two-layer projection | <0.01 | – | 40.5 | 50.8 | 33.9 | 30.5 | **46.7** | 49.7 | **63.2** | **45.6** | 36.3 | 53.5 |
| Hierarchical MLP | 0.04 | – | 35.9 | 50.8 | 33.1 | 15.3 | 44.5 | 50.3 | **63.5** | 45.1 | **39.3** | 53.4 |

**A richer patch embedding does not close the gap.** We also study replacing the encoder-free linear patch embedding with two richer ones: a normalized two-layer projection inspired by Gemma 4 (Gemma Team, 2026), and a four-layer hierarchical MLP (Touvron et al., 2022), as in Inkling (Thinking Machines Lab, 2026a). Trained on $224\times224$ images, neither comes close to the encoder (Table 2), though the hierarchical MLP matches it when compressing $640\times640$ images into the same 256 tokens (Appendix D.3). Subsequent encoder-free experiments keep the default patch embedding, so dedicated capacity is placed deliberately.

**Takeaway 2: Visual understanding benefits from dedicated vision capacity.** Encoder-free models trail the from-scratch encoder on Avg. VQA. Adding backbone capacity with the same encoder parameter budget does not close the gap.




**(a)** **In-fusion**: vision modules add to a shared residual stream **(b)** **MoT**: separate streams, joint attention

**Figure 4 In-fusion vision capacity adds to the shared backbone instead of replacing it.** **(a)** All text ■ and vision tokens ■ are processed by shared modules (**gray**); vision tokens *also* pass through vision-only modules (**blue** with **pink** outline) that add to the shared outputs; in-fusion (FFN-only) omits the vision attention projections (ΔQKV, ΔO). **(b)** MoT uses entirely separate modules for text and vision, so the modality streams interact only through attention.

# 5 Does dedicated capacity need to precede multimodal fusion?

Section 4 shows that visual understanding benefits from dedicated capacity. We now ask whether that capacity must precede multimodal fusion. The encoder's parameters are both *dedicated*, used only by vision tokens, and *pre-fusion*, processing vision tokens before they interact with text. We test whether pre-fusion placement is needed by moving the encoder's parameter budget in-fusion, into vision-only modules inside the shared backbone (Section 5.1). Section 5.2 studies where this capacity should sit, which tokens should use it, and which modules it adds. Sections 5.3 and 5.4 then compare it with the encoder over training and with scale.

## 5.1 Moving the encoder's capacity inside the backbone

In-fusion vision capacity adds vision-only modules to selected backbone layers. In each of these layers, the attention projections and the feed-forward network (FFN) keep their shared weights and add vision-only counterparts, whose outputs add to the shared outputs after each module (Figure 4). Each vision-only module thus acts as an additive expert routed by modality: vision tokens use it alongside the shared weights. Unlike mixture-of-transformers (MoT; Liang et al., 2025), which gives text and vision tokens entirely separate parameters, in-fusion still passes every token through the shared parameters. Section 6.2 compares the two when starting from pretrained language models.

For token $i$ with vision indicator $m_i \in \{0, 1\}$, each attention projection $W_P$ gains a vision-only counterpart $\Delta W_P$ that applies only to vision tokens,

$$P(\mathbf{z}_i) = W_P\,\mathbf{z}_i + m_i\,\Delta W_P\,\mathbf{z}_i, \qquad P \in \{Q, K, V, O\}, \tag{2}$$

where $\mathbf{z}_i$ is the projection's input (the normalized hidden state for $Q$, $K$, $V$ and the attention output for $O$). Attention thus remains joint across vision and text tokens. The FFN gains a vision-only branch,

$$\mathbf{h}_i' = \mathbf{h}_i + F(N(\mathbf{h}_i)) + m_i\,\Delta F(N(\mathbf{h}_i))\,, \tag{3}$$

where $F$ is the shared FFN, $N$ is its normalization, and $\Delta F$ is a vision-only FFN of the same form that reads the same normalized input (Appendix E.1). The vision-only projections $\Delta W_P$ and the output projection of $\Delta F$ are zero-initialized, so training starts from the shared model. Because these modules act only when $m_i = 1$, text-only inputs are processed exactly as they would have been by the shared backbone.

We use two variants: "in-fusion", our default, adds vision FFN branches of smaller hidden width and vision-only counterparts of all four attention projections, while "in-fusion (FFN-only)" adds only vision FFN branches. We size both variants to match the encoder's vision-only parameter budget (Table 23), so comparisons with the encoder vary only where dedicated capacity is placed. Existing vision-specific architectures do not fit this comparison: they either duplicate vision weights that scale with the backbone (Wang et al., 2024; Liang et al., 2025) or use vision adapters that rely on an encoder (Dong et al., 2024; Wang et al., 2025).

**Table 3 Added in-fusion capacity is most effective when placed early and reserved for vision tokens.** Every variant adds 0.4B parameters to the 1.7B backbone. Schematics show backbone layers from early to late; gray serves all tokens, blue only vision tokens, and wider layers are taller. Highlighted: our default placement. Bold: within one point of the best.

| Architecture | | | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Variant | Design | Used by | Placement | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| Scratch encoder | | Vision | Pre-fusion | **44.4** | **57.2** | 35.7 | **38.2** | 46.5 | 49.7 | **63.9** | 45.0 | 36.7 | 53.4 |
| Encoder-free | | – | – | 39.0 | 50.7 | 35.0 | 25.2 | 45.0 | **50.7** | **64.1** | **45.8** | 38.2 | **54.8** |
| *Where to add (vision tokens)* | | | | | | | | | | | | | |
| In-fusion (FFN-only) | | Vision | Every layer (narrow) | 39.4 | 46.4 | 33.4 | 31.3 | 46.6 | **50.7** | **64.5** | **45.6** | 39.8 | 53.1 |
| In-fusion (FFN-only) | | Vision | Spread (8 of 28) | 37.6 | 46.7 | 33.1 | 28.1 | 42.4 | 50.0 | **63.9** | **45.6** | 40.0 | 50.5 |
| In-fusion (FFN-only) | | Vision | Layers 21–28 | 35.1 | 42.6 | 31.4 | 23.3 | 43.3 | 49.8 | **64.1** | 44.6 | 38.5 | 52.1 |
| In-fusion (FFN-only) | | Vision | Layers 1–8 | **45.0** | **57.0** | **36.7** | 36.0 | **50.4** | **50.4** | 63.0 | **45.3** | **40.6** | 52.9 |
| *All-token controls (layers 1–8)* | | | | | | | | | | | | | |
| Shared FFN widening | | All | Layers 1–8 | 40.1 | 51.5 | 34.1 | 29.6 | 45.1 | **51.2** | **64.5** | **45.6** | **40.6** | **54.2** |
| Separate FFN branch | | All | Layers 1–8 | 42.3 | 54.1 | **37.4** | 30.8 | 46.7 | **51.2** | **64.8** | **46.3** | **41.6** | 52.0 |

## 5.2 Where and how to add in-fusion capacity

We compare in-fusion designs that reallocate the encoder's 0.4B vision parameters within the backbone.

**Placement of capacity within fusion.** We study placement with the FFN-only variant, because FFN branches can be narrowed to fit any placement. Full-rank vision-only attention projections cannot, because adding them to every layer would take 88% of the 0.4B budget at 1.7B and exceed it at 4B. With the budget fixed, we compare four placements (Table 3): a narrow branch in every layer, or eight wider branches spread evenly across depth, in the last eight layers, or in the first eight layers. The first three variants do no better than the encoder-free model on Avg. VQA, whereas placing the modules in the first eight layers matches the scratch encoder. Early placement also helps most on OCR and chart questions, which depend on fine visual detail, whereas late placement does not. Dedicated in-fusion capacity is therefore most useful where the backbone first processes vision tokens, similar to an encoder. We hypothesize that vision tokens benefit most from vision-only processing while they are still close to raw patch features. Only early placement concentrates the budget there, whereas the every-layer and spread placements dilute it across the backbone. Layerwise representation similarity shows that the in-fusion models change vision tokens mostly within the first eight blocks, where their vision-only modules sit, and little afterward, while the encoder-free model keeps changing them through shared blocks 4–13 (Appendix E.4). Therefore, we use the first eight layers from here on.

**Reserving the capacity for vision tokens matters.** To check that the gain comes from reserving this capacity for vision tokens, we also give the same parameters in the first eight layers to all tokens (Table 3). Widening the shared FFNs barely improves over the encoder-free model. A separate branch used by all tokens recovers some of the vision-only branch's gain, but still trails it even with twice the training (Table 24). The all-token variants slightly improve on Avg. Text, because text also uses the added parameters. We hypothesize that a vision-only branch gives vision tokens processing of their own without competing with text for the same parameters, while both still share the backbone's attention and FFNs.

**Adding vision-only attention projections.** With the capacity confined to the first eight layers, the QKVO attention projections fit within the budget (0.1B at 1.7B), so "in-fusion" adds them alongside narrower FFN branches. It is comparable to FFN-only at 1.7B and slightly ahead at every 4B budget (Figure 5), and its vision-only attention projections help on their own at 4B (Appendix E.2), so we use this variant as our default for the remainder of the paper.

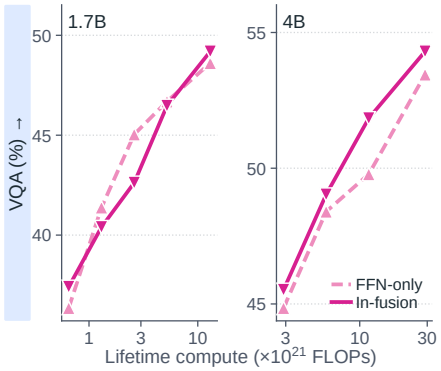



**Figure 5 In-fusion is comparable to FFN-only at 1.7B and slightly ahead at every 4B budget.**

## 5.3 Can in-fusion capacity match a pre-fusion encoder?

With the same dedicated budget on either side of the multimodal fusion boundary, we compare in-fusion with a scratch encoder at matched lifetime compute over training at 1.7B and 4B (Figure 6). In-fusion is comparable to the encoder throughout. Like the encoder,

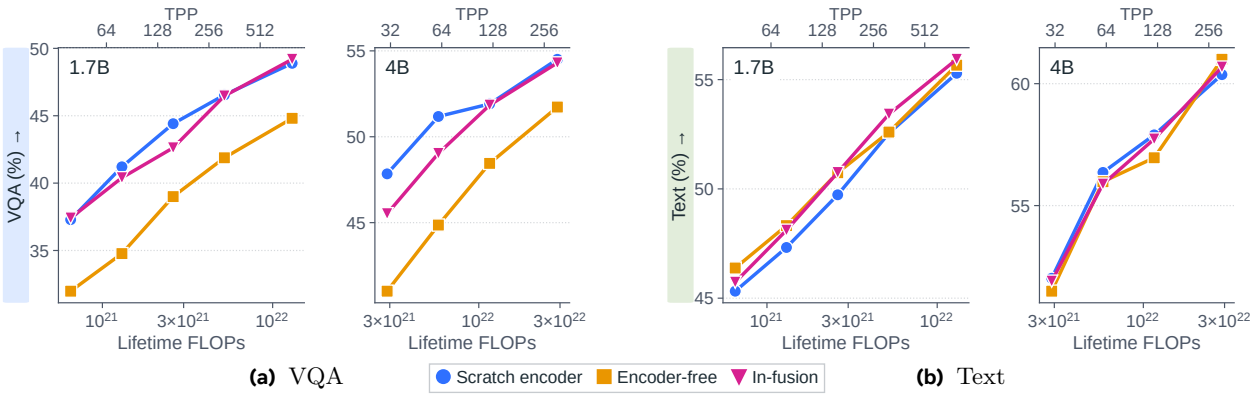



**Figure 6 In-fusion is comparable to the pre-fusion encoder.** Encoder-free is shown for reference. Table 26 gives per-category results at the longest budget.

in-fusion clearly leads encoder-free at both sizes; Section 5.4 compares the recipes across backbone sizes. Part of the in-fusion budget sits in vision-only attention projections, which help on their own in additional 4B ablations (Appendix E.2). Dedicated vision capacity therefore does not need to precede fusion to benefit visual understanding. In-fusion can also lower time to first token for image requests, since no encoder runs before the backbone (Section 6.3).

## 5.4 Results hold with scale

So far, we have compared the recipes at 1.7B and 4B with 0.4B of dedicated vision capacity. To see how they behave with scale, we train them on a ladder of five backbone sizes from 0.6B to 6.4B (setup in Appendix G). The dedicated vision capacity grows with the backbone, keeping its share of parameters from the 1.7B model, as in jointly scaled models (Chen et al., 2023; Tian et al., 2025). For each recipe, we fit Avg. VQA, caption loss, and text loss as functions of model size $N$ and token-equivalents $D$ on all of its ladder runs. At every size, encoder-free needs approximately three times the encoder's compute to match it on Avg. VQA and caption loss, while in-fusion tracks the encoder (Figure 7). On text loss and Avg. Text, the three recipes are comparable (Tables 29 and 30). Taken together, the findings of Sections 4 and 5 hold with scale.

**Takeaway 3: Dedicated vision capacity can move into the backbone's earliest layers.** Placed in the earliest layers, vision-only modules with the encoder's budget are comparable to a from-scratch encoder at every backbone size.

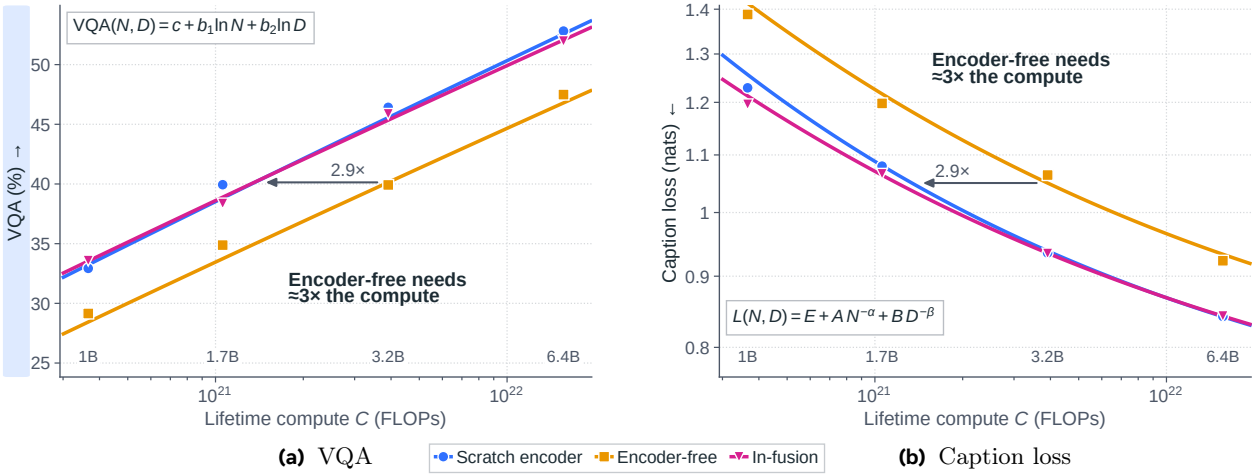



**Figure 7 Encoder-free trails the encoder at every size, while in-fusion tracks it.** Points and curves shown are 64 TPP. Each recipe's log-linear VQA fit **(a)** and power-law loss fit **(b)** use all of its runs, across sizes and TPPs (Appendix G).

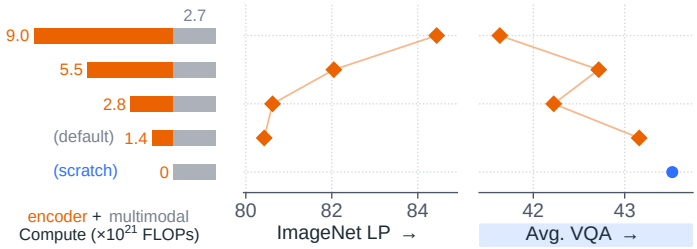



**Figure 8 Better encoder ⇏ better VQA, even if encoder pretraining is free.** We compare MetaCLIP encoders pretrained with 1.4–$9.0 \times 10^{21}$ FLOPs (**orange**) followed by identical $2.7 \times 10^{21}$-FLOP multimodal stages (**gray**) with a 1.7B backbone. ImageNet linear-probe accuracy is measured after pretraining, and VQA after the multimodal stage. **Blue** shows a scratch encoder (no pretraining).

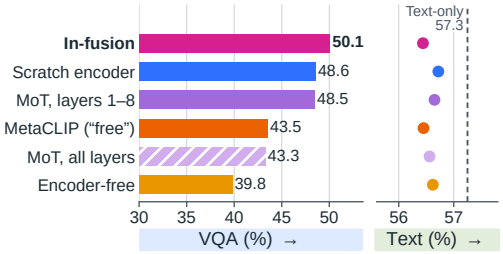



**Figure 9 In midtraining, in-fusion, early-layer MoT, and the from-scratch encoder are comparable on VQA.** All-layer MoT, pretrained MetaCLIP encoder, and encoder-free trail behind. Each recipe continues from a 1.7B text-only model at 1T token-equivalents to 1.33T.

# 6 Vision encoding beyond lifetime compute

Our main investigations (Sections 3 to 5) are deliberately controlled: they train every component from scratch and charge each stage's compute to one lifetime budget, which enables us to isolate the three design choices of a vision encoder. In practice, however, multimodal models are often built from separately pretrained components: a language model and, usually, a vision encoder. Inference speed for serving also shapes these design decisions. We find that a "free" pretrained encoder still does not help a backbone trained from scratch (Section 6.1), and that instead midtraining a 1.7B text-only model leaves the comparisons of Sections 3 to 5 unchanged (Section 6.2). Finally, in-fusion can lower the time to first token for image requests (Section 6.3).

## 6.1 Zero-cost pretrained encoders

Section 3 charges encoder pretraining to the lifetime budget, but in practice a pretrained encoder is often available for free. Here, we hold multimodal training fixed at $2.7 \times 10^{21}$ FLOPs and vary MetaCLIP pretraining from $1.4 \times 10^{21}$ to $9.0 \times 10^{21}$ FLOPs. This raises the encoder's ImageNet linear-probe accuracy before multimodal training from 80.4% to 84.4% (Figure 8; Appendix C.3), yet Avg. VQA is highest after the shortest pretraining, and an encoder trained from scratch with the same multimodal training matches it. Thus, even when pretraining is free, more of it does not help.

## 6.2 Midtraining from pretrained language models

Many multimodal models add vision to a language model pretrained on text alone (Alayrac et al., 2022; Liu et al., 2023; Tong et al., 2024a; Bai et al., 2025a). We call this stage of continued training on multimodal data *midtraining*. We test whether the comparisons of Sections 3 to 5 hold in this setting. We pretrain our own 1.7B language model on 1T tokens of the same text data and then midtrain it to 1.33T token-equivalents with each recipe at equal compute (Figure 9). Following DeepSeek-VL (Lu et al., 2024a), midtraining keeps 70% text data; as in Section 6.1, MetaCLIP's encoder pretraining is not counted. Besides the four recipes, we compare MoT (Section 5.1, initialized from the language model) and a variant that copies only its first eight layers, matching the encoder's 0.4B vision-only parameters.

In-fusion, MoT layers 1–8, and the from-scratch encoder perform comparably, and all lead encoder-free. The full MoT model falls behind despite 3.5 times as many vision-only parameters—adding vision-only copies of the later layers to MoT layers 1–8 hurts, consistent with late placement not helping in Section 5.2. A free MetaCLIP encoder, which matches the from-scratch encoder when the backbone is also trained from scratch (Section 6.1), trails it here, mostly on OCR, chart, and vision-centric questions. Tuning each encoder's learning rate does not change these conclusions (Appendix H.1). We hypothesize that the from-scratch encoder learns features suited to the pretrained language model from its first update, whereas MetaCLIP's contrastive features, which miss fine visual detail (Tong et al., 2024b), must be reshaped by a language model that already leans on text. Every recipe keeps Avg. Text within a point of a text-only control. Thus, the conclusions from our controlled experiments (Sections 3 to 5) carry over to midtraining, where a free pretrained encoder trails the from-scratch encoder, encoder-free trails on VQA, and in-fusion is comparable to the pre-fusion encoder.

### 6.3 Training and inference speed

A separate encoder must finish processing each image before the backbone can start, which adds a serial step to inference. Following the inference benchmark of Mono-InternVL (Luo et al., 2025b), we measure time to first token (TTFT) on one H200 with 16 concurrent requests, each with one 448-pixel image and 256 text tokens (Appendix H.3). At 1.7B with 0.4B vision-only parameters, in-fusion and its FFN-only variant are clearly faster than the encoder (Figure 10) at comparable Avg. VQA (Table 26). MoT, which copies the first $k$ backbone layers for vision tokens, is about as fast as in-fusion up to 0.4B vision-only parameters, but its added TTFT grows faster with the budget, while staying below the encoder's TTFT. Encoder-free is fastest but clearly trails on Avg. VQA (Table 26). In-fusion's TTFT advantage over the encoder grows with the vision-only budget (Figure 10), which scales with the backbone (Section 5.4); at a fixed budget, vision adds nearly the same time at 1.7B and 4B (Table 36). Earlier encoder-free models report larger TTFT reductions, up to 69% (Luo et al., 2025b,a), although their gains also reflect other efficiency changes. Inference benefits also depend on deployment details. Under continuous batching of mixed text and image requests, in-fusion's vision-only modules run in the same forward passes as text-only requests, whereas a separate encoder adds latency only to image requests. Finally, both in-fusion variants train as fast as encoder-free, whereas the encoder recipe is 3% slower at 4B (Table 35).

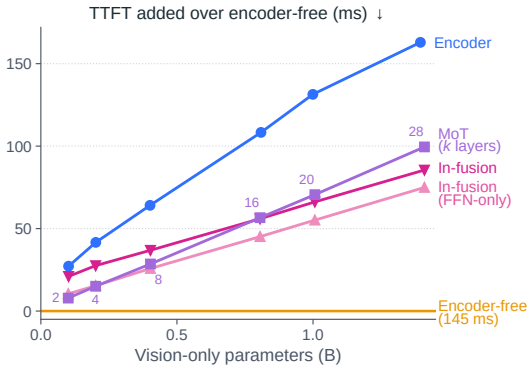



**Figure 10 In-fusion serves images faster than the encoder at every vision parameter budget.** Designs are parameter-matched on the 1.7B backbone.

**Takeaway 4: Our findings hold beyond lifetime compute, and in-fusion can be faster to serve.** A free pretrained encoder still does not help, the comparisons carry over to midtraining, and in-fusion lowers TTFT at comparable VQA.

## 7 Related work

**Multimodal model recipes.** Most multimodal models connect a separately pretrained vision encoder (Radford et al., 2021; Tschannen et al., 2025; Bolya et al., 2025), kept frozen (Liu et al., 2023; Li et al., 2023b; Muse Glimmer Team, 2026) or updated (Beyer et al., 2024; Steiner et al., 2024; Bai et al., 2025a), to a pretrained language model. More recent models train the encoder from scratch with the language model (Kimi Team, 2026b; Tian et al., 2025), embed image patches without an encoder (Adept AI, 2023; Diao et al., 2026; Liu et al., 2026; Gemma Team, 2026; Thinking Machines Lab, 2026a), or quantize images with a separately trained tokenizer (Chameleon Team, 2024; Wang et al., 2026). Inkling was released as "encoder-free", yet its successor, Inkling 2 Small, uses a 292M-parameter vision encoder (Thinking Machines Lab, 2026a,b). Because these models also differ in data, pretrained components, and compute, comparing them directly does not isolate a single design choice. We therefore train every stage of each recipe class from scratch on common data sources and compare the recipes at matched lifetime compute (Section 2).

**Controlled studies.** Prior controlled studies of encoder pretraining typically start from a pretrained language model, count only multimodal training, and compare encoder pretraining objectives and data (Karamcheti et al., 2024; McKinzie et al., 2024; Tong et al., 2024a; Laurençon et al., 2024; Fan et al., 2025; Li et al., 2025) rather than studying whether separate pretraining is needed at all. The few works that compare pretrained and from-scratch encoders vary in their findings, possibly because they control neither the encoder's pretraining data nor lifetime compute (Tian et al., 2025; Kimi Team, 2026b). Studies raise a similar question for language pretraining, finding that adding vision before text-only pretraining ends, rather than after it, improves visual understanding (Keh et al., 2025; Han et al., 2026; Kimi Team, 2026a), as in our delayed-vision experiments (Table 11). Encoder pretraining also confounds encoder vs. encoder-free comparisons, which typically pit a pretrained encoder against an encoder-free model (Beyer et al., 2024; Diao et al., 2024; Lei et al., 2025; Diao et al., 2025; Tong et al., 2026; Liu et al., 2026; Chen et al., 2026). Such comparisons remove separate pretraining, dedicated capacity, and pre-fusion placement at once, and often attribute the encoder's advantage to pretraining data they do not control. Shukor et al. (2025), who train both designs from scratch, find that encoder-free models match or slightly outperform an encoder. In contrast, we find that the encoder leads on VQA and held-out caption loss, which we trace to their setup (Appendix D.4).

**Dedicated vision capacity.** Testing whether dedicated capacity must precede fusion requires moving the encoder's capacity inside the backbone, which the encoder vs. encoder-free comparisons cannot accomplish. Prior works on fusion depth come closer, varying where modalities begin to interact (Nagrani et al., 2021; Hao et al., 2026) or where encoder features enter the backbone (Meng et al., 2024), but none moves a fixed amount of vision-only capacity into layers that vision and text share. Other multimodal architectures add vision-specific parameters to the backbone, but not in a form that fits this comparison. Modality-expert designs duplicate the backbone's weights for vision tokens, typically in every layer, and route vision tokens through these copies instead of the shared weights (Bao et al., 2022; Lin et al., 2024; Liang et al., 2025). Their vision-only capacity is thus set by the backbone's size and spread over its depth rather than held at an encoder's budget early in the model, and most also build on a pretrained language model (Wang et al., 2024; Luo et al., 2025b; Tian et al., 2025; Diao et al., 2025; Shi et al., 2025). Adapter architectures fine-tune a pretrained model through small parallel or low-rank modules shared by all tokens (He et al., 2022; Chen et al., 2022; Hu et al., 2022), and their multimodal variants keep an encoder alongside the adapters (Dong et al., 2024; Shapourian et al., 2026) or distill from one (Wang et al., 2025). The encoders in these works still supply pre-fusion capacity or a pretrained prior, the two factors our comparison aims to remove. We instead give vision tokens their own attention projections and feed-forward branches in selected backbone layers. Their outputs add to the shared ones and together match an encoder's parameter budget, so only the placement of this capacity changes (Section 5). Placed in the first eight layers, in-fusion capacity is comparable to a pre-fusion encoder on VQA, so dedicated vision capacity helps whether it sits pre-fusion or in-fusion.

## 8 Discussion

With multimodal models revisiting the vision encoder, we asked which of its design choices are still beneficial. We proposed unbundling the encoder into three choices: a visual prior from separate pretraining, dedicated vision capacity, and pre-fusion placement of that capacity. We tested each by training every recipe from scratch on common data, with one architecture and optimization setup, and comparing recipes on text and visual understanding benchmarks at matched lifetime compute.

**Takeaways from unbundling.** Of the three choices an encoder bundles, only dedicated vision capacity maintains its advantage. Separate pretraining is unnecessary: an encoder trained jointly with the backbone is on par with pretrained ones, and longer pretraining improves recognition but not VQA, even when it is free (Sections 3 and 6.1). Dedicated capacity matters: with the encoder's parameters added to their shared backbone, encoder-free models gain on Avg. Text but still trail the encoder on Avg. VQA (Section 4), most on OCR and chart questions (Table 2). This capacity belongs early, but it need not precede the backbone: vision-only modules in the first backbone layers are comparable to a pre-fusion encoder (Section 5). For recipes that drop the encoder, this suggests reallocating its parameters as vision-only capacity in the earliest layers rather than as shared parameters.

**Limitations and future work.** To keep comparisons controlled, we pretrain encoders on the same images as multimodal training, so our results do not rule out a benefit from released encoders trained on additional data. Encoder pretraining uses the images' original alt-text, as in standard image–text pretraining; pretraining on our regenerated captions instead changes little (Table 9). Our experiments are on dense backbones with up to 6.4B parameters; the most compelling next step is extending them to sparse mixture-of-experts backbones, whose experts could naturally host in-fusion's modality-routed capacity. Multimodal training also uses $224 \times 224$ MetaCLIP images, leaving higher-resolution training and more text-rich data to future work. The text–image data mixture was selected with the from-scratch encoder at 1.7B, so mixtures tuned for each recipe remain underexplored (Appendix A.4). Finally, how architecture interacts with interleaved image–text and video data remains open.

**Conclusion.** We presented a controlled study that disentangles the design choices behind how multimodal models process visual input, studying each choice in isolation instead of as part of a bundled recipe. Rather than assembling multimodal models from components optimized for different objectives, our findings favor training them end-to-end as one system.

## Acknowledgments

We thank Jack Lu, Philippe Hansen-Estruch, Fei Zhang, Lili Yu, Shengbang Tong, John Nguyen, and David Fan for helpful discussions and feedback.

## References

Adept AI. Fuyu-8B: A multimodal architecture for AI agents, 2023. https://www.adept.ai/blog/fuyu-8b/.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023.

Ibrahim M. Alabdulmohsin, Xiaohua Zhai, Alexander Kolesnikov, and Lucas Beyer. Getting ViT in shape: Scaling laws for compute-optimal model design. In *NeurIPS*, 2023.

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a visual language model for few-shot learning. In *NeurIPS*, 2022.

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, CK Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *ASPLOS*, 2024.

Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C. Lawrence Zitnick, and Devi Parikh. VQA: Visual question answering. In *ICCV*, 2015.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025a.

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025b.

Hangbo Bao, Wenhui Wang, Li Dong, Qiang Liu, Owais Khan Mohammed, Kriti Aggarwal, Subhojit Som, Songhao Piao, and Furu Wei. VLMo: Unified vision-language pre-training with mixture-of-modality-experts. In *NeurIPS*, 2022.

Lucas Beyer. On the speed of ViTs and CNNs, 2024. https://lb.eyer.be/a/vit_cnn_speed.html.

Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, Thomas Unterthiner, Daniel Keysers, Skanda Koppula, Fangyu Liu, Adam Grycner, Alexey Gritsenko, Neil Houlsby, Manoj Kumar, Keran Rong, Julian Eisenschlos, Rishabh Kabra, Matthias Bauer, Matko Bošnjak, Xi Chen, Matthias Minderer, Paul Voigtlaender, Ioana Bica, Ivana Balazevic, Joan Puigcerver, Pinelopi Papalampidi, Olivier Henaff, Xi Xiong, Radu Soricut, Jeremiah Harmsen, and Xiaohua Zhai. PaliGemma: A versatile 3B VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *AAAI*, 2020.

Daniel Bolya, Po-Yao Huang, Peize Sun, Jang Hyun Cho, Andrea Madotto, Chen Wei, Tengyu Ma, Jiale Zhi, Jathushan Rajasegaran, Hanoona Rasheed, Junke Wang, Marco Monteiro, Hu Xu, Shiyu Dong, Nikhila Ravi, Daniel Li, Piotr Dollár, and Christoph Feichtenhofer. Perception Encoder: The best visual embeddings are not at the output of the network. In *NeurIPS*, 2025.

Ellis Brown, Jihan Yang, Shusheng Yang, Rob Fergus, and Saining Xie. Benchmark designers should "train on the test set" to expose exploitable non-visual shortcuts. In *COLM*, 2026.

Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *ICCV*, 2021.

Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024.

Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, and Feng Zhao. Are we on the right way for evaluating large vision-language models? In *NeurIPS*, 2024.

Lin Chen, Bolin Ni, Qi Yang, Lan Jiang, Kun Ding, Xiaoran Fan, Hower Yang, Ying Wang, and Shiming Xiang. How far are we from removing the visual encoder? Scaling laws for encoder-free multimodal pretraining. *arXiv preprint arXiv:2609.35457*, 2026.

Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. AdaptFormer: Adapting vision transformers for scalable visual recognition. In *NeurIPS*, 2022.

Xi Chen, Xiao Wang, Soravit Changpinyo, AJ Piergiovanni, Piotr Padlewski, Daniel Salz, Sebastian Goodman, Adam Grycner, Basil Mustafa, Lucas Beyer, Alexander Kolesnikov, Joan Puigcerver, Nan Ding, Keran Rong, Hassan Akbari, Gaurav Mishra, Linting Xue, Ashish V. Thapliyal, James Bradbury, Weicheng Kuo, Mojtaba Seyedhosseini, Chao Jia, Burcu Karagol Ayan, Carlos Riquelme, Andreas Steiner, Anelia Angelova, Xiaohua Zhai, Neil Houlsby, and Radu Soricut. PaLI: A jointly-scaled multilingual language-image model. In *ICLR*, 2023.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. PaLM: Scaling language modeling with pathways. *JMLR*, 2023.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *NAACL*, 2019.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, Rodolphe Jenatton, Lucas Beyer, Michael Tschannen, Anurag Arnab, Xiao Wang, Carlos Riquelme Ruiz, Matthias Minderer, Joan Puigcerver, Utku Evci, Manoj Kumar, Sjoerd van Steenkiste, Gamaleldin Fathy Elsayed, Aravindh Mahendran, Fisher Yu, Avital Oliver, Fantine Huot, Jasmijn Bastings, Mark Collier, Alexey A. Gritsenko, Vighnesh Birodkar, Cristina Nader Vasconcelos, Yi Tay, Thomas Mensink, Alexander Kolesnikov, Filip Pavetić, Dustin Tran, Thomas Kipf, Mario Lučić, Xiaohua Zhai, Daniel Keysers, Jeremiah J. Harmsen, and Neil Houlsby. Scaling vision transformers to 22 billion parameters. In *ICML*, 2023.

Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.

Haiwen Diao, Yufeng Cui, Xiaotong Li, Yueze Wang, Huchuan Lu, and Xinlong Wang. Unveiling encoder-free vision-language models. In *NeurIPS*, 2024.

Haiwen Diao, Xiaotong Li, Yufeng Cui, Yueze Wang, Haoge Deng, Ting Pan, Wenxuan Wang, Huchuan Lu, and Xinlong Wang. EVEv2: Improved baselines for encoder-free vision-language models. In *ICCV*, 2025.

Haiwen Diao, Mingxuan Li, Silei Wu, Linjun Dai, Xiaohua Wang, Hanming Deng, Lewei Lu, Dahua Lin, and Ziwei Liu. From pixels to words – towards native vision-language primitives at scale. In *ICLR*, 2026.

Xiaoyi Dong, Pan Zhang, Yuhang Zang, Yuhang Cao, Bin Wang, Linke Ouyang, Xilin Wei, Songyang Zhang, Haodong Duan, Maosong Cao, Wenwei Zhang, Yining Li, Hang Yan, Yang Gao, Xinyue Zhang, Wei Li, Jingwen Li, Kai Chen, Conghui He, Xingcheng Zhang, Yu Qiao, Dahua Lin, and Jiaqi Wang. InternLM-XComposer2: Mastering free-form text-image composition and comprehension in vision-language large model. *arXiv preprint arXiv:2401.16420*, 2024.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

David Fan, Shengbang Tong, Jiachen Zhu, Koustuv Sinha, Zhuang Liu, Xinlei Chen, Michael Rabbat, Nicolas Ballas, Yann LeCun, Amir Bar, and Saining Xie. Scaling language-free visual representation learning. In *ICCV*, 2025.

Chaoyou Fu, Peixian Chen, Yunhang Shen, Yulei Qin, Mengdan Zhang, Xu Lin, Jinrui Yang, Xiawu Zheng, Ke Li, Xing Sun, Yunsheng Wu, Rongrong Ji, Caifeng Shan, and Ran He. MME: A comprehensive evaluation benchmark for multimodal large language models. In *NeurIPS (Datasets and Benchmarks Track)*, 2025.

Gemma Team. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026.

Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *CVPR*, 2017.

Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro Von Werra, and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. In *NeurIPS*, 2024.

Junlin Han, Shengbang Tong, David Fan, Minghao Chen, Philip Torr, Filippos Kokkinos, and Mike Lewis. Towards physics of multimodal pretraining: Knowledge flow, modality synergy, early unification, and recipes. *arXiv preprint arXiv:2608.05000*, 2026.

Jitai Hao, Hao Liu, Xinyan Xiao, Qiang Huang, and Jun Yu. Uni-X: Mitigating modality conflict with a two-end-separated architecture for unified multimodal models. In *ICLR*, 2026.

Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. Towards a unified view of parameter-efficient transfer learning. In *ICLR*, 2022.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *ICLR*, 2021.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. Training compute-optimal large language models. In *NeurIPS*, 2022.

Cheng-Yu Hsieh, Jieyu Zhang, Zixian Ma, Aniruddha Kembhavi, and Ranjay Krishna. SugarCrepe: Fixing hackable benchmarks for vision-language compositionality. In *NeurIPS (Datasets and Benchmarks Track)*, 2023.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zhen Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. MiniCPM: Unveiling the potential of small language models with scalable training strategies. In *COLM*, 2024.

Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *NeurIPS*, 2019.

Drew A. Hudson and Christopher D. Manning. GQA: A new dataset for real-world visual reasoning and compositional question answering. In *CVPR*, 2019.

Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. https://kellerjordan.github.io/posts/muon/.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. Prismatic VLMs: Investigating the design space of visually-conditioned language models. In *ICML*, 2024.

Sedrick Keh, Jean Mercat, Samir Yitzhak Gadre, Kushal Arora, Igor Vasiljevic, Benjamin Burchfiel, Shuran Song, Russ Tedrake, Thomas Kollar, Ludwig Schmidt, and Achal Dave. Should VLMs be pre-trained with image data? In *ICLR*, 2025.

Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *ECCV*, 2016.

Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026a.

Kimi Team. Kimi K3: Open frontier intelligence. *arXiv preprint arXiv:2607.24653*, 2026b.

Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. RACE: Large-scale ReAding comprehension dataset from examinations. In *EMNLP*, 2017.

Hugo Laurençon, Léo Tronchon, Matthieu Cord, and Victor Sanh. What matters when building vision-language models? In *NeurIPS*, 2024.

Weixian Lei, Jiacong Wang, Haochen Wang, Xiangtai Li, Jun Hao Liew, Jiashi Feng, and Zilong Huang. The scalability of simplicity: Empirical analysis of vision-language learning with a single transformer. In *ICCV*, 2025.

Alexander C. Li, Ellis Brown, Alexei A. Efros, and Deepak Pathak. Internet Explorer: Targeted representation learning on the open web. In *ICML*, 2023a.

Bohao Li, Yuying Ge, Yixiao Ge, Guangzhi Wang, Rui Wang, Ruimao Zhang, and Ying Shan. SEED-Bench: Benchmarking multimodal large language models. In *CVPR*, 2024.

Bozhou Li, Xinda Xue, Sihan Yang, Yang Shi, Xinlong Chen, Yushuo Guan, Yuanxing Zhang, and Wentao Zhang. The unseen bias: How norm discrepancy in pre-norm MLLMs leads to visual information loss. In *ICLR*, 2026.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*, 2023b.

Xianhang Li, Yanqing Liu, Haoqin Tu, Hongru Zhu, and Cihang Xie. OpenVision: A fully-open, cost-effective family of advanced vision encoders for multimodal learning. In *ICCV*, 2025.

Weixin Liang, Lili Yu, Liang Luo, Srinivasan Iyer, Ning Dong, Chunting Zhou, Gargi Ghosh, Mike Lewis, Wen-tau Yih, Luke Zettlemoyer, and Xi Victoria Lin. Mixture-of-Transformers: A sparse and scalable architecture for multi-modal foundation models. *TMLR*, 2025.

Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft COCO: Common objects in context. In *ECCV*, 2014.

Xi Victoria Lin, Akshat Shrivastava, Liang Luo, Srinivasan Iyer, Mike Lewis, Gargi Ghosh, Luke Zettlemoyer, and Armen Aghajanyan. MoMa: Efficient early-fusion pre-training with mixture of modality-aware experts. *arXiv preprint arXiv:2407.21770*, 2024.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.

Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, Kai Chen, and Dahua Lin. MMBench: Is your multi-modal model an all-around player? In *ECCV*, 2024a.

Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xu-Cheng Yin, Cheng-Lin Liu, Lianwen Jin, and Xiang Bai. OCRBench: On the hidden mystery of OCR in large multimodal models. *Science China Information Sciences*, 2024b.

Zhiheng Liu, Weiming Ren, Xiaoke Huang, Shoufa Chen, Tianhong Li, Mengzhao Chen, Yatai Ji, Sen He, Jonas Schult, Belinda Zeng, Tao Xiang, Wenhu Chen, Ping Luo, Luke Zettlemoyer, and Yuren Cong. Tuna-2: Pixel embeddings beat vision encoders for multimodal understanding and generation. In *NeurIPS*, 2026.

Haoyu Lu, Wen Liu, Bo Zhang, Bingxuan Wang, Kai Dong, Bo Liu, Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, Yaofeng Sun, Chengqi Deng, Hanwei Xu, Zhenda Xie, and Chong Ruan. DeepSeek-VL: Towards real-world vision-language understanding. *arXiv preprint arXiv:2403.05525*, 2024a.

Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *NeurIPS*, 2022.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. In *ICLR*, 2024b.

Gen Luo, Wenhan Dou, Wenhao Li, Zhaokai Wang, Xue Yang, Changyao Tian, Hao Li, Weiyun Wang, Wenhai Wang, Xizhou Zhu, Yu Qiao, and Jifeng Dai. Mono-InternVL-1.5: Towards cheaper and faster monolithic multimodal large language models. *arXiv preprint arXiv:2507.12566*, 2025a.

Gen Luo, Xue Yang, Wenhan Dou, Zhaokai Wang, Jiawen Liu, Jifeng Dai, Yu Qiao, and Xizhou Zhu. Mono-InternVL: Pushing the boundaries of monolithic multimodal large language models with endogenous visual pre-training. In *CVPR*, 2025b.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of ACL*, 2022.

Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. DocVQA: A dataset for VQA on document images. In *WACV*, 2021.

Brandon McKinzie, Zhe Gan, Jean-Philippe Fauconnier, Sam Dodge, Bowen Zhang, Philipp Dufter, Dhruti Shah, Xianzhi Du, Futang Peng, Floris Weers, Anton Belyi, Haotian Zhang, Karanjeet Singh, Doug Kang, Ankur Jain, Hongyu Hè, Max Schwarzer, Tom Gunter, Xiang Kong, Aonan Zhang, Jianyu Wang, Chong Wang, Nan Du, Tao Lei, Sam Wiseman, Guoli Yin, Mark Lee, Zirui Wang, Ruoming Pang, Peter Grasch, Alexander Toshev, and Yinfei Yang. MM1: Methods, analysis and insights from multimodal LLM pre-training. In *ECCV*, 2024.

Lingchen Meng, Jianwei Yang, Rui Tian, Xiyang Dai, Zuxuan Wu, Jianfeng Gao, and Yu-Gang Jiang. DeepStack: Deeply stacking visual tokens is surprisingly simple and effective for LMMs. In *NeurIPS*, 2024.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? A new dataset for open book question answering. In *EMNLP*, 2018.

Muse Glimmer Team. Muse Glimmer 30B: An on-device agentic model. *arXiv preprint arXiv:2610.24628*, 2026.

Arsha Nagrani, Shan Yang, Anurag Arnab, Aren Jansen, Cordelia Schmid, and Chen Sun. Attention bottlenecks for multimodal fusion. In *NeurIPS*, 2021.

NVIDIA. Nemotron 3 Nano: Open, efficient mixture-of-experts hybrid Mamba-Transformer model for agentic reasoning. *arXiv preprint arXiv:2512.20848*, 2025a.

NVIDIA. NVIDIA Nemotron Nano 2: An accurate and efficient hybrid Mamba-Transformer reasoning model. *arXiv preprint arXiv:2508.14444*, 2025b.

Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *TMLR*, 2024.

Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving discrepancies in compute-optimal scaling of language models. In *NeurIPS*, 2024.

Jianing Qi, Jiawei Liu, Hao Tang, and Zhigang Zhu. Beyond semantics: Rediscovering spatial awareness in vision-language models. In *COLM*, 2026.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI Spring Symposium*, 2011.

Alexandre Sablayrolles, Matthijs Douze, Cordelia Schmid, and Hervé Jégou. Spreading vectors for similarity search. In *ICLR*, 2019.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial Winograd schema challenge at scale. In *AAAI*, 2020.

Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. Social IQa: Commonsense reasoning about social interactions. In *EMNLP*, 2019.

Hassan Shapourian, Kasra Hejazi, Olabode M. Sule, and Beren Millidge. ZAYA1-VL-8B technical report. *arXiv preprint arXiv:2605.08560*, 2026.

Noam Shazeer. GLU variants improve Transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Weijia Shi, Xiaochuang Han, Chunting Zhou, Weixin Liang, Xi Victoria Lin, Luke Zettlemoyer, and Lili Yu. LMFusion: Adapting pretrained language models for multimodal generation. In *NeurIPS*, 2025.

Mustafa Shukor, Enrico Fini, Victor Guilherme Turrisi da Costa, Matthieu Cord, Joshua Susskind, and Alaaeldin El-Nouby. Scaling laws for native multimodal models. In *ICCV*, 2025.

Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seung Eun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3. *TMLR*, 2026.

Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards VQA models that can read. In *CVPR*, 2019.

Andreas Steiner, André Susano Pinto, Michael Tschannen, Daniel Keysers, Xiao Wang, Yonatan Bitton, Alexey Gritsenko, Matthias Minderer, Anthony Sherbondy, Shangbang Long, Siyang Qin, Reeve Ingle, Emanuele Bugliarello, Sahar Kazemzadeh, Thomas Mesnard, Ibrahim Alabdulmohsin, Lucas Beyer, and Xiaohua Zhai. PaliGemma 2: A family of versatile VLMs for transfer. *arXiv preprint arXiv:2412.03555*, 2024.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

Thinking Machines Lab. Inkling: Our open-weights model, 2026a. https://thinkingmachines.ai/news/introducing-inkling/.

Thinking Machines Lab. Introducing Inkling 2 Small, 2026b. https://thinkingmachines.ai/news/inkling-2-small/.

Changyao Tian, Hao Li, Gen Luo, Xizhou Zhu, Weijie Su, Hanming Deng, Jinguo Zhu, Jie Shao, Ziran Zhu, Yunpeng Liu, Lewei Lu, Wenhai Wang, Hongsheng Li, and Jifeng Dai. NaViL: Rethinking scaling properties of native multimodal large language models under data constraints. In *NeurIPS*, 2025.

Shengbang Tong, Ellis Brown, Penghao Wu, Sanghyun Woo, Manoj Middepogu, Sai Charitha Akula, Jihan Yang, Shusheng Yang, Adithya Iyer, Xichen Pan, Ziteng Wang, Rob Fergus, Yann LeCun, and Saining Xie. Cambrian-1: A fully open, vision-centric exploration of multimodal LLMs. In *NeurIPS*, 2024a.

Shengbang Tong, Zhuang Liu, Yuexiang Zhai, Yi Ma, Yann LeCun, and Saining Xie. Eyes Wide Shut? Exploring the visual shortcomings of multimodal LLMs. In *CVPR*, 2024b.

Shengbang Tong, David Fan, John Nguyen, Ellis Brown, Gaoyue Zhou, Shengyi Qian, Boyang Zheng, Théophane Vallaeys, Junlin Han, Rob Fergus, Naila Murray, Marjan Ghazvininejad, Mike Lewis, Nicolas Ballas, Amir Bar, Michael Rabbat, Jakob Verbeek, Luke Zettlemoyer, Koustuv Sinha, Yann LeCun, and Saining Xie. Beyond language modeling: An exploration of multimodal pretraining. In *ICML*, 2026.

Hugo Touvron, Matthieu Cord, Alaaeldin El-Nouby, Jakob Verbeek, and Hervé Jégou. Three things everyone should know about vision transformers. In *ECCV*, 2022.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Michael Tschannen, Manoj Kumar, Andreas Steiner, Xiaohua Zhai, Neil Houlsby, and Lucas Beyer. Image captioners are scalable vision learners too. In *NeurIPS*, 2023.

Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, Olivier Hénaff, Jeremiah Harmsen, Andreas Steiner, and Xiaohua Zhai. SigLIP 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

Han Wang, Yongjie Ye, Bingru Li, Yuxiang Nie, Jinghui Lu, Jingqun Tang, Yanjie Wang, and Can Huang. Vision as LoRA. *arXiv preprint arXiv:2503.20680*, 2025.

Weihan Wang, Qingsong Lv, Wenmeng Yu, Wenyi Hong, Ji Qi, Yan Wang, Junhui Ji, Zhuoyi Yang, Lei Zhao, Xixuan Song, Jiazheng Xu, Keqin Chen, Bin Xu, Juanzi Li, Yuxiao Dong, Ming Ding, and Jie Tang. CogVLM: Visual expert for pretrained language models. In *NeurIPS*, 2024.

Xinlong Wang, Yufeng Cui, Jinsheng Wang, Fan Zhang, Yueze Wang, Xiaosong Zhang, Zhengxiong Luo, Quan Sun, Zhen Li, Yuqi Wang, Qiying Yu, Yingli Zhao, Yulong Ao, Xuebin Min, Chunlei Men, Boya Wu, Bo Zhao, Bowen Zhang, Liangdong Wang, Guang Liu, Zheqi He, Xi Yang, Jingjing Liu, Yonghua Lin, Zhongyuan Wang, and Tiejun Huang. Multimodal learning with next-token prediction for large multimodal models. *Nature*, 2026.

Kaiyue Wen, Zhiyuan Li, Jason Wang, David Hall, Percy Liang, and Tengyu Ma. Understanding warmup-stable-decay learning rates: A river valley loss landscape view. In *ICLR*, 2025.

xAI. Grok-1.5 Vision preview, 2024. https://x.ai/news/grok-1.5v.

Hu Xu, Saining Xie, Xiaoqing Ellen Tan, Po-Yao Huang, Russell Howes, Vasu Sharma, Shang-Wen Li, Gargi Ghosh, Luke Zettlemoyer, and Christoph Feichtenhofer. Demystifying CLIP data. In *ICLR*, 2024.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert AGI. In *CVPR*, 2024.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *ACL*, 2019.

Xiaohua Zhai, Alexander Kolesnikov, Neil Houlsby, and Lucas Beyer. Scaling vision transformers. In *CVPR*, 2022.

Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *ICCV*, 2023.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *NeurIPS*, 2019.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 2023.

Jinghao Zhou, Chen Wei, Huiyu Wang, Wei Shen, Cihang Xie, Alan Yuille, and Tao Kong. iBOT: Image BERT pre-training with online tokenizer. In *ICLR*, 2022.

# Appendix

§A **Controlled protocol** (Section 2): architectures, encoder design selection, optimization, training data and mixture (including when image–text data enters training), and compute accounting.
§B **Evaluation** (Section 2.5): VQA benchmarks with instruction tuning and the blank-image control, language benchmarks and the text-only reference, held-out losses, and ImageNet probes.
§C **Separate encoder pretraining** (Sections 3 and 6.1): objectives, upstream budgets, and pretraining at fixed multimodal compute.
§D **Dedicated vision capacity** (Section 4): the encoder's advantage by VQA category, shared-capacity controls, richer patch embeddings, and the encoder learning rate behind the comparison with Shukor et al. (2025).
§E **In-fusion vision capacity** (Section 5): implementation and sizing; routing, depth, and attention-projection controls; the longest budget; and where vision tokens are processed.
§F **Decay, learning rate, and caption loss** (Sections 2.4 and 4): protocol checks, with every decayed endpoint, learning rate, and held-out caption loss as a proxy for Avg. VQA.
§G **Scaling ladder** (Section 5.4): the ladder's setup, the scaling laws fitted to it, and their predictions for runs they were not fitted on.
§H **Beyond lifetime compute: midtraining and inference** (Section 6): midtraining and its vision learning rate, training time, and inference latency.
§I **Per-benchmark results**: every VQA and text benchmark for each main-text exhibit.

Tables abbreviate the Avg. Text categories as Com (commonsense reasoning), Sci (science question answering), Knw (broad knowledge), and Rdg (reading comprehension), and the Avg. VQA categories as Gen (general), Knw (knowledge), OCR (OCR and chart), and Vis (vision-centric). Appendix B defines them.

## A Controlled protocol

Unless stated otherwise, every comparison of the 1.7B and 4B backbones uses the settings below, which detail the protocol of Section 2.

### A.1 Architectures

The multimodal recipes differ in where their vision-only capacity sits and how it is trained: in a pre-fusion vision encoder that is pretrained and then frozen or updated, or trained from scratch, encoder-free with no module besides a patch embedding, or vision-only modules inside the shared backbone (in-fusion vision capacity; Appendix E.1). Unless a variable is the stated ablation, comparisons hold the backbone, the 256 vision tokens per image in multimodal training, the data, the optimization recipe, and lifetime compute fixed. Table 4 lists the backbone and encoder architectures.

**Backbone.** The shared backbone is a dense decoder-only Transformer that follows the Qwen3-1.7B and Qwen3-4B configurations (Yang et al., 2025) and is trained from random initialization. Its FFNs use SwiGLU (Shazeer, 2020), and its attention uses grouped key–value heads with RMSNorm on queries and keys. The vision-only FFN branches of in-fusion vision capacity use the same form and read the same normalized input as the shared FFN (Appendix E.1).

**Vision encoder.** The encoder follows MoonViT-V2, the vision encoder of Kimi K3 (Kimi Team, 2026b), in depth, patch size, normalization, and size: 27 layers, patch size 14, RMSNorm, and about 0.4B parameters in total. It has width 1,024 with 4,096-wide MLPs and uses learned spatial position embeddings combined with interleaved two-dimensional rotary position embeddings. During multimodal training, each image is resized bilinearly to a shorter side of 224 pixels, center-cropped to $224 \times 224$, and divided into $14 \times 14$ patches, producing 256 vision tokens. For an image $x$, the encoder recipe computes

$$v_{\text{enc}}(x) = W_{\text{proj}} f_\phi(x), \tag{4}$$

where $f_\phi$ is the vision encoder and the linear projector $W_{\text{proj}}$ maps its output width to the backbone width. Pretrained variants (Section 3) use the same architecture and differ only in the pretraining objective. MetaCLIP and WebSSL use a class token during upstream training; we discard it and pass only the 256 patch tokens to the backbone (Liu et al., 2023).

**Table 4 Every multimodal recipe shares one backbone configuration per scale, and the encoder adds a 0.4B ViT.** Parameter counts exclude the linear projection from the encoder into the backbone.

| | Backbone | | Vision encoder |
|---|---|---|---|
| | 1.7B | 4B | 0.4B |
| Configuration | Qwen3-1.7B | Qwen3-4B | MoonViT-V2 |
| Layers | 28 | 36 | 27 |
| Width | 2,048 | 2,560 | 1,024 |
| Query heads | 16 | 32 | 12 |
| FFN width | 6,144 | 9,728 | 4,096 |
| Tied embeddings | ✓ | ✓ | – |
| Parameters (B) | 1.72 | 4.02 | 0.40 |
| Non-embedding parameters (B) | 1.41 | 3.63 | – |
| | *Shared* | | |
| Key–value heads | 8 | | 12 |
| Head dimension | 128 | | 128 |
| FFN activation | SwiGLU | | GELU |
| Normalization | RMSNorm | | RMSNorm |
| Position encoding | 1D RoPE | | learned 2D + 2D RoPE |
| Input | 152K-token vocabulary | | $14 \times 14$ patches |
| Tokens | 4,096 per sequence | | 256 per image |

**Encoder-free patch embedding.** The encoder-free recipe projects each flattened $14 \times 14$ RGB patch linearly into the backbone width, as in Fuyu and Gemma 4 (Adept AI, 2023; Gemma Team, 2026), and applies RMSNorm and a learned scalar gate (Qi et al., 2026; Li et al., 2026), giving 256 backbone tokens per image:

$$v_{\text{free}}(x) = g\ \text{RMSNorm}\left(W_{\text{patch}}\,\text{Patchify}(x) + p_{\text{2D}}\right), \tag{5}$$

where $p_{\text{2D}}$ is a fixed two-dimensional sine–cosine position embedding and the learned scalar gate $g$ is initialized to 0.1. The in-fusion models of Appendix E.1 use the same patch embedding.

## A.2 Encoder design selection

We chose the from-scratch encoder's shape and size at matched lifetime compute, and every main comparison uses the resulting 27-layer, 0.4B encoder. The shape and size comparisons use the 1.7B backbone at $2.6 \times 10^{21}$ FLOPs (Table 5).

The configurations in Table 5 vary the structure of the dedicated visual computation. Half-Depth, Wide, and Shallow + Wide are pre-fusion encoders that change the default encoder's depth, width, or both. The backbone-style encoder is also a pre-fusion encoder. It has its own patch embedding and eight Transformer blocks shaped like those of the 1.7B backbone, and it connects to the backbone through the linear projection of Equation 4. The Default, Shallow + Wide, and backbone-style variants use about the same vision-only parameter budget; the Half-Depth and Wide variants test smaller and larger budgets, respectively. The "Size" section of Table 5 scales the default encoder's depth and width together, from Small (0.05B) to Huge (0.92B). At matched lifetime compute, encoders with more parameters process fewer tokens.

The default encoder is comparable to Wide on VQA while using fewer vision-only parameters. Avg. Text does not follow encoder size: Small and Huge score highest, and the default slightly lower.

**At 4B.** We further ablate encoder size at the 4B model scale, using the same from-scratch recipe (Table 6). The default Large encoder is strongest on both Avg. Text and Avg. VQA. At both scales, the default encoder gives the highest Avg. VQA, and Avg. Text does not follow encoder size.

**At 6.4B.** On the 6.4B backbone of the size ladder (Appendix G), we also keep the 0.4B encoder instead of scaling it to 1.7B with the backbone. It is comparable to the scaled encoder on both averages at every budget (Table 7), so the default encoder does not limit the encoder recipe at larger backbones either.

**Table 5 The default 27-layer, 0.4B encoder gives the strongest VQA and comparable performance on text.** Larger encoders do not automatically translate to stronger visual understanding. All rows use a 1.7B backbone at $2.6 \times 10^{21}$ FLOPs of lifetime compute. Bold: within one point of the best.

| Architecture | | | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Variant | Depth | Width | Vision-only params (B) | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| • Large (default) | 27 | 1,024 | 0.40 | **44.4** | **57.2** | 35.7 | **38.2** | 46.5 | 49.7 | 63.9 | 45.0 | 36.7 | 53.4 |
| *Shape* | | | | | | | | | | | | | |
| Half-Depth | 14 | 1,024 | 0.21 | 43.0 | 54.3 | 35.0 | 34.8 | **47.8** | 49.6 | 63.6 | **45.8** | 37.5 | 51.5 |
| Wide | 27 | 2,048 | 1.26 | **44.0** | 56.1 | 35.1 | **37.8** | **47.1** | 49.2 | 63.6 | 44.8 | 35.2 | 53.2 |
| Shallow + Wide | 9 | 2,048 | 0.42 | 42.1 | 54.9 | 35.6 | 34.2 | 43.9 | **49.9** | **64.1** | 44.8 | 36.3 | 54.2 |
| Backbone-style | 8 | 2,048 | 0.40 | **43.9** | **56.9** | **38.2** | **37.7** | 42.8 | **50.1** | 63.6 | 44.1 | 37.7 | **54.9** |
| *Size* | | | | | | | | | | | | | |
| Small | 12 | 512 | 0.05 | 41.1 | 52.9 | 35.6 | 33.2 | 42.5 | **50.9** | 62.9 | **45.8** | **39.1** | **55.9** |
| Base | 18 | 768 | 0.15 | 42.5 | 54.6 | 34.8 | 34.5 | 46.2 | **49.9** | **64.8** | **46.1** | 35.4 | 53.2 |
| Huge | 40 | 1,280 | 0.92 | 42.6 | 55.0 | 35.8 | 36.1 | 43.4 | **50.7** | **65.1** | 44.8 | 37.8 | **55.1** |

**Table 6 On the 4B backbone, the Large encoder attains the strongest VQA.** All variants are trained to $5.8 \times 10^{21}$ FLOPs of lifetime compute. Bold: within one point of the best.

| Architecture | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Variant | Vision-only params (B) | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| Base | 0.15 | 48.5 | 58.7 | 41.8 | 40.6 | 52.7 | **55.9** | **67.2** | **49.0** | **49.2** | 58.1 |
| • Large (default) | 0.40 | **51.2** | **62.7** | **44.5** | **43.5** | **54.0** | **56.4** | **67.5** | **49.8** | **49.4** | **58.8** |
| Huge | 0.92 | 47.6 | 58.7 | 42.2 | 39.7 | 49.6 | **55.8** | **67.0** | **49.3** | 47.9 | **59.2** |

**Table 7 At 6.4B, keeping the encoder at 0.4B is comparable to scaling it to 1.7B.** All runs follow the size ladder's recipe (Appendix G). Bold: within one point of the best.

| Setup | | VQA (%) ↑ | | | Text (%) ↑ | | |
|---|---|---|---|---|---|---|---|
| TPP | FLOPs | 0.4B encoder | 1.7B encoder | Encoder-free | 0.4B encoder | 1.7B encoder | Encoder-free |
| 16 | $3.90 \times 10^{21}$ | **45.9** | **46.0** | 40.4 | **52.3** | **53.0** | **52.6** |
| 32 | $7.81 \times 10^{21}$ | **49.7** | **50.2** | 44.3 | **56.4** | **56.4** | **56.4** |
| 64 | $1.56 \times 10^{22}$ | **53.1** | **52.8** | 47.5 | **59.9** | **59.1** | **58.9** |

## A.3 Optimization settings

Table 8 lists the optimization settings for multimodal training and instruction tuning. We train the Transformer weight matrices with Muon (Jordan et al., 2024), applied per head to the query, key, and value projections (Kimi Team, 2026b), and the embeddings, output heads, normalization parameters, and biases with AdamW. The peak learning rate scales with the inverse square root of the backbone's total parameter count, and training follows a warmup–stable–decay schedule (Hu et al., 2024; Wen et al., 2025; Hägele et al., 2024). Checkpoints taken before the final decay use a constant learning rate. The decay lowers the learning rate linearly to 5% of its peak over the last 10% of updates, so a run whose stable phase ends at compute $C$ finishes at $C/0.9$ (for example, $1.17 \times 10^{22}$ and $1.30 \times 10^{22}$ FLOPs at 1.7B and $2.61 \times 10^{22}$ and $2.90 \times 10^{22}$ FLOPs at 4B; Table 12).

**Table 8 The 1.7B and 4B backbones share one optimization recipe, with the peak learning rate scaled to backbone size.** Muon and AdamW use the same peak learning rate.

| | Multimodal training | Instruction tuning |
|---|---|---|
| Optimizer | Muon, AdamW | AdamW |
| Peak learning rate | $3 \times 10^{-4}$ (1.7B), $2 \times 10^{-4}$ (4B) | $1 \times 10^{-5}$ |
| Warmup updates | 4,000 | 1,000 |
| Schedule | warmup–stable–decay | cosine |
| Final learning rate | 5% of peak | 5% of peak |
| Weight decay | 0.1 | 0.1 |
| Gradient-norm clip | 1.0 | 5.0 |
| Batch (sequences × length) | $1{,}024 \times 4{,}096$ | $256 \times 4{,}096$ |
| Data | text and image–text, 70:30 | Cambrian-7M, 3 epochs |

## A.4 Training data and mixture

Text-only sequences come from Nemotron-CC v2 (NVIDIA, 2025b) and v2.1 (NVIDIA, 2025a), and image–text sequences use MetaCLIP 1.2 image–text pairs (English subset) (Xu et al., 2024). Every training sequence has 4,096 tokens and packs several documents. Attention is bidirectional among the vision tokens of each image and causal otherwise. It stays within each document in image–text sequences and in every sequence of the text-only reference, whereas the text sequences of image-trained models attend across packed documents. Multimodal training uses a 70:30 mixture of text and image–text sequences, which we select below (Table 10). Each image is processed at $224 \times 224$ resolution and yields 256 vision tokens (Appendix A.1). An image–text sequence packs about 12.8 MetaCLIP 1.2 pairs, each one image and its caption, so vision tokens fill about 80% of an image–text sequence and, under the 70:30 mixture, about 24% of all training tokens. Below, the data ablation varies the mixture and the study of when image–text data enters training uses a 90:10 mixture; the richer patch embeddings of Appendix D.3 read higher-resolution images.

**Recaptioning.** Multimodal training uses captions regenerated once for the MetaCLIP 1.2 images by Qwen2.5-VL-32B-Instruct (Bai et al., 2025b) and shared by every recipe, while encoder pretraining uses the original alt-text (Section 2). We use nucleus sampling with temperature 0.6 and top-$p$ 0.95, generating at most 128 new tokens. The system prompt is:

> `You are an expert image captioner. Write one specific, information-dense caption (1–2 sentences). Do not begin with 'A', 'An', or 'The'. Name the main subject first, then key objects, attributes, actions, and scene context. Avoid hedging and meta phrases.`

The user prompt is:

> `Generate a caption for this image.`

**Caption source for encoder pretraining.** To ablate the choice of regenerated captions versus alt-text, we pretrain the CapPa encoder for $7.7 \times 10^{20}$ FLOPs on either the alt-text or our regenerated captions and train it with the 1B ladder backbone (Table 28), a 0.4B encoder, and the 70:30 mixture, against the from-scratch encoder, to 32 and 64 tokens per parameter (Table 9). The regenerated-caption encoder scores slightly higher Avg. VQA than the alt-text encoder and matches the from-scratch encoder.

**Data mixture.** We select the 70:30 mixture of text and image–text sequences by an ablation on the 1.7B backbone with a from-scratch vision encoder, consistent with controlled studies of data balance in early multimodal training (Han et al., 2026). At the same lifetime compute ($2.34 \times 10^{21}$ FLOPs, at a constant learning rate), we compare it with 100:0, 90:10, and 50:50 mixtures (Table 10). The 70:30 mixture gives the best Avg. VQA with slight degradation to text, whereas 90:10 keeps Avg. Text closer to the text-only model.

We use 70:30 for the main comparisons as it gives a strong balance of visual understanding and language ability.

**When image–text data enters.** We also vary the point in training at which image–text data enters. Every schedule allocates 10% of lifetime training examples to image–text data, a 90:10 mixture rather than the constant 70:30 mixture above. We compare schedules that begin image–text training at different points while keeping the total number of image–text examples and lifetime compute fixed (Figure 11). Later schedules

**Table 9 Pretraining the encoder on regenerated captions slightly outperforms alt-text.** All variants are trained to either $1.8 \times 10^{20}$ or $3.7 \times 10^{20}$ FLOPs of multimodal training (32 and 64 TPP, respectively).

| Setup | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Encoder | TPP | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| • Scratch encoder | 32 | 28.6 | 33.8 | 27.5 | 9.7 | 43.4 | 42.8 | 56.5 | 39.6 | 26.4 | 48.6 |
| • CapPa, alt-text | 32 | 28.7 | 36.5 | 26.4 | 8.0 | 43.9 | 43.5 | 57.1 | 40.7 | 26.8 | 49.2 |
| • CapPa, recaptioned | 32 | 29.2 | 37.6 | 26.9 | 8.8 | 43.5 | 42.9 | 57.9 | 39.4 | 26.2 | 48.1 |
| • Scratch encoder | 64 | 34.6 | 43.0 | 27.9 | 22.1 | 45.2 | 44.7 | 58.9 | 41.6 | 27.3 | 50.9 |
| • CapPa, alt-text | 64 | 33.2 | 42.2 | 28.1 | 19.1 | 43.5 | 44.7 | 59.2 | 41.1 | 27.6 | 50.7 |
| • CapPa, recaptioned | 64 | 33.9 | 43.1 | 28.8 | 20.7 | 42.9 | 44.9 | 59.1 | 42.1 | 27.8 | 50.4 |

**Table 10 The 70:30 mixture favors visual understanding, whereas 90:10 preserves more language ability.** All mixtures use a 1.7B backbone at $2.34 \times 10^{21}$ FLOPs of lifetime compute at a constant learning rate. For 100:0, images are replaced with zeros during instruction tuning and evaluation, so its starred VQA entries are blank-image controls (Appendix B.1).

| Data mixture | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Text/image–text | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| 100:0 | 28.0* | 33.4* | 31.3* | 3.7* | 43.5* | 50.1 | 64.3 | 44.8 | 36.2 | 55.3 |
| 90:10 | 41.2 | 53.1 | 34.5 | 31.0 | 46.3 | **49.3** | 62.8 | 44.3 | **36.3** | **53.9** |
| 70:30 | **43.4** | **55.2** | **36.0** | **37.0** | 45.6 | 48.0 | 62.6 | **44.5** | 31.7 | 53.2 |
| 50:50 | 43.0 | 54.3 | 35.2 | 35.9 | **46.4** | 47.7 | **63.7** | 44.2 | 31.4 | 51.5 |

continue the language model and optimizer, add a randomly initialized vision encoder and projector, and warm up the new parameters for 4,000 updates. Starting image–text training at the first update gives the strongest visual understanding. Starting at 25%, 50%, or 75% of training lowers Avg. VQA, with the largest drops on general and on OCR and chart questions, while Avg. Text stays comparable (Table 11).

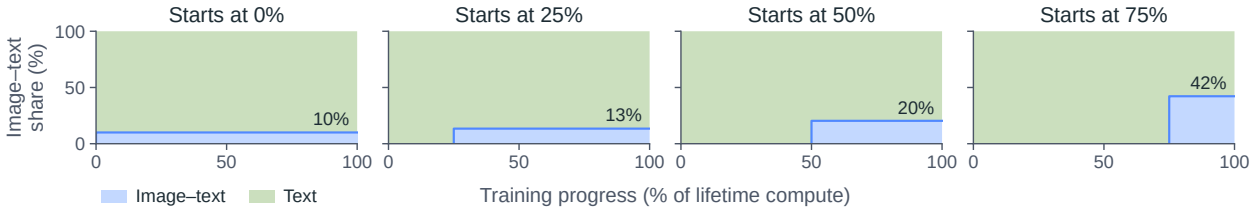



**Figure 11 Later starts concentrate the same image–text data into fewer updates.** Every schedule sees the same number of image–text examples, so the blue areas are about equal. Labels give each schedule's final image–text fraction.

**Table 11 Starting image–text training from the first update gives the strongest visual understanding.** All rows use a 1.7B backbone at $2.6 \times 10^{21}$ FLOPs of lifetime compute and a 90:10 mixture overall, scheduled as in Figure 11. Bold: within one point of the best.

| Image–text start | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| From first update | **42.5** | **55.1** | **36.0** | **32.5** | **46.2** | 50.8 | **64.6** | **46.2** | 38.0 | **54.4** |
| 25% start | 35.0 | 43.8 | 31.9 | 19.4 | 44.8 | **52.0** | **64.6** | **46.5** | **41.9** | **55.0** |
| 50% start | 35.1 | 43.8 | 34.0 | 17.0 | **45.5** | **51.0** | **64.2** | **46.3** | 40.2 | 53.1 |
| 75% start | 34.4 | 42.8 | 33.7 | 17.2 | 44.0 | 50.8 | **64.6** | **46.0** | 39.2 | 53.4 |

## A.5 Compute accounting

Lifetime compute (Equation 1) sums the training compute of every stage that produces the final model: multimodal training for every recipe, encoder pretraining for pretrained encoders, and, in midtraining (Section 6.2), the text-only pretraining of the language model. As with fine-tuning in transfer evaluation,

instruction tuning is a fixed downstream step applied to every checkpoint to measure VQA (Appendix B), so it is not counted. Each term uses the same algorithmic count of forward- and backward-pass matrix multiplication and convolution FLOPs for the modules active in that stage (Kaplan et al., 2020; Hoffmann et al., 2022). The count excludes activation recomputation (Chowdhery et al., 2023), optimizer updates, communication, elementwise operations, and input processing. A frozen encoder receives no backward pass during multimodal training, so it is charged only its forward pass, one third of its training cost. The free-encoder comparison of Section 6.1 instead matches multimodal training compute and does not charge encoder pretraining.

**Charging pretrained checkpoints.** A pretrained encoder is charged the full compute of its upstream training up to the checkpoint used. Because a MetaCLIP example costs less compute than a captioning example, the same upstream compute buys MetaCLIP about 12% more epochs. The objective comparisons hold upstream compute equal across objectives, so they use MetaCLIP checkpoints matched to the compute of 1, 2, 4, and 8 captioning epochs (Appendix C.1). The sweep of Appendix C.3 compares MetaCLIP only with itself, so it uses MetaCLIP's own 2-, 4-, 8-, and 13-epoch checkpoints.

**Token-equivalent budgets.** To provide intuition about compute scales, we also express each budget in *token-equivalents*, the number of tokens that a text-only model of the same backbone size would train on with the same compute:

$$H_{\text{eq}} = \frac{C_{\text{lifetime}}}{c_{\text{LM}}}, \tag{6}$$

where $c_{\text{LM}}$ is that text-only model's training FLOPs per token, $1.17 \times 10^{10}$ at 1.7B and $2.61 \times 10^{10}$ at 4B. For example, a budget of 200B token-equivalents is $2.34 \times 10^{21}$ FLOPs at 1.7B, and Table 12 gives every decayed budget in FLOPs and tokens per parameter; a constant-learning-rate checkpoint sits at 0.9 times its decayed budget (Appendix A.3).

**Table 12 Compute budgets can be expressed as token-equivalents, FLOPs, or TPP.** TPP divides by each backbone's non-embedding parameter count.

| | 1.7B | | 4B | |
|---|---|---|---|---|
| Budget | FLOPs | TPP | FLOPs | TPP |
| 55.6B | $6.5\times10^{20}$ | 39 | $1.45\times10^{21}$ | 15 |
| 111.1B | $1.30\times10^{21}$ | 79 | $2.90\times10^{21}$ | 31 |
| 222.2B | $2.60\times10^{21}$ | 158 | $5.79\times10^{21}$ | 61 |
| 444.4B | $5.21\times10^{21}$ | 315 | $1.16\times10^{22}$ | 122 |
| 666.7B | $7.81\times10^{21}$ | 473 | $1.74\times10^{22}$ | 183 |
| 888.9B | $1.04\times10^{22}$ | 631 | $2.32\times10^{22}$ | 245 |
| 1.11T | $1.30\times10^{22}$ | 788 | $2.90\times10^{22}$ | 306 |

Architecture and data mixture change the number of updates at fixed compute. For instance, at $2.34 \times 10^{21}$ FLOPs, the 1.7B text-only model takes 47.7K updates of $1{,}024 \times 4{,}096$ tokens (200B tokens), and the from-scratch encoder recipe with the 70:30 mixture takes 45.3K, because the encoder's computation also counts toward the budget.

# B Evaluation

This section describes the two main evaluation metrics, Avg. VQA and Avg. Text. We also define the held-out losses that Sections 4 and 5.4 and Appendix F.3 use as well as two additional metrics, out-of-distribution caption loss and ImageNet linear-probe accuracy.

## B.1 VQA benchmarks

**Visual instruction tuning.** We use instruction tuning as an evaluation protocol (Tong et al., 2024a). Every multimodal model checkpoint is instruction-tuned with one fixed recipe before we measure its visual understanding. The recipe runs three epochs of Cambrian-7M (Tong et al., 2024a) before evaluation. Multimodal training reads $224 \times 224$ images, but instruction tuning and evaluation raise the resolution to

$448 \times 448$ (1,024 vision tokens), since OCR and chart reading needs fine detail (Tong et al., 2024a; Beyer, 2024). Exhibits whose captions say so instead retain the multimodal-training resolution of $224 \times 224$ (256 vision tokens), otherwise with the same recipe. The encoder's learned position table is interpolated to the larger patch grid, and the fixed sine–cosine positions of encoder-free and in-fusion models are recomputed for it. Cambrian-7M is a curated instruction-tuning mixture spanning general visual tasks, OCR and charts, counting, math, science, code, and language-only data. Table 8 provides the optimizer settings and batch size. Instruction tuning updates the same parameters as multimodal training. Therefore, a frozen encoder, e.g., the frozen MetaCLIP encoder in Table 16, stays frozen.

**Benchmarks and Avg. VQA.** Evaluation follows the Cambrian-1 visual understanding suite (Tong et al., 2024a), using greedy decoding with at most 256 output tokens. Each task is capped at its first 5,000 examples in dataset order. Table 13 lists the 16 benchmarks in four categories (general, knowledge, OCR and chart, and vision-centric), with the split, metric, and number of examples for each. Exact match compares the normalized parsed answer or option letter with the reference answers, relaxed accuracy tolerates small numeric errors, and VQA accuracy is the consensus metric (Antol et al., 2015) that TextVQA uses. Each category mean averages its four benchmarks. The suite contains 15 benchmarks scored in $[0, 100]$ and MME-Perception (MME-P), whose ten subtasks each score out of 200. We divide the MME-P total by 20 to put it on the same scale, and Avg. VQA is the mean of the 16 scores, which equals the mean of the four category means.

**Table 13 Avg. VQA weights four categories of visual understanding benchmarks,** Each category contains four tasks. MME-P is normalized to $[0, 100]$ before averaging. Examples counts the questions evaluated.

| Benchmark | Reference | Split | Metric | Examples |
|---|---|---|---|---|
| *General (Gen)* | | | | |
| MME-P | Fu et al. (2025) | test | MME score / 20 | 2,114 |
| MMBench-EN | Liu et al. (2024a) | dev | Exact match | 4,329 |
| SEED-Bench | Li et al. (2024) | test | Exact match | 5,000 |
| GQA | Hudson and Manning (2019) | test-dev | Exact match | 5,000 |
| *Knowledge (Knw)* | | | | |
| ScienceQA (image) | Lu et al. (2022) | test | Exact match | 2,017 |
| MMMU | Yue et al. (2024) | validation | Exact match | 900 |
| MathVista | Lu et al. (2024b) | testmini | Exact match | 936 |
| AI2D | Kembhavi et al. (2016) | test | Exact match | 3,088 |
| *OCR and chart (OCR)* | | | | |
| ChartQA | Masry et al. (2022) | test | Relaxed accuracy | 2,500 |
| OCRBench | Liu et al. (2024b) | test | Exact match | 1,000 |
| TextVQA | Singh et al. (2019) | validation | VQA accuracy | 5,000 |
| DocVQA | Mathew et al. (2021) | validation | Exact match | 5,000 |
| *Vision-centric (Vis)* | | | | |
| MMVP | Tong et al. (2024b) | full | Exact match | 300 |
| RealWorldQA | xAI (2024) | test | Exact match | 765 |
| CV-Bench-2D | Tong et al. (2024a) | test | Exact match | 1,438 |
| CV-Bench-3D | Tong et al. (2024a) | test | Exact match | 1,200 |

**Blank-image control.** The *blank-image* control separates visual understanding from what the question alone supports. It replaces every image with zeros during both instruction tuning and evaluation while keeping the vision-token positions. We use it only for the text-only (100:0) row of the data-mixture ablation (Table 10), which has no trained visual pathway. At 1.7B, this row reaches 28 Avg. VQA, a measure of how much the questions alone support (Goyal et al., 2017; Chen et al., 2024; Brown et al., 2026).

## B.2 Language benchmarks

Language evaluations use the multimodal checkpoint without instruction tuning. Table 14 lists the 12 benchmarks, their categories, and how each is scored. All tasks rank the answer choices by log-likelihood; MMLU uses five subject-matched demonstrations and averages its 57 subjects equally, and the other tasks

are zero-shot. Avg. Text is the mean of the four category scores, each the mean of its benchmarks, with ARC-Easy and ARC-Challenge averaged into one ARC score and RACE-middle and RACE-high combined by test-question count.

**Table 14 Avg. Text averages four categories of text benchmarks.**

| Task | Reference | Shots | Scoring |
|---|---|---|---|
| *Commonsense reasoning (Com)* | | | |
| COPA | Roemmele et al. (2011) | 0 | Summed log-likelihood |
| HellaSwag | Zellers et al. (2019) | 0 | Mean token log-likelihood |
| PIQA | Bisk et al. (2020) | 0 | Mean token log-likelihood |
| Social IQa | Sap et al. (2019) | 0 | Summed log-likelihood |
| WinoGrande | Sakaguchi et al. (2020) | 0 | Summed log-likelihood |
| *Science question answering (Sci)* | | | |
| ARC-Easy | Clark et al. (2018) | 0 | Mean token log-likelihood |
| ARC-Challenge | Clark et al. (2018) | 0 | Mean token log-likelihood |
| OpenBookQA | Mihaylov et al. (2018) | 0 | Mean token log-likelihood |
| *Broad knowledge (Knw)* | | | |
| MMLU (57 subjects) | Hendrycks et al. (2021) | 5 | Likelihood ranking, macro-average |
| *Reading comprehension (Rdg)* | | | |
| BoolQ | Clark et al. (2019) | 0 | Summed log-likelihood |
| RACE-middle (1,436 test) | Lai et al. (2017) | 0 | Summed log-likelihood |
| RACE-high (3,498 test) | Lai et al. (2017) | 0 | Summed log-likelihood |

## B.3 Text-only reference

At the same text-token budget, the multimodal recipes score about as well on Avg. Text as a text-only reference trained on the same text: on average 0.1 point below it at 1.7B and 0.6 below at 4B (Figure 12). The encoder, encoder-free, and in-fusion recipes behave alike. At matched lifetime compute, the reference leads by 0.5 to 3.9 points (Table 27), because multimodal training gives 30% of its sequences to image–text data and so sees about 30% fewer text tokens. Vision therefore costs text little beyond the text data it displaces.

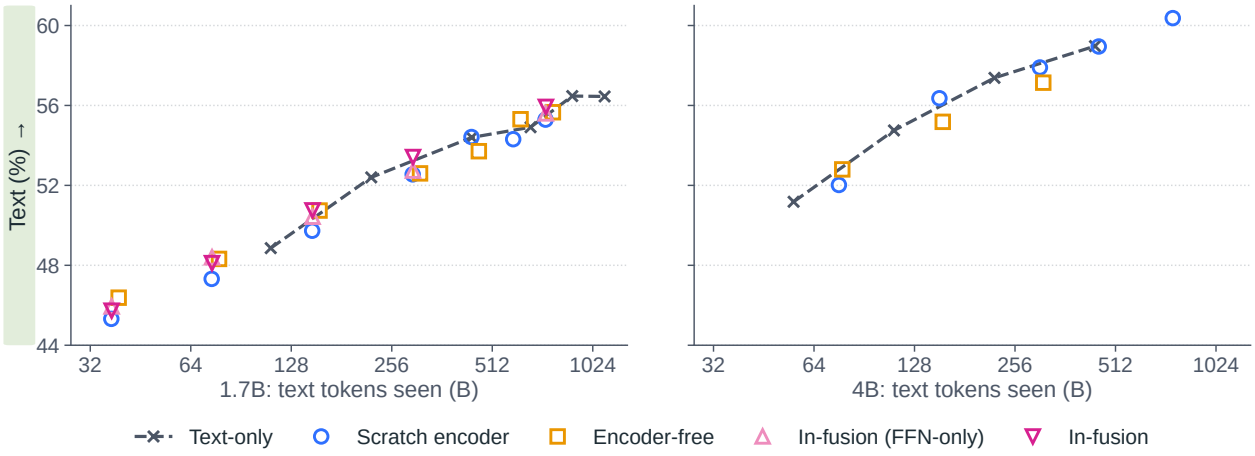



**Figure 12 At the same text-token budget, the multimodal recipes score about as well on Avg. Text as the text-only reference.** Each point is one decayed endpoint, placed by the text-only tokens it trained on (70% of multimodal training).

## B.4 Held-out losses

We measure losses on two kinds of held-out data, both as per-token negative log-likelihood in nats. The held-out text and caption losses score unseen samples of the training distributions, and Sections 4 and 5.4

and Appendix F.3 use them. Out-of-distribution caption loss scores caption sets outside those distributions, and Table 25 reports it.

**In-distribution.** Held-out text loss is the per-token negative log-likelihood on held-out text from the training text corpus, weighted as in the training mixture. It is scored with the attention masking each model was trained with. Held-out caption loss is the per-token negative log-likelihood of the regenerated captions of MetaCLIP 1.2 images never used in training, so it samples the training caption distribution.

**Out-of-distribution.** Out-of-distribution caption loss averages the caption loss of three sets of 5,000 images with equal weight: the COCO validation images with the first of their human reference captions (Lin et al., 2014), MetaCLIP 1.2 pairs with their original alt-text (Xu et al., 2024), and images from a licensed stock-photo collection. It is scored with the attention masking each model was trained with.

### B.5 ImageNet linear probes

We use three linear probes on ImageNet-1K (Deng et al., 2009). The *downstream* probe reads the multimodal model with all weights frozen. It mean-pools the backbone's final normalized vision-token states and fits an $\ell_2$-regularized linear classifier on 50 training images per class. A fixed 10% holdout selects the regularization strength from $\{10^{-4}, 10^{-3}, 10^{-2}\}$, and the classifier is then refit on all 50,000 images. We report top-1 accuracy on the full validation split. The *upstream* probe (Table 16) fits the same classifier to a frozen pretrained encoder before multimodal training. It mean-pools the encoder's 256 final normalized patch tokens of a $224 \times 224$ center crop; for WebSSL, it reads the exponential-moving-average teacher. The *full-data* probe applies the DINOv2 linear evaluation (Oquab et al., 2024) to the frozen pretrained encoder, training on the full ImageNet-1K training set. It gives the upstream accuracy in Figure 8 and Appendix C.3.

## C Separate encoder pretraining

This section details the pretrained-encoder comparisons of Section 3 and the sweep behind the zero-cost pretrained-encoder comparison of Section 6.1.

### C.1 Objectives and upstream training

We compare four upstream objectives on the same vision encoder architecture (Appendix A.1). MetaCLIP (Xu et al., 2024) uses a symmetric image–text contrastive objective (Radford et al., 2021). If $z_i$ and $w_i$ are normalized image and text representations in a batch of size $B$, its loss is

$$\mathcal{L}_{\text{clip}} = -\frac{1}{2B}\sum_{i=1}^{B}\left[\log\frac{\exp(z_i^\top w_i/\tau)}{\sum_j \exp(z_i^\top w_j/\tau)} + \log\frac{\exp(w_i^\top z_i/\tau)}{\sum_j \exp(w_i^\top z_j/\tau)}\right]. \tag{7}$$

Here $\tau$ is a learned softmax temperature, initialized at 0.07 and kept at or above 0.01. Cap (Tschannen et al., 2023) uses a causal decoder during upstream pretraining to predict caption tokens from the encoder output $f_\phi(x)$ (Equation 4),

$$\mathcal{L}_{\text{cap}} = -\mathbb{E}_{(x,y)}\left[\frac{1}{|y|}\sum_j \log p(y_j \mid f_\phi(x), y_{<j})\right]. \tag{8}$$

CapPa (Tschannen et al., 2023) combines this causal objective with fully masked parallel caption prediction. If $\mathcal{L}_{\text{par}}$ is the loss that predicts all caption positions in parallel from the image and a fully masked caption input, CapPa's expected loss is

$$\mathcal{L}_{\text{cappa}} = 0.25\,\mathcal{L}_{\text{cap}} + 0.75\,\mathcal{L}_{\text{par}}. \tag{9}$$

WebSSL (Fan et al., 2025) follows DINOv2 (Oquab et al., 2024) and combines class-token self-distillation (Caron et al., 2021) and KoLeo regularization (Sablayrolles et al., 2019) with iBOT masked-patch prediction (Zhou et al., 2022),

$$\mathcal{L}_{\text{webssl}} = \lambda_{\text{DINO}}\mathcal{L}_{\text{DINO}} + \lambda_{\text{iBOT}}\mathcal{L}_{\text{iBOT}} + \lambda_{\text{KoLeo}}\mathcal{L}_{\text{KoLeo}}. \tag{10}$$

We use the DINOv2 weights $\lambda_{\text{DINO}} = \lambda_{\text{iBOT}} = 1$ and $\lambda_{\text{KoLeo}} = 0.1$. The DINO term sums over the global and local crops, and the KoLeo term over the two global crops. We discard the text tower, caption decoder,

and self-supervised prediction heads before multimodal training. We follow the original objective definitions. MetaCLIP, Cap, and CapPa train on the original alt-text of the MetaCLIP 1.2 image–text pairs, and WebSSL trains on their images alone. Sharing one data source isolates the objective, since the pretraining data alone changes the learned representations (Li et al., 2023a; Fan et al., 2025). Table 15 lists the upstream training settings. MetaCLIP's text tower is a 12-layer Transformer of width 768 with 12 attention heads over 77 text tokens. The Cap and CapPa decoder follows the published CapPa decoder: 12 layers of width 1,024 with 16 attention heads, a 4,096-wide MLP, and cross-attention to the encoder's patch tokens in every layer, over captions of up to 64 tokens.

**Table 15 Every objective pretrains the same encoder on the same data and compute.** The training schedules span the entire horizon of each encoder pretraining run.

| | MetaCLIP | WebSSL | Cap and CapPa |
|---|---|---|---|
| Global batch (examples) | 32,768 | 3,072 | 8,192 |
| Optimizer | Muon, AdamW | Muon, AdamW | Muon, AdamW |
| Peak learning rate | $4 \times 10^{-4}$ | $3.5 \times 10^{-4}$ | $2.5 \times 10^{-4}$ |
| Warmup updates | 2,000 | 100,000 | 10,000 |
| Schedule | cosine to 0 | cosine to $10^{-6}$ | cosine to 0 |
| Weight decay | 0.2 | 0.04 to 0.2 (cosine) | $10^{-4}$ |
| Gradient-norm clip | none | 3.0 | 1.0 |
| Image crops | one $224 \times 224$ random resized crop | two $224 \times 224$ global, eight $98 \times 98$ local | one $224 \times 224$ random resized crop |

We compare the objectives at matched upstream compute. Each method uses the first checkpoint that reaches the compute of two epochs of the Cap objective over the image–text pairs ($1.54 \times 10^{21}$ FLOPs). For Cap and CapPa, this is 2 billion image–caption examples. MetaCLIP and WebSSL reach the same compute point after different numbers of examples. Appendix C.2 uses these checkpoints at both backbone sizes. Appendix C.2 also uses the first checkpoints that reach the compute of one, four, and eight Cap epochs (0.77, 3.08, and $6.16 \times 10^{21}$ FLOPs), the 7.4%, 30%, and 60% shares of Figure 2a. The sweep of Appendix C.3 compares MetaCLIP only with itself, so it uses MetaCLIP's own epoch checkpoints instead (Appendix A.5).

The ImageNet columns of Table 16 give the probe accuracy of each upstream encoder at this matched compute, before multimodal training. MetaCLIP has the highest and WebSSL the lowest top-1 accuracy, yet after multimodal training WebSSL reaches the highest Avg. VQA of the four (Appendix C.2), so their ImageNet recognition ranking does not carry over to Avg. VQA.

## C.2 Comparison of encoder objectives

At 1.7B, Table 16 compares the from-scratch encoder with encoders pretrained on each of the four objectives and updated during multimodal training, and with a frozen MetaCLIP encoder. The updated pretrained rows are the points at a 15% pretraining share in Figure 2a. The from-scratch encoder and WebSSL lead on Avg. VQA, with the other objectives 0.7 to 1.8 points behind, and all updated encoders are comparable on Avg. Text. MetaCLIP serves as the pretrained reference elsewhere because CLIP-style pretraining is the most widely used (Section 3), and the from-scratch encoder also matches WebSSL, the strongest objective here. No objective leads every category, as self-supervised WebSSL is best on general and vision-centric questions and the from-scratch encoder on knowledge and on OCR and chart questions. Every pretrained encoder trails the from-scratch encoder on OCR and chart questions, and only WebSSL and Cap gain from their priors, on vision-centric questions. Freezing MetaCLIP gives slightly higher Avg. Text but noticeably lower Avg. VQA, so an encoder that adapts during multimodal training is the stronger choice.

**At 4B.** We repeat the comparison on the 4B backbone with three of the objectives, at the $5.8 \times 10^{21}$-FLOP budget of Figure 2b (61 TPP), whose from-scratch and MetaCLIP points are the 4B rows of Table 16. WebSSL again scores highest among the objectives and MetaCLIP lowest, on both averages. The from-scratch encoder leads all three on Avg. VQA and Avg. Text, and on every VQA category.

**Table 16 The from-scratch encoder matches the best pretrained encoders at 1.7B and leads every objective at 4B.** The models are trained to $1.04 \times 10^{22}$ FLOPs at 1.7B and $5.8 \times 10^{21}$ FLOPs at 4B, including encoder pretraining. ImageNet accuracy is measured before multimodal training. Bold: within one point of the best at each backbone size.

| Encoder recipe | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | | ImageNet (%) ↑ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Backbone | Variant | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg | Top-1 | Top-5 |
| 1.7B | Scratch encoder | **49.0** | **60.3** | **41.4** | **43.4** | 51.1 | **54.3** | **66.1** | **47.6** | 47.3 | 56.3 | – | – |
| | MetaCLIP | 47.5 | 59.4 | 40.3 | 40.3 | 50.2 | **54.7** | **65.9** | **48.3** | **48.0** | 56.5 | **73.0** | **93.1** |
| | MetaCLIP (frozen) | 45.5 | 56.5 | 40.0 | 36.6 | 48.8 | **55.1** | **66.4** | **48.0** | **47.5** | **58.7** | **73.0** | **93.1** |
| | WebSSL | **49.3** | **60.6** | **41.3** | 39.8 | **55.5** | **54.6** | **66.1** | 46.5 | **48.5** | 57.3 | 61.5 | 86.1 |
| | Cap | **48.3** | 58.6 | **41.2** | 39.8 | 53.3 | **54.4** | **66.1** | **47.7** | 46.2 | 57.5 | **72.0** | 91.6 |
| | CapPa | 47.2 | 58.7 | 39.8 | 40.1 | 50.1 | **54.3** | **66.5** | **47.5** | 46.3 | 57.0 | 69.6 | 90.2 |
| 4B | Scratch encoder | **51.2** | **62.7** | **44.5** | **43.5** | **54.0** | **56.4** | **67.5** | **49.8** | **49.4** | **58.8** | – | – |
| | MetaCLIP | 46.5 | 58.4 | 39.2 | 40.3 | 48.1 | 53.2 | 66.1 | 47.8 | 43.6 | 55.5 | **73.0** | **93.1** |
| | WebSSL | 48.1 | 60.8 | 40.2 | 41.5 | 49.8 | 54.4 | **66.5** | 48.4 | 45.4 | 57.4 | 61.5 | 86.1 |
| | Cap | 46.6 | 58.3 | 40.3 | 39.2 | 48.4 | 53.7 | 66.1 | 48.5 | 44.6 | 55.8 | **72.0** | 91.6 |

**Training trajectories.** Figure 13 follows the from-scratch, updated MetaCLIP, and frozen MetaCLIP encoders through training at the 15% share. Every checkpoint before the last precedes the final learning-rate decay (Appendix A.3), so only the final points match Table 16.

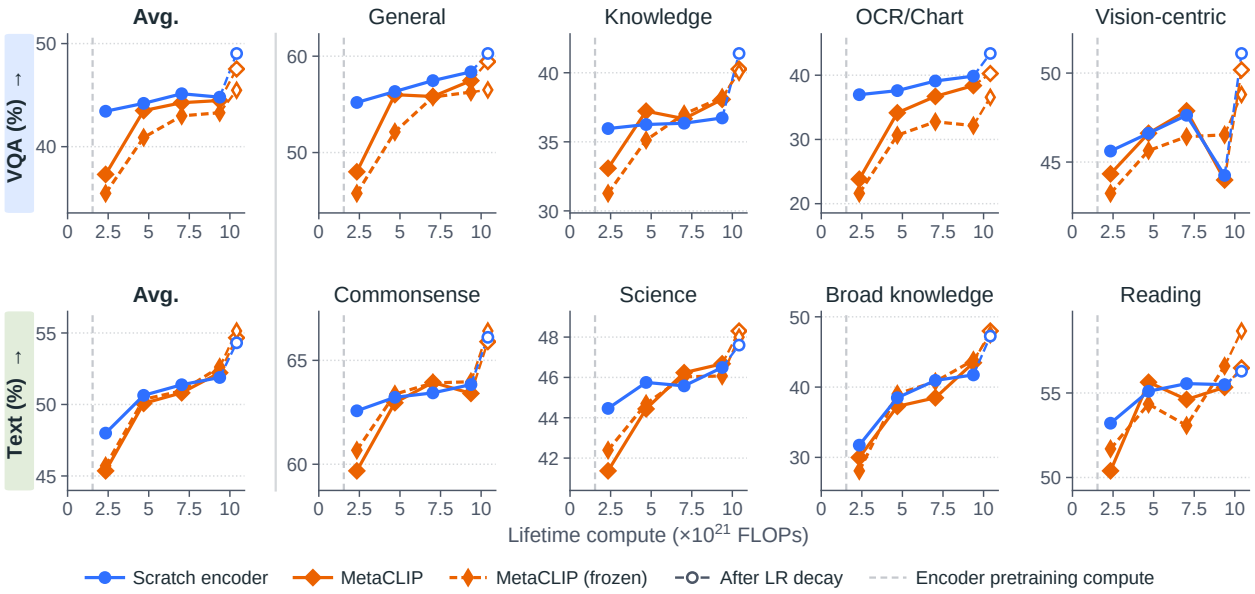



**Figure 13 Joint training with the scratch encoder is comparable to a pretrained encoder.** All models use a 1.7B backbone, and the curves start after the encoder's pretraining compute (dashed line).

**Compositional matching.** SugarCrepe pairs each image with a correct caption and a hard negative that replaces, swaps, or adds an object, attribute, or relation (Hsieh et al., 2023). We score the multimodal-training checkpoints before instruction tuning by caption likelihood on over 7,511 items in seven subsets. An item is correct when the correct caption's summed token loss is lower than the negative's loss. A control scores the same items with an unrelated image shared by both captions. Within each budget, every recipe reaches a comparable replace-and-swap mean, within about 1.5 points, and differs most on the small object-swap subset (Table 17). Unlike VQA (Section 4), this readout separates neither encoder-free from the encoders nor pretrained from from-scratch encoders.

**Table 17 All recipes are comparable on compositional caption matching.** Accuracy (%) is measured before instruction tuning. The last two columns average the replace and swap subsets, leaving out the add subsets because the text alone decides them. Bold: within one point of the best in each block, except the shuffled control.

| | Replace | | | Swap | | Add | | Replace and swap | |
|---|---|---|---|---|---|---|---|---|---|
| Model | Object | Attribute | Relation | Object | Attribute | Object | Attribute | Image | Shuffled |
| *1.7B,* $1.04 \times 10^{22}$ *FLOPs* | | | | | | | | | |
| Scratch encoder | **79.2** | **82.0** | 77.0 | 71.4 | 84.8 | **95.5** | **96.5** | **78.9** | 64.7 |
| Encoder-free | **79.7** | **82.2** | 77.3 | 72.2 | **86.2** | **95.9** | **97.4** | **79.5** | 66.8 |
| MetaCLIP | 78.2 | 80.6 | 77.4 | 73.5 | 84.2 | **96.1** | **96.7** | 78.8 | 64.7 |
| WebSSL | **78.8** | **81.2** | **79.2** | 73.9 | **86.6** | **96.0** | **96.4** | **79.9** | 65.1 |
| Cap | 78.3 | 81.1 | 76.6 | 71.0 | 85.1 | **95.5** | 96.2 | 78.4 | 64.3 |
| CapPa | 76.6 | 80.3 | 77.8 | **75.1** | 84.4 | **95.8** | 96.0 | **78.9** | 64.4 |
| *1.7B,* $1.30 \times 10^{22}$ *FLOPs* | | | | | | | | | |
| Scratch encoder | 77.6 | 80.6 | **78.9** | **74.7** | **84.7** | **95.4** | **97.1** | **79.3** | 65.3 |
| Encoder-free | **79.4** | **82.7** | 76.5 | 71.4 | **85.4** | **95.5** | **97.3** | **79.1** | 65.9 |
| In-fusion | 78.1 | 80.1 | 77.7 | 68.6 | **85.0** | **95.4** | **96.7** | 77.9 | 64.9 |
| *4B,* $1.16 \times 10^{22}$ *FLOPs* | | | | | | | | | |
| Scratch encoder | **79.5** | **81.2** | **78.2** | **72.2** | **85.4** | **95.6** | **97.4** | **79.3** | 64.3 |
| MetaCLIP | 77.3 | 79.3 | **78.4** | 69.8 | **86.3** | **95.4** | **96.5** | 78.2 | 65.1 |

## C.3 Encoder pretraining at fixed multimodal compute

Figure 8 plots a sweep of experiments that holds multimodal training fixed and does not charge encoder pretraining, unlike the lifetime-budget comparisons of Figure 2. Every model starts from a MetaCLIP checkpoint taken after 2, 4, 8, or 13 epochs over the image–text pairs (Appendix A.5). Every model then receives the same multimodal training: $2.7 \times 10^{21}$ FLOPs at 1.7B with the 70:30 mixture. Longer pretraining raises the pretrained encoder's full-data ImageNet linear-probe accuracy (Appendix B.5) but does not improve Avg. VQA after multimodal training. The shortest pretraining scores highest on Avg. VQA (Table 18). A lower encoder learning rate also does not recover a benefit from the encoder pretrained prior. Updating the shortest-pretrained MetaCLIP encoder at a tenth of the shared learning rate gives about the same Avg. VQA and Avg. Text as the full learning rate, and a from-scratch encoder with the same multimodal training matches both (Table 18).

**Table 18 At fixed multimodal compute, longer encoder pretraining improves ImageNet accuracy but not visual understanding.** All rows use a 1.7B backbone and $2.7 \times 10^{21}$ FLOPs of multimodal training; lifetime FLOPs add the encoder pretraining. ImageNet LP is a linear probe of the pretrained encoder before multimodal training. Bold: within one point of the best in each VQA and Text column.

| Setup | | | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Encoder | Pretraining FLOPs | Lifetime FLOPs | ImageNet LP | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| ◆ MetaCLIP | $1.4\times10^{21}$ | $4.1\times10^{21}$ | 80.4 | **43.2** | 55.2 | 34.8 | **36.6** | 46.0 | **50.5** | **63.9** | **45.2** | **39.9** | 52.9 |
| | $2.8\times10^{21}$ | $5.5\times10^{21}$ | 80.6 | 42.2 | 52.9 | 34.4 | **36.2** | 45.4 | **51.0** | **64.7** | **45.4** | 38.6 | **55.4** |
| | $5.5\times10^{21}$ | $8.3\times10^{21}$ | 82.0 | **42.7** | 53.9 | 35.8 | 35.6 | 45.5 | **51.0** | **64.5** | **45.3** | **39.2** | **54.9** |
| | $9.0\times10^{21}$ | $1.17\times10^{22}$ | 84.4 | 41.6 | 53.0 | 34.9 | 35.0 | 43.6 | **50.7** | **64.2** | **45.4** | 37.9 | **55.5** |
| ◆ MetaCLIP, 0.1× LR | $1.4\times10^{21}$ | $4.1\times10^{21}$ | 80.4 | **43.1** | 53.9 | **37.1** | 32.9 | **48.4** | **50.8** | **63.9** | **45.3** | **39.4** | **54.6** |
| ● Scratch encoder | – | $2.7\times10^{21}$ | – | **43.5** | **56.3** | 35.5 | **37.2** | 45.0 | **50.6** | **64.3** | **46.2** | 36.3 | **55.5** |

# D Dedicated vision capacity

This section describes per-category breakdowns of evaluation metrics and experimental details for Section 4.

## D.1 Encoder advantage by VQA category

Figure 14 splits the encoder-free deficit compared to the from-scratch encoder by VQA category. The deficit is largest on OCR and chart questions at every budget at both 1.7B and 4B backbone sizes. Avg. Text stays within one point (Figure 3). Appendix I gives every VQA and text benchmark for the runs of each main-text exhibit.

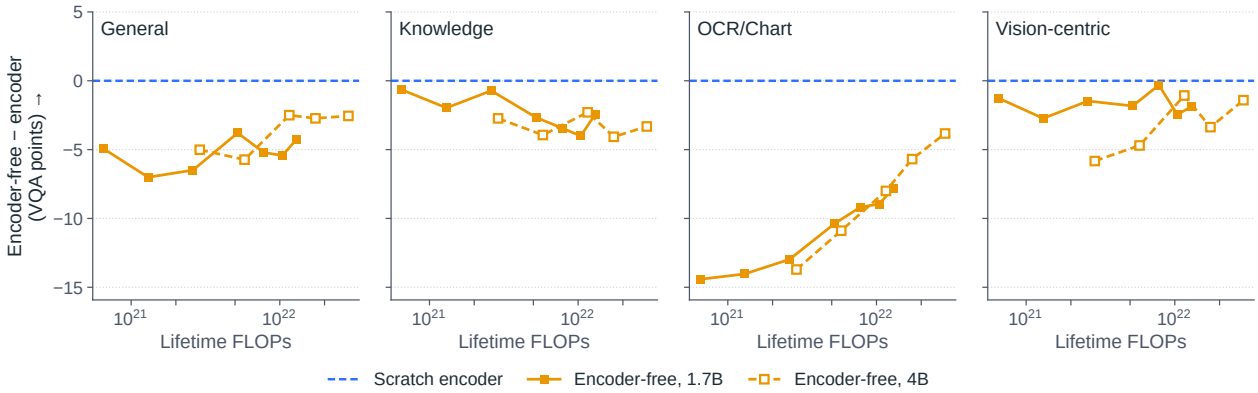



**Figure 14 The encoder's advantage over encoder-free is primarily in OCR and chart questions.** Each point compares encoder-free with the encoder at one budget of Table 27.

## D.2 Encoder-free with increased shared capacity

**Parameter-matched encoder-free.** The parameter-matched encoder-free baselines in Table 2 give the encoder-free backbone 0.4B extra parameters. The default 1.7B backbone has 28 layers of hidden size 2,048, the wider backbone has hidden size 2,304, the deeper backbone has 36 layers, and the wider and deeper backbone has 30 layers of hidden size 2,176, each with about 2.1B parameters. Table 19 shows how the wider variant does at 4B, using 122-TPP runs of Table 27. Compared to the default 4B backbone with 36 layers of hidden size 2,560, the wider backbone has hidden size 2,688 and 4.4B parameters. Here, wider means a larger hidden size, whereas the fixed-token study in Table 20 widens only the FFNs and keeps the hidden size at 2,048.

**Table 19 At 4B, a wider encoder-free backbone still trails the encoder on VQA.** All variants are trained to 122 TPP, i.e., $1.16 \times 10^{22}$ FLOPs. Bold: within one point of the best.

| Architecture | Added params (B) | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Variant | Vision-only | Shared | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| • Scratch encoder | 0.4 | – | **51.9** | **63.0** | **46.0** | **45.5** | **53.1** | **57.9** | **68.6** | **50.5** | **52.3** | **60.2** |
| ▪ Encoder-free | – | – | 48.4 | 60.5 | 43.8 | 37.5 | 52.0 | 57.0 | **68.3** | 50.0 | **52.3** | 57.3 |
| + wider | – | 0.4 | 48.1 | 59.7 | 43.4 | 39.1 | 50.2 | **58.3** | **68.7** | **51.2** | **52.3** | **61.0** |

**Parameter-matched encoder-free at a fixed token count.** Table 2 matches lifetime compute, so each larger encoder-free variant trains on fewer tokens than the encoder. We therefore also train 1.7B encoder-free backbones with wider FFNs or more layers on the encoder's token count, which gives each encoder-free variant more compute than the encoder (Table 20). We evaluate these runs at a constant learning rate, without a final decay, so we compare rows here only within Table 20. At approximately the encoder's parameter count, added FFN width or depth still leaves the backbone 2–3 Avg. VQA points behind the encoder. Wider or deeper variants both narrow the gap, but wider FFNs need about twice the encoder's parameters to match it, and the deeper backbone still trails it slightly. Every wider or deeper variant scores higher than the encoder on Avg. Text, consistent with how the shared parameters can also be used by language.

## D.3 Richer patch embeddings

The default encoder-free patch embedding is a single linear projection followed by RMSNorm and a learned gate (Appendix A.1). We compare it with two richer patch embedding designs: a normalized two-layer

**Table 20 Shared capacity needs about twice the encoder's parameters to match it on VQA.** All variants use a 1.7B backbone, train on the encoder's token count, and use a constant learning rate.

| Architecture | | | Added params (B) | | Compute | VQA (%) ↑ | Text (%) ↑ |
|---|---|---|---|---|---|---|---|
| Variant | Layers | FFN width | Vision-only | Shared | FLOPs | Avg | Avg |
| • Scratch encoder | 28 | 6,144 | 0.4 | – | $4.7\times10^{21}$ | 44.2 | 50.6 |
| ▪ Encoder-free | 28 | 6,144 | – | – | $4.7\times10^{21}$ | 40.2 | 49.8 |
| + wider FFN | 28 | 9,024 | – | 0.5 | $5.6\times10^{21}$ | 41.1 | 51.5 |
| + wider FFN | 28 | 10,752 | – | 0.8 | $6.3\times10^{21}$ | 44.4 | 51.7 |
| + wider FFN | 28 | 11,904 | – | 1.0 | $6.7\times10^{21}$ | 44.6 | 53.3 |
| + deeper | 36 | 6,144 | – | 0.4 | $5.6\times10^{21}$ | 41.8 | 52.8 |
| + deeper | 44 | 6,144 | – | 0.8 | $6.6\times10^{21}$ | 43.6 | 54.7 |

projection inspired by Gemma 4 (Gemma Team, 2026), and a four-layer hierarchical MLP (Touvron et al., 2022) inspired by Inkling (Thinking Machines Lab, 2026a). The lower part of Table 21 describes each variant. The normalized two-layer projection changes patch size, resolution, position embeddings, and normalization together. It normalizes each patch with LayerNorm (Ba et al., 2016), projects it linearly, adds learned factorized 2D position embeddings, and then applies LayerNorm, a parameter-free RMSNorm, and a linear map into the backbone. The hierarchical MLP has four linear layers that fold neighboring pixels into channels in stages, with RMSNorm and GELU (Hendrycks and Gimpel, 2016) between them, a final RMSNorm, and no position embedding. In Table 21, both richer variants read larger images in larger patches ($768\times768$ in $48\times48$ patches and $640\times640$ in $40\times40$ patches, rather than $224\times224$ in $14\times14$ patches), so they produce the same $16\times16$ grid of 256 vision tokens as the encoder. At $5.2\times10^{21}$ FLOPs, both narrow the gap to the encoder, and the hierarchical MLP matches it on Avg. VQA (Table 21). The largest gain is on OCR and chart questions, where the default patch embedding trails the encoder most. Avg. Text is comparable across patch embeddings.

However, when training on $224\times224$ images at $2.6\times10^{21}$ FLOPs, the two-layer projection gains 1.5 Avg. VQA over the default patch embedding whereas the hierarchical MLP loses 3.1, and both remain far behind the encoder (Table 2). The hierarchical MLP helps only when it compresses a higher-resolution image into the same number of tokens. With $224\times224$ instruction tuning, it matches the default patch embedding on $224\times224$ images, while processing $640\times640$ images into the same 256 tokens raises it to 38.8, mostly on OCR and chart questions (24.0 against 10.2). Raising only the instruction-tuning resolution to $448\times448$ quadruples the tokens without compressing them, and the hierarchical MLP then gains 1.4 points, against 4.4 for the default patch embedding. The richer patch embeddings therefore help most with higher-resolution images.

## D.4 Comparison with prior work

Prior work, Shukor et al. (2025), find that encoder-free models match or slightly outperform an encoder trained from scratch, whereas we find that the encoder leads (Section 4). To investigate this discrepancy, we examine two choices in their setup that may work against the encoder, a reduced encoder learning rate and using the averaged validation loss. We also study a third difference, using alt-text captions rather than regenerated captions. First, we evaluate the runs of the scaling ladder (Appendix G) on their averaged loss. We also retrain the ladder's 0.6B and 1B backbones, sizes within the range of their study, with a reduced encoder learning rate, alt-text captions, or both (Table 22). Every other setting matches the ladder's runs, which serve as baselines.

**Encoder learning rate.** Shukor et al. (2025) train their from-scratch encoder at a reduced learning rate, the best of the 0.1×, 0.01×, and 0.001× multipliers they compare. We train the encoder at 0.01× the backbone's peak learning rate, and also at 0.1× at 1B, with the projector at the full rate. Instruction tuning trains every encoder at its full rate. We find that lowering the rate shrinks the encoder's lead (Table 22), and most of the drop comes between the full rate and 0.1×, whereas their sweep starts at 0.1× and never trains the encoder at the full rate. At 0.01×, the encoder is comparable to encoder-free on Avg. VQA at 1B and loses its caption-loss lead entirely at 0.6B.

**Table 21 Richer patch embeddings narrow the gap to the encoder only when processing higher-resolution images.** All variants train a 1.7B backbone at 315 TPP to $5.2 \times 10^{21}$ FLOPs. VQA uses $224 \times 224$ instruction tuning. Bold: within one point of the best. Each architecture variant is described in the table below.

| Architecture | Params | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Variant | Vision-only (M) | Total (B) | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| • Scratch encoder | 401 | 2.1 | **40.2** | **54.6** | 34.5 | **26.9** | **44.9** | **52.5** | **65.0** | **47.2** | 41.5 | **56.4** |
| ▪ Encoder-free | 1 | 1.7 | 36.9 | 50.8 | 34.3 | 17.5 | **45.2** | **52.6** | **65.3** | **46.6** | 42.9 | **55.6** |
| Two-layer projection | 18 | 1.7 | 38.8 | 52.8 | 34.4 | 23.7 | **44.2** | **52.3** | **65.9** | **46.7** | **43.5** | 53.2 |
| Hierarchical MLP | 44 | 1.8 | **40.2** | **54.2** | **35.9** | **26.7** | **44.2** | **52.8** | **65.1** | 46.1 | **44.3** | **55.7** |

| Architecture | | | | Input | | |
|---|---|---|---|---|---|---|
| Variant | Vision-only layers | Vision-only params (M) | Position embedding | Patch size | Resolution | Vision tokens |
| • Scratch encoder | 27 Transformer blocks | 401 | learned + 2D RoPE | $14 \times 14$ | $224 \times 224$ | 256 |
| ▪ Encoder-free | 1 linear | 1 | fixed 2D sine–cosine | $14 \times 14$ | $224 \times 224$ | 256 |
| Two-layer projection | 2 linear | 18 | learned factorized 2D | $48 \times 48$ | $768 \times 768$ | 256 |
| Hierarchical MLP | 4 linear | 44 | none | $40 \times 40$ | $640 \times 640$ | 256 |

**Averaged validation loss.** Shukor et al. (2025) fit their scaling laws to validation loss averaged over interleaved, caption, and text data, with captions that are web alt-text. The text loss does not depend on the image, and in our ladder the interleaved loss barely differs between the encoder and encoder-free. Differences between vision components therefore have a weaker effect on the average, which makes it a noisier signal for comparing them. In our scaling ladder, the averaged loss barely separates the encoder from encoder-free, whereas held-out caption loss separates them clearly and tracks Avg. VQA (Figure 18). At 1.7B and 128 tokens per parameter, the two reach the same averaged loss while the encoder leads by 4.8 Avg. VQA points (Figure 15).

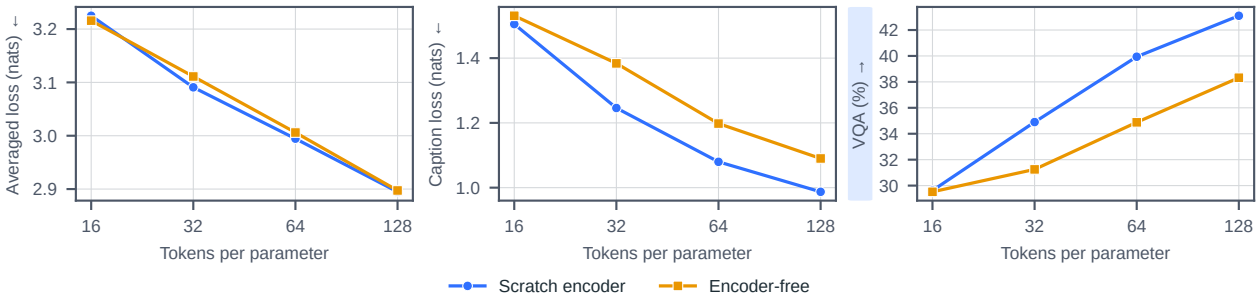



**Figure 15 Comparing on averaged loss misses the encoder's lead on caption loss and Avg. VQA.** All runs use the 1.7B model from the scaling ladder. The averaged loss is the mean of the alt-text caption, interleaved, and text losses.

**Alt-text captions.** We also train on the images' original alt-text instead of the regenerated captions, for the encoder and encoder-free alike. This delays the encoder's lead, as the two tie on alt-text caption loss up to 32 tokens per parameter. With both changes (a 0.01× encoder learning rate and alt-text captions), encoder-free reaches a lower alt-text caption loss than the encoder at both sizes and every budget, by 0.04 to 0.07 nats, as Shukor et al. (2025) report. Overall, their finding may reflect an under-trained encoder or an averaged loss that can hide its benefit.

**Table 22 Reducing encoder learning rate removes most or all of the encoder's lead.** All variants use the 1B backbone from the scaling ladder at 64 or 128 TPP, i.e., $3.7 \times 10^{20}$ or $7.3 \times 10^{20}$ FLOPs of lifetime compute. The encoder learning rate scales only the encoder's peak learning rate. Alt-text rows use the original alt-text instead of regenerated captions, in training and in the held-out loss. Bold: best in each column within each caption source.

| Setup | | Held-out caption loss ↓ | | Avg. VQA (%) ↑ | |
|---|---|---|---|---|---|
| Recipe | Encoder learning rate | 64 TPP | 128 TPP | 64 TPP | 128 TPP |
| *Regenerated captions* | | | | | |
| Scratch encoder | 1× (ours) | **1.229** | **1.096** | **32.9** | **36.3** |
| Scratch encoder | 0.1× | 1.347 | 1.211 | 30.6 | 32.8 |
| Scratch encoder | 0.01× (Shukor et al., 2025) | 1.372 | 1.230 | 28.8 | 31.2 |
| Encoder-free | – | 1.388 | 1.243 | 29.1 | 31.0 |
| *Alt-text captions* | | | | | |
| Scratch encoder | 1× (ours) | **3.167** | **2.826** | **29.4** | **35.5** |
| Scratch encoder | 0.01× (Shukor et al., 2025) | 3.291 | 3.167 | 27.6 | 28.5 |
| Encoder-free | – | 3.247 | 3.126 | 26.9 | 27.3 |

# E In-fusion vision capacity

This section describes the in-fusion vision capacity variants of Section 5, which add vision-only modules inside the backbone.

## E.1 Implementation and sizing

In each augmented layer, in-fusion (FFN-only) adds a vision-only SwiGLU branch of width $F$ beside the shared FFN, reading the same normalized input. For backbone width $D$ and $L_a$ augmented layers, it adds $P_{\text{FFN}} = L_a(3DF + D)$ parameters. In-fusion also adds zero-initialized vision-only query, key, value, and output projections with the shapes of the shared ones. For $H_Q$ query heads, $H_{KV}$ key–value heads, and head dimension $d_h$, they add $P_{\text{QKVO}} = 2L_a D d_h(H_Q + H_{KV})$ parameters. Attention runs once over the combined projections, so there is no second attention stream. Unless otherwise noted, every allocation augments the first eight layers ($L_a = 8$), and Table 23 lists the resulting widths and parameter counts.

The depth ablation uses in-fusion (FFN-only) because its branch can be narrowed to fit the budget in any number of layers. Vision-only attention projections have the shapes of the shared ones, so in every layer they would take 0.35B of the 0.4B budget at 1.7B (28 layers) and 0.94B at 4B (36 layers), more than the whole budget.

**Table 23 The in-fusion variant is matched with in-fusion (FFN-only).** Both have a fixed added-parameter budget. Note that the proportion spent on vision-only attention projections grows with backbone width. All rows are checkpoints before the learning rate decay, at $2.34 \times 10^{21}$ FLOPs (1.7B) and $1.04 \times 10^{22}$ FLOPs (4B) of lifetime compute. Bold: within one point of the better allocation at each backbone scale.

| Architecture | | Added params (M) | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Variant | FFN width | FFN | QKVO | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| *1.7B backbone* | | | | | | | | | | | | | |
| ▲ In-fusion (FFN-only) | 8,192 | 402.7 | 0.0 | 41.5 | 53.8 | 33.2 | **33.9** | 45.1 | **48.8** | **62.5** | **44.4** | 35.9 | **52.6** |
| ▼ In-fusion | 6,144 | 302.0 | 100.7 | **43.1** | **54.9** | **36.7** | **33.8** | **47.0** | **48.7** | **62.2** | **43.4** | **37.1** | **51.9** |
| *4B backbone* | | | | | | | | | | | | | |
| ▲ In-fusion (FFN-only) | 6,528 | 401.1 | 0.0 | **49.3** | 59.0 | **43.8** | **40.0** | **54.2** | **55.8** | **66.5** | **49.0** | **49.5** | 58.1 |
| ▼ In-fusion | 3,136 | 192.7 | 209.7 | **49.4** | **61.9** | **43.9** | **40.5** | 51.2 | **56.7** | **67.3** | **49.5** | **49.8** | **60.0** |

## E.2 Routing, depth, and attention-projection controls

In Table 3, shared FFN widening grows the shared FFN of layers 1–8 from width 6,144 to 14,336. The FFN branches have width 8,192 when placed in eight layers and 2,304 when placed in all 28.

**Table 24 Reserving the FFN branch for vision tokens keeps its lead with twice the training.** All rows use the 1.7B backbone, decayed at the lifetime compute of their block. Both FFN branches add 0.4B parameters to layers 1–8 of the encoder-free backbone. Bold: within one point of the best in each block.

| Architecture | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Variant | Used by | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| $2.6 \times 10^{21}$ *FLOPs (158 TPP)* | | | | | | | | | | | |
| • Scratch encoder | Vision | **44.4** | **57.2** | 35.7 | **38.2** | 46.5 | 49.7 | **63.9** | 45.0 | 36.7 | 53.4 |
| ▪ Encoder-free | – | 39.0 | 50.7 | 35.0 | 25.2 | 45.0 | **50.7** | **64.1** | **45.8** | 38.2 | **54.8** |
| Separate FFN branch | All | 42.3 | 54.1 | **37.4** | 30.8 | 46.7 | **51.2** | **64.8** | **46.3** | **41.6** | 52.0 |
| ▲ In-fusion (FFN-only) | Vision | **45.0** | **57.0** | **36.7** | 36.0 | **50.4** | **50.4** | 63.0 | **45.3** | **40.6** | 52.9 |
| $5.2 \times 10^{21}$ *FLOPs (315 TPP)* | | | | | | | | | | | |
| • Scratch encoder | Vision | **46.5** | **58.5** | **38.9** | **39.4** | 49.4 | **52.5** | 65.0 | **47.2** | 41.5 | **56.4** |
| ▪ Encoder-free | – | 41.9 | 54.7 | 36.3 | 29.0 | 47.6 | **52.6** | 65.3 | **46.6** | 42.9 | **55.6** |
| Separate FFN branch | All | 45.4 | 57.3 | **39.7** | 34.9 | **49.7** | **53.5** | **66.5** | **46.4** | **46.6** | 54.5 |
| ▲ In-fusion (FFN-only) | Vision | **46.7** | **59.3** | **38.9** | 38.1 | **50.5** | **52.7** | 64.8 | **47.2** | **46.3** | 52.6 |

**Routing at matched compute.** In-fusion (FFN-only) and the separate FFN branch differ only in which tokens use the branch. Because the all-token branch also runs on text tokens, it trains on fewer tokens at matched compute. Routing the branch to vision tokens leads on Avg. VQA at both budgets, while the all-token branch is slightly ahead on Avg. Text (Table 24). At the longer budget, routing also raises downstream ImageNet probe accuracy from 61.2 to 66.0 (Appendix B.5). With the same parameters, the separate FFN branch beats shared FFN widening on Avg. VQA (Table 3).

**Attention-projection controls.** In two 4B contrasts at matched training tokens, the vision-only attention projections improve visual understanding both on their own and alongside an FFN branch (Table 25). The first adds only the attention projections of in-fusion to the encoder-free model, and the second compares in-fusion with a model that has only its FFN branch.

**Table 25 Vision-only attention projections improve visual understanding both alone and alongside an FFN branch.** Each row is the difference between two 4B models trained on the same number of tokens, before the final decay. Caption-loss changes are in nats, where negative is better.

| Contrast | VQA (%) ↑ | Other metrics | |
|---|---|---|---|
| | Δ Avg [95% CI] | Δ OOD caption loss | Δ ImageNet top-1 |
| QKVO-only − encoder-free | +1.8 [1.0, 2.5] | −0.059 | +3.73 |
| In-fusion − same-width FFN branch | +1.1 [0.4, 1.9] | −0.013 | +0.75 |

## E.3 Comparison at longer training

At the longest training budget, 1.11T token-equivalents, in-fusion is comparable to the encoder on both Avg. Text and Avg. VQA at both model scales (Table 26). In-fusion (FFN-only) is comparable to the encoder at 1.7B and slightly behind it on Avg. VQA at 4B, where it also trails in-fusion at every budget (Figure 5).

## E.4 Where vision tokens are processed

To see where each design processes vision tokens, we probe the 1.7B models at the longest training budget (Figure 16), adapting the probes of Chen et al. (2026). For each backbone block, we measure how much the vision tokens change as one minus the cosine similarity between their hidden state before and after the block, and how much attention caption tokens place on them. In both in-fusion models, vision tokens change mostly in the blocks with vision-only modules and little after them. The encoder-free model keeps changing them

**Table 26 In-fusion is comparable to the scratch encoder at longer training budgets.** Both model scales train to 1.11T token-equivalents. Bold: within one point of the best at each scale.

| Architecture | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Model | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| *1.7B backbone,* $1.30\times10^{22}$ *FLOPs* | | | | | | | | | | |
| ● Scratch encoder | **48.9** | **60.6** | 41.3 | **42.5** | 51.2 | **55.3** | **66.2** | **48.1** | 49.3 | 57.6 |
| ■ Encoder-free | 44.8 | 56.4 | 38.8 | 34.7 | 49.3 | **55.7** | **66.5** | **48.1** | 48.6 | **59.4** |
| ▲ In-fusion (FFN-only) | **48.6** | **60.1** | 40.9 | 40.7 | **52.8** | **55.6** | **66.1** | **48.5** | **50.5** | 57.3 |
| ▼ In-fusion | **49.2** | **60.0** | **43.0** | **41.6** | **52.1** | **55.9** | **66.9** | **48.5** | 49.3 | **59.0** |
| *4B backbone,* $2.90\times10^{22}$ *FLOPs* | | | | | | | | | | |
| ● Scratch encoder | **54.5** | **66.4** | **49.1** | **45.5** | **57.0** | **60.4** | **69.5** | 51.5 | **57.5** | **63.0** |
| ■ Encoder-free | 51.7 | 63.9 | 45.8 | 41.7 | 55.6 | **61.0** | **69.5** | **52.9** | **58.3** | **63.3** |
| ▲ In-fusion (FFN-only) | 53.4 | 65.1 | 46.9 | **45.3** | 56.5 | **60.2** | **70.0** | **52.2** | 56.4 | **62.4** |
| ▼ In-fusion | **54.3** | **65.6** | 47.9 | **45.7** | **57.9** | **60.7** | **69.2** | **52.6** | **57.8** | **63.1** |

through its early and middle shared blocks. The encoder's tokens enter the backbone already processed, so they change mostly in the first few blocks. After the in-fusion blocks, caption tokens attend to vision tokens more than in the encoder and encoder-free models. The same pattern holds at a fifth of the training budget. These probes suggest that in-fusion capacity performs vision-specific processing in its own early blocks and passes stable vision tokens to the shared backbone, whereas an encoder-free model builds them up across its shared layers.

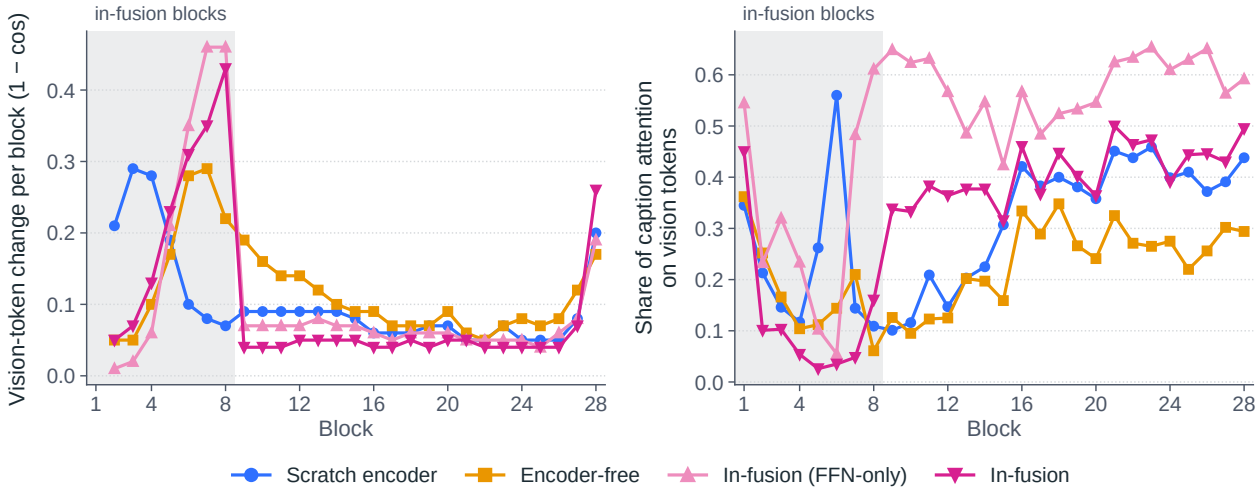



**Figure 16 In-fusion capacity confines vision processing to the earliest blocks, while encoder-free spreads it across shared layers.** Vision-token change is defined as $1 - \cos(h_L, h_{L-1})$ between consecutive blocks. The shaded band marks blocks 1–8, where the in-fusion modules sit. All models use the 1.7B backbone at $1.30 \times 10^{22}$ FLOPs of lifetime compute.

# F Decay, learning rate, and caption loss

This section lists the evaluation performance of each endpoint (post-decay) of our main comparisons, shows that the shared peak learning rate (Section 2.4) does not decide the comparisons, and relates held-out caption loss to Avg. VQA (Section 4).

## F.1 Decayed endpoints

Table 27 lists the evaluation performance of each recipe's endpoint at each budget of Section 2.4, next to a text-only reference trained to the same lifetime compute. Each endpoint is evaluated after learning rate decay.

**Table 27 Encoder-free trails the encoder at every compute budget, while in-fusion is comparable to it.** Enc is the from-scratch encoder, EF encoder-free, and FFN and QKVO mark in-fusion (FFN-only) and in-fusion. All models in a row share the same lifetime compute. Dashes mark models not trained to that budget. Bold: within one point of the best in each row, excluding the text-only reference.

| Budget | | VQA (%) ↑ | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| FLOPs | TPP | Enc | EF | FFN | QKVO | Text-only | Enc | EF | FFN | QKVO |
| *1.7B backbone* | | | | | | | | | | |
| $6.5\times10^{20}$ | 39 | **37.3** | 32.0 | 36.3 | **37.4** | – | 45.3 | **46.4** | **45.9** | **45.7** |
| $1.30\times10^{21}$ | 79 | **41.2** | 34.8 | **41.4** | **40.4** | 48.9 | 47.3 | **48.3** | **48.4** | **48.1** |
| $2.60\times10^{21}$ | 158 | **44.4** | 39.0 | **45.0** | 42.6 | 52.4 | **49.7** | **50.7** | **50.4** | **50.7** |
| $5.21\times10^{21}$ | 315 | **46.5** | 41.9 | **46.7** | **46.5** | 54.4 | **52.5** | **52.6** | **52.7** | **53.4** |
| $7.81\times10^{21}$ | 473 | **47.5** | 43.0 | – | – | 54.9 | **54.4** | **53.7** | – | – |
| $1.04\times10^{22}$ | 631 | **49.0** | 43.8 | – | – | 56.5 | **54.3** | **55.3** | – | – |
| $1.30\times10^{22}$ | 788 | **48.9** | 44.8 | **48.6** | **49.2** | 56.5 | **55.3** | **55.7** | **55.6** | **55.9** |
| *4B backbone* | | | | | | | | | | |
| $1.45\times10^{21}$ | 15 | – | – | – | – | 51.2 | – | – | – | – |
| $2.90\times10^{21}$ | 31 | **47.8** | 41.0 | 44.8 | 45.5 | 54.8 | **52.0** | **51.5** | 50.8 | **51.9** |
| $5.79\times10^{21}$ | 61 | **51.2** | 44.9 | 48.4 | 49.0 | 57.4 | **56.4** | **56.0** | **55.8** | **55.9** |
| $1.16\times10^{22}$ | 122 | **51.9** | 48.4 | 49.8 | **51.8** | 59.0 | **57.9** | **57.0** | **57.1** | **57.7** |
| $1.74\times10^{22}$ | 183 | **54.2** | 50.2 | – | – | – | **58.9** | **59.2** | – | – |
| $2.90\times10^{22}$ | 306 | **54.5** | 51.7 | 53.4 | **54.3** | – | **60.4** | **61.0** | **60.2** | **60.7** |

## F.2 Learning rate

All recipes share one peak learning rate per backbone size, a choice that could bias compute comparisons (Porian et al., 2024). Therefore, at 1.7B backbone size, we repeat the encoder, in-fusion, and encoder-free recipes at half and twice the peak learning rate (encoder-free also at four times), at 39 and 79 TPP (Figure 17). At every learning rate, the encoder leads encoder-free and in-fusion matches or exceeds the encoder. Higher learning rates raise Avg. Text for all three recipes, and have lesser effects on Avg. VQA. The shared learning rate therefore does not decide the visual-understanding comparisons of Sections 4 and 5.

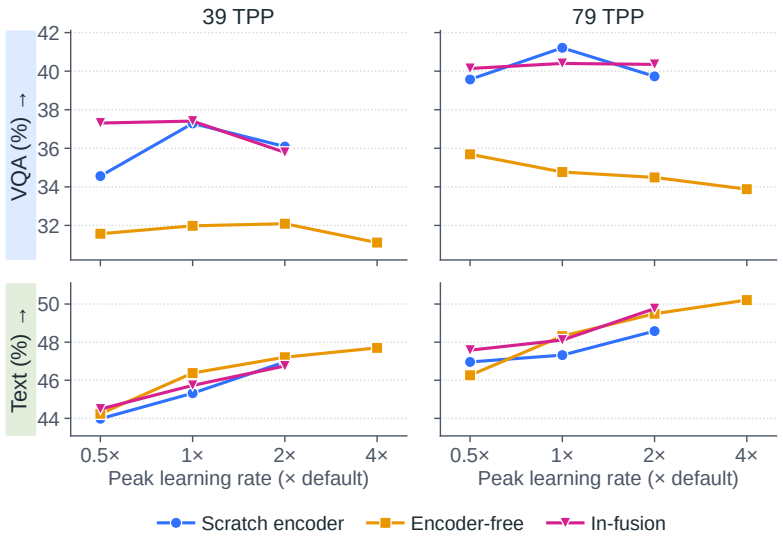



**Figure 17 The encoder maintains its lead over encoder-free, and in-fusion is comparable to the encoder, at every peak learning rate.** All models are 1.7B and trained to 39 and 79 TPP, i.e., $6.5\times10^{20}$ and $1.3\times10^{21}$ FLOPs of lifetime compute.

### F.3 Held-out caption loss

We find that across all four of the recipes, caption loss tracks Avg. VQA along one trend ($r = -0.96$; Figure 18). By category, OCR and chart questions track it most closely.

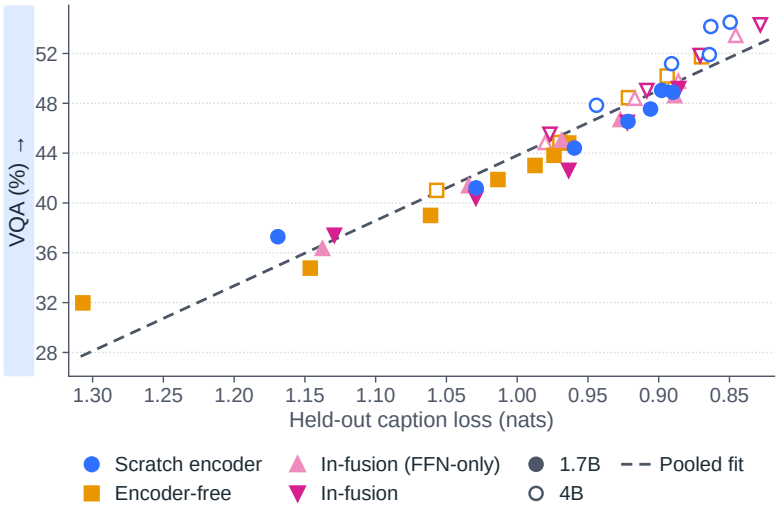



**Figure 18 Held-out caption loss tracks Avg. VQA along one trend shared by all four recipes.** Held-out caption loss uses images never seen in training. The line is a least-squares fit over all endpoints. All points are evaluated after learning rate decay.

## G Scaling ladder

Section 5.4 tests whether the comparisons of Sections 4 and 5 hold as the backbone grows, using a ladder of backbone sizes. This section gives the ladder's setup, the scaling laws fitted to it, and how well the laws predict runs they were not fitted on.

**Setup.** The ladder trains the encoder-free, encoder, and in-fusion recipes under the protocol of Section 2 on backbones from 0.6B to 6.4B parameters. The backbones follow the architecture of the 1.7B backbone, with query heads of dimension 128 spanning the hidden size, half as many key–value heads, and a feed-forward width three times the hidden size. The vision module grows with the backbone (Table 28). The encoder keeps the proportions of the 1.7B pairing: half the backbone's hidden size $d$, one layer fewer than the backbone, an MLP four times its own width, and $\mathrm{round}(0.75\,d/128)$ attention heads of dimension 128. Its parameter count therefore stays at a nearly fixed ratio to the backbone's non-embedding parameters. The in-fusion modules occupy the first $\mathrm{round}(8L/28)$ of the backbone's $L$ layers, 29% of the depth, as at 1.7B. Their widths are set so that they match the encoder's parameter count.

All recipes at a backbone size share one peak learning rate and one batch size (Table 28). The peak learning rate is $3 \times 10^{-4}\sqrt{1.72 \times 10^{9}/N_{\mathrm{tot}}}$, rounded, where $N_{\mathrm{tot}}$ is the total backbone parameter count. The batch size grows by about 1.5 times per backbone size. Warmup lasts 2 tokens per non-embedding backbone parameter (2 TPP), e.g., 2.8B tokens for the 1.7B backbone, which has 1.4B non-embedding parameters. The other settings follow Table 8. The 1.7B and 4B runs of Sections 3 to 5 instead keep a batch of 1,024 sequences and 4,000 warmup updates at both sizes. At 1.7B, where the ladder's batch is also 1,024, each ladder recipe is within two Avg. VQA points of those runs when interpolated to their budgets (Sections 4 and 5).

Each recipe and size has one constant-learning-rate run, decayed over the final 10% of training at several budgets (Table 29). Each point in Figures 7 and 19 is one decayed endpoint. At 1.7B, the ladder's encoder is the 0.4B encoder of Section 2. At 6.4B, we also train the encoder recipe with the 0.4B encoder of the main runs, to check that this smaller encoder does not limit larger backbones (Appendix A.2). Table 29 lists the downstream scores of every run the laws are fitted on.

**Scaling laws.** For Avg. VQA, caption loss, and text loss, we fit a separate law for each recipe. For the losses it takes the form $L(N, D) = E + A\,N^{-\alpha} + B\,D^{-\beta}$ (Hoffmann et al., 2022), with $N$ the non-embedding backbone parameters and $D$ the token-equivalents (Section 2.3), fitted by minimizing a Huber loss on log loss. For Avg. VQA, it is linear in $\ln N$ and $\ln D$, fitted by least squares. Each is fitted on the recipe's 17 ladder runs (Table 30). From 1.7B to 6.4B, encoder-free needs 2.6 to 3.1 times the encoder's compute on Avg. VQA, 2.4 to 3.8 times on caption loss, and about the same on text loss. In-fusion needs about the same compute as the encoder on Avg. VQA and text loss, and slightly less on caption loss. Table 31 gives the scaling laws' errors on every run and on each left-out backbone size. Fitted on the ladder alone, the caption-loss laws also predict the separately trained 1.7B and 4B runs of Sections 3 to 5 with an average absolute error of 1.6% (Table 32). Their warmup differs from the ladder's, and at 4B, so do the batch size and the size of the vision module. Figure 19 shows each recipe's caption-loss law at every backbone size against the runs it is fitted on.

**Table 28 Architectures and optimization settings of the scaling ladder.** The vision module grows with the backbone size. Ratio is the vision-only parameter count over the backbone's non-embedding count, which stays at around 0.29. The in-fusion columns describe the FFN+QKVO variant (Section 5.1). Batch counts sequences of 4,096 tokens, and warmup counts updates. Backbones are named by total parameters, including the tied embedding.

| | Backbone | | Encoder | | | In-fusion | | Optimization | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Non-emb. (M) | Width / layers | Width / layers | Vision-only (M) | Ratio | Layers | Width | Peak LR | Batch | Warmup |
| 0.6B | 393 | 1,280 / 20 | 640 / 19 | 115 | 0.29 | 1–6 | 3,584 | $5.1 \times 10^{-4}$ | 448 | 429 |
| 1B | 793 | 1,536 / 28 | 768 / 27 | 227 | 0.29 | 1–8 | 4,544 | $3.9 \times 10^{-4}$ | 640 | 605 |
| 1.7B | 1,409 | 2,048 / 28 | 1,024 / 27 | 401 | 0.28 | 1–8 | 6,144 | $3.0 \times 10^{-4}$ | 1,024 | 672 |
| 3.2B | 2,831 | 2,560 / 36 | 1,280 / 35 | 809 | 0.29 | 1–10 | 7,936 | $2.2 \times 10^{-4}$ | 1,536 | 900 |
| 6.4B | 5,848 | 3,328 / 44 | 1,664 / 43 | 1,693 | 0.29 | 1–13 | 9,664 | $1.6 \times 10^{-4}$ | 2,304 | 1,239 |

**Table 29 In-fusion is comparable to the encoder at every size, whereas encoder-free trails both on VQA but stays comparable on text.** This shows metrics for every scaling ladder run (Figure 19). Bold: within one point of the best in each row.

| Setup | | | VQA (%) ↑ | | | Text (%) ↑ | | |
|---|---|---|---|---|---|---|---|---|
| Backbone | TPP | FLOPs | Scratch encoder | Encoder-free | In-fusion | Scratch encoder | Encoder-free | In-fusion |
| 0.6B | 128 | $2.09\times10^{20}$ | **31.6** | 26.1 | **30.7** | **43.3** | **42.4** | **42.7** |
| | 256 | $4.18\times10^{20}$ | **33.4** | 27.9 | **32.6** | **44.1** | 43.3 | **44.4** |
| 1B | 64 | $3.66\times10^{20}$ | **32.9** | 29.1 | **33.6** | **43.3** | **43.5** | **44.0** |
| | 128 | $7.32\times10^{20}$ | **36.3** | 31.0 | **36.0** | **46.1** | **45.7** | **45.7** |
| | 256 | $1.46\times10^{21}$ | **40.1** | 33.4 | 38.0 | **47.6** | **47.9** | **47.3** |
| 1.7B | 32 | $5.29\times10^{20}$ | 34.9 | 31.2 | **36.1** | **46.4** | **45.6** | **46.0** |
| | 64 | $1.06\times10^{21}$ | **39.9** | 34.9 | 38.4 | **48.2** | **48.3** | **48.1** |
| | 128 | $2.12\times10^{21}$ | **43.1** | 38.3 | **42.2** | **50.5** | **50.4** | **50.5** |
| | 256 | $4.23\times10^{21}$ | **44.8** | 41.0 | 43.5 | **52.4** | **52.8** | **52.7** |
| 3.2B | 16 | $9.78\times10^{20}$ | 36.0 | 31.7 | **39.3** | **48.1** | 46.9 | 46.5 |
| | 32 | $1.96\times10^{21}$ | 41.3 | 35.7 | **42.8** | **50.4** | **51.1** | 49.8 |
| | 64 | $3.91\times10^{21}$ | **46.4** | 39.9 | **45.9** | **53.0** | **53.7** | **52.7** |
| | 128 | $7.82\times10^{21}$ | **50.6** | 43.6 | **50.2** | **55.4** | **56.2** | **55.9** |
| 6.4B | 16 | $3.90\times10^{21}$ | **46.0** | 40.4 | **46.8** | **53.0** | **52.6** | **52.1** |
| | 32 | $7.81\times10^{21}$ | **50.2** | 44.3 | 48.9 | **56.4** | **56.4** | **56.2** |
| | 64 | $1.56\times10^{22}$ | **52.8** | 47.5 | **52.0** | **59.1** | **58.9** | **59.1** |
| | 128 | $3.12\times10^{22}$ | **54.6** | 49.1 | **54.2** | **61.4** | **60.7** | **61.5** |

**Table 30 Encoder-free needs 2.6 to 3.1 times the encoder's compute on Avg. VQA, while in-fusion needs about as much as the encoder.** $N$ and $D$ are in billions. Each factor is the compute at which a recipe's law reaches a metric value, divided by the compute at which the encoder's law reaches the same value. The factors are read at the compute of each run at 64 TPP from 1.7B to 6.4B.

| Metric | Recipe | Law | Factor |
|---|---|---|---|
| Avg. VQA | Scratch encoder | $13.7 + 4.40 \ln N + 5.25 \ln D$ | – |
| | Encoder-free | $10.8 + 4.40 \ln N + 4.75 \ln D$ | 2.6–3.1 |
| | In-fusion | $19.3 + 5.29 \ln N + 3.94 \ln D$ | 1.0–1.1 |
| Caption loss | Scratch encoder | $E = 0.59$, $A = 0.28$, $\alpha = 0.27$, $B = 7.54$, $\beta = 0.77$ | – |
| | Encoder-free | $E = 0.70$, $A = 0.18$, $\alpha = 0.59$, $B = 4.18$, $\beta = 0.54$ | 2.4–3.8 |
| | In-fusion | $E = 0.63$, $A = 0.25$, $\alpha = 0.40$, $B = 3.39$, $\beta = 0.61$ | 0.9–1.0 |
| Text loss | Scratch encoder | $E = 1.08$, $A = 0.58$, $\alpha = 0.19$, $B = 1.03$, $\beta = 0.19$ | – |
| | Encoder-free | $E = 1.08$, $A = 0.58$, $\alpha = 0.19$, $B = 1.00$, $\beta = 0.19$ | 1.0 |
| | In-fusion | $E = 1.09$, $A = 0.57$, $\alpha = 0.19$, $B = 1.03$, $\beta = 0.19$ | 1.0 |

**Table 31 One law per recipe fits every run and predicts each left-out backbone size.** Fit gives the RMS and largest residual over all runs. Size out gives the mean and largest absolute error over all runs, each predicted by the laws refitted without its backbone size. The 6.4B out columns give the same over the 6.4B runs, predicted by the laws fitted on 0.6B to 3.2B.

| | Fit | | Size out | | 6.4B out | |
|---|---|---|---|---|---|---|
| Metric | RMS | Max | Mean | Max | Mean | Max |
| Avg. VQA (pts) | 0.93 | 2.27 | 1.03 | 3.40 | 0.95 | 1.78 |
| Caption loss (%) | 1.20 | 4.81 | 2.03 | 7.42 | 1.91 | 5.00 |
| Text loss (%) | 0.36 | 0.83 | 0.41 | 1.52 | 0.45 | 0.95 |

**Table 32 The ladder's caption-loss laws predict the separately trained 1.7B and 4B runs within a few percent.** Each entry is the measured held-out caption loss divided by the loss that the recipe's law of Table 30 predicts, minus one, in percent.

| | | Token-equivalents | | | | | |
|---|---|---|---|---|---|---|---|
| Backbone | Recipe | 111B | 222B | 444B | 667B | 889B | 1.11T |
| 1.7B | Scratch encoder | −1.6 | −0.4 | +0.6 | +0.9 | +1.2 | +1.0 |
| | Encoder-free | −2.7 | −1.3 | +0.9 | +1.4 | +1.9 | +2.1 |
| | In-fusion | −0.8 | −0.7 | −0.5 | – | – | −0.6 |
| 4B | Scratch encoder | −4.3 | −1.5 | +0.9 | +3.0 | – | +3.3 |
| | Encoder-free | −5.0 | −4.0 | −1.9 | −1.7 | – | −1.0 |
| | In-fusion | +0.8 | +0.6 | +1.4 | – | – | +0.6 |

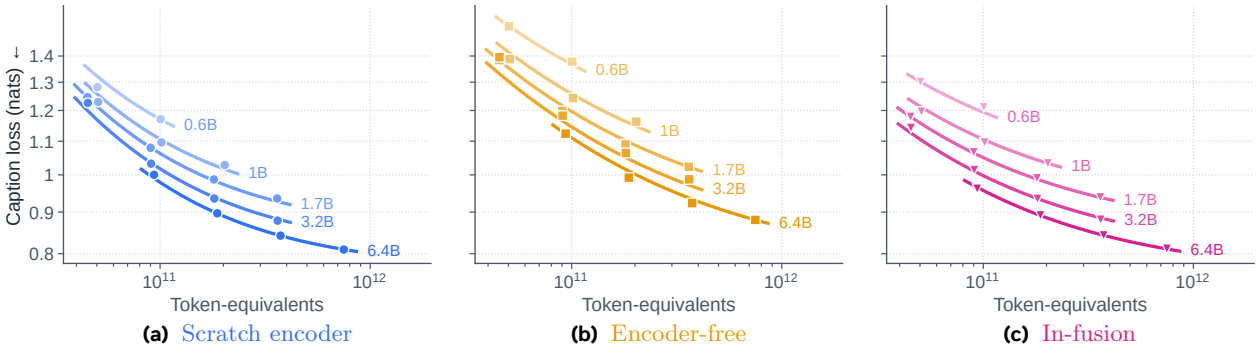



**Figure 19 A single caption-loss law per recipe fits the runs at every backbone size.** Points are the held-out caption loss of every ladder run. Each line is the recipe's law evaluated at one backbone size.

# H Beyond lifetime compute: midtraining and inference

This section describes the full midtraining comparison of Section 6.2 and the training and inference speed measurements behind Section 6.3.

## H.1 Midtraining

Table 33 provides the full midtraining comparison of Section 6.2 with per-category metrics. We also show the Avg. VQA of the three main recipes when trained with vision from the start. Each recipe's schedule lowers the text mixture proportion from 100% to 70% over the first 20B tokens and then maintains the 70:30 mixture. The pretrained MetaCLIP encoder stays frozen for the first 4,000 steps while its new linear projector and the backbone are updated. It then unfreezes with a 2,000-step learning rate warmup. Each midtraining run continues at a constant learning rate to 1.2T token-equivalents and then decays to 1.333T. All midtraining runs use $1.56 \times 10^{22}$ FLOPs of lifetime compute, including the backbone's text-only pretraining but not MetaCLIP's encoder pretraining. Vision-only parameters exclude an encoder's linear projection into the backbone and include the patch embedding of the other recipes.

**Table 33 In-fusion vision capacity and the from-scratch encoder maintain their VQA lead in midtraining.** From start shows VQA performance when training the same recipe with vision from the first step. Each recipe extends the 1.7B text-only model from 1T token-equivalents. Bold: within one point of the best.

| Recipe | | VQA (%) ↑ | | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Model | Vision-only params (B) | Avg | Gen | Knw | OCR | Vis | From start | Avg | Com | Sci | Knw | Rdg |
| • Scratch encoder | 0.40 | 48.6 | 58.8 | 42.0 | 38.9 | **54.5** | **49.3** | **56.7** | **67.5** | **48.9** | 50.5 | **60.0** |
| ▪ Encoder-free | <0.01 | 39.8 | 49.1 | 38.7 | 25.6 | 45.7 | 45.4 | **56.6** | **67.7** | **48.6** | **51.4** | 58.9 |
| ▾ In-fusion | 0.40 | **50.1** | **61.0** | **43.1** | **41.0** | **55.2** | **49.2** | **56.4** | 66.7 | 47.8 | **51.0** | **60.2** |
| MoT, layers 1–8 | 0.40 | 48.5 | **60.1** | 41.8 | 39.5 | 52.5 | – | **56.7** | 66.8 | **48.8** | **51.6** | 59.4 |
| MoT, all layers | 1.41 | 43.3 | 50.9 | 39.3 | 38.2 | 44.8 | – | **56.6** | **67.4** | **48.0** | **50.9** | **59.9** |
| ◆ MetaCLIP ("free") | 0.40 | 43.5 | 55.2 | 39.7 | 29.3 | 49.8 | – | **56.5** | **67.3** | **48.0** | **50.9** | **59.6** |
| Text-only control | – | – | – | – | – | – | – | **57.3** | **68.0** | **48.9** | **51.6** | **60.5** |

**Vision learning rate.** Table 33 uses the same peak learning rate for every parameter. Table 34 instead adjusts the learning rate of the vision path alone, while every other parameter keeps the shared rate. A higher vision learning rate brings the from-scratch encoder to within a point of in-fusion and partially closes MetaCLIP's gap to the from-scratch encoder, but the from-scratch encoder and in-fusion still lead.

## H.2 Training time

Here, we compare the training time of the encoder recipe with encoder-free, in-fusion (FFN-only), and in-fusion. Training time can be compared in two ways: per-step comparisons fix the number of optimizer updates, whereas equal-FLOP comparisons fix total modeled training compute. Because encoder computation counts toward that budget, the encoder recipe completes fewer updates at equal FLOPs, so its equal-FLOP time premium is smaller than its per-step premium.

We measure training time with complete optimizer updates on H200 GPUs with fully sharded data parallelism (FSDP) (Zhao et al., 2023). The matched 4B comparison in Table 35 runs the four recipes on 32 GPUs with a global batch of 1,024 sequences. Each row projects training time from measured steady-state update times to the same modeled lifetime compute, $1.04 \times 10^{22}$ FLOPs (400B token-equivalents at 4B; Appendix A.5).

Table 35 times the encoder and the in-fusion configurations of Table 23, which match the encoder's 0.4B vision-only parameters, under the 4B training setup above. At equal lifetime compute, neither in-fusion variant measurably changes training time relative to encoder-free. The encoder takes 2.9% longer than encoder-free and 2.6% longer than in-fusion at equal FLOPs. Per step, the encoder takes 5.3% longer than encoder-free. Note that the encoder used here is an unoptimized reference implementation, and encoder-specific optimization can narrow its gap. For instance, a compiled (Ansel et al., 2024) version of the Base encoder in Table 5 trains within 0.2% of an in-fusion (FFN-only) model with the same vision-only parameters at 4B.

**Table 34 Tuning the encoder learning rate does not impact midtraining findings.** LR multiple scales the learning rate of each recipe's respective vision pathway, relative to the shared peak learning rate ($3 \times 10^{-4}$; Table 8). Bold: within one point of the best on VQA. Avg. Text varies by only 0.6 points across all rows, so Text metrics are not bolded.

| Recipe | | VQA (%) ↑ | | | | | Text (%) ↑ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Model | LR multiple | Avg | Gen | Knw | OCR | Vis | Avg | Com | Sci | Knw | Rdg |
| • Scratch encoder | 1× | 48.6 | 58.8 | 42.0 | 38.9 | **54.5** | 56.7 | 67.5 | 48.9 | 50.5 | 60.0 |
| | 3× | **49.4** | **61.4** | 41.4 | **42.1** | 52.5 | 56.3 | 66.8 | 48.0 | 50.9 | 59.3 |
| ▾ In-fusion | 1× | **50.1** | **61.0** | 43.1 | 41.0 | **55.2** | 56.4 | 66.7 | 47.8 | 51.0 | 60.2 |
| | 3× | **49.6** | **60.5** | **44.3** | 40.2 | 53.6 | 56.8 | 67.5 | 48.7 | 51.4 | 59.6 |
| MoT, layers 1–8 | 1× | 48.5 | 60.1 | 41.8 | 39.5 | 52.5 | 56.7 | 66.8 | 48.8 | 51.6 | 59.4 |
| | 3× | **49.2** | **60.4** | 43.1 | 39.4 | 53.7 | 56.8 | 67.2 | 48.8 | 51.0 | 60.1 |
| MoT, all layers | 1× | 43.3 | 50.9 | 39.3 | 38.2 | 44.8 | 56.6 | 67.4 | 48.0 | 50.9 | 59.9 |
| | 3× | 45.1 | 53.6 | 41.3 | 37.6 | 47.9 | 56.5 | 66.8 | 48.0 | 51.4 | 59.9 |
| ◆ MetaCLIP ("free") | 0.3× | 38.8 | 46.6 | 39.2 | 23.9 | 45.7 | 56.7 | 67.0 | 48.8 | 51.5 | 59.7 |
| | 1× | 43.5 | 55.2 | 39.7 | 29.3 | 49.8 | 56.5 | 67.3 | 48.0 | 50.9 | 59.6 |
| | 3× | 46.1 | 57.9 | 40.2 | 35.4 | 50.7 | 56.6 | 67.1 | 48.6 | 50.7 | 59.8 |
| | 10× | 42.9 | 52.9 | 39.9 | 28.1 | 50.7 | 56.5 | 67.6 | 47.9 | 50.7 | 60.0 |

**Encoder pipelining.** Overlapping or pipelining (Huang et al., 2019) the encoder recovers little of its cost. At 4B, running the next microbatch's encoder forward pass alongside the backbone hides 20–36% of the encoder's per-step cost. Only the forward pass overlaps, because the encoder's backward pass waits for the backbone's gradients. With or without an encoder, a four-stage pipeline has about half the throughput of the encoder recipe under FSDP on the same 32 GPUs. FSDP optimizations applied to the encoder alone, such as prefetching parameters or skipping gradient synchronization between accumulation steps, change step time by less than 0.7%.

**Table 35 In-fusion vision capacity trains as fast as encoder-free, while the encoder-based model takes longer.** Training time is for a 4B model projected to $1.04 \times 10^{22}$ FLOPs of lifetime compute (400B token-equivalents). Intervals are paired 95% bootstrap intervals over 17 common timing windows.

| Architecture | Training time | |
|---|---|---|
| Variant | Δ vs. encoder-free (%) | 95% CI |
| Encoder | +2.9 | [+1.9, +4.2] |
| In-fusion (FFN-only) | −0.3 | [−0.8, +0.4] |
| In-fusion | +0.3 | [−0.5, +1.1] |

**Visual workload.** Per step, encoder-free and in-fusion (FFN-only) both train faster than the encoder recipe. Encoder-free saves more because the vision-only FFNs of in-fusion (FFN-only) cost about 95% of the encoder's FLOPs. In an 8-GPU setup at 1.7B, raising the proportion of image–text data from 10% to 50% widens encoder-free's saving from 10.0% to 13.2%, while in-fusion (FFN-only) saves 6–7% throughout (Figure 20).

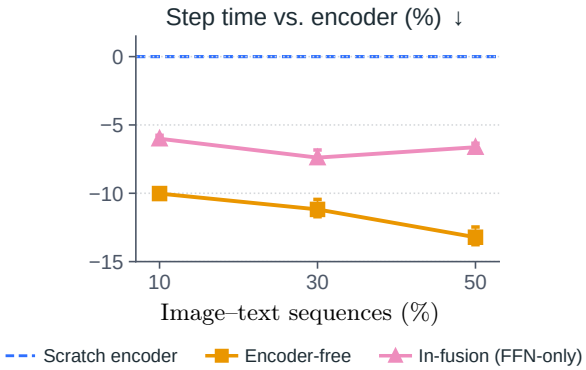



**Figure 20 Encoder-free and in-fusion (FFN-only) reduce training step time relative to encoder.** All runs use the 1.7B backbone. Intervals are paired 95% bootstrap intervals.

## H.3 Inference latency

Section 6.3 compares how fast each recipe serves image requests. We run every design in our own implementation on one H200, so all designs share the same inference code. Each request has one image and 256 text tokens and generates 120 tokens. Largely following the inference benchmark setup of Mono-InternVL (Luo et al., 2025b), we report time to first token (TTFT) and output throughput with 16 concurrent requests and 448-pixel images (1,024 vision tokens) unless noted. Each configuration is measured in four runs, and we report medians with full ranges.

Without a separate encoder, the backbone no longer waits for each image to be encoded, so TTFT falls (Figure 21). In-fusion (FFN-only) still processes every vision token with its vision-only FFNs, and at matched vision-only parameters, it performs approximately the same computation per image as the encoder. Its advantage comes from running in eight wide layers instead of the encoder's deeper stack of narrower layers (27 at the 0.4B budget), in our eager-mode implementation. The inference time saving grows with resolution, because more vision tokens make the encoder a larger share of the time to first token.

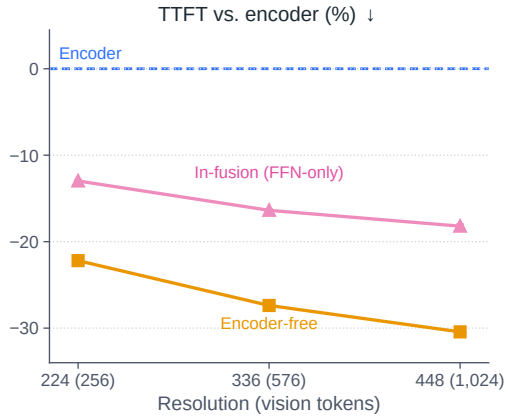



**Figure 21 Both encoder-free and in-fusion (FFN-only) reduce TTFT at higher resolution.** TTFT is measured relative to the encoder at 1.7B with 16 concurrent requests, where intervals are full ranges over four runs.

**Varying the vision budget.** We also try varying the vision-only parameters across the encoder sizes of Appendix A.2, with both in-fusion variants sized to match each encoder (Table 36). In-fusion is faster than the encoder at every budget. The time that vision adds over encoder-free is set by the vision budget rather than the backbone, so in-fusion (FFN-only) saves as much time at 4B as at 1.7B. Its relative inference time saving is smaller at 4B only because the backbone's own prefill time is longer.

**Table 36 In-fusion capacity serves image requests faster than the encoder at every vision budget.** Encoder-free takes 146 ms at 1.7B and 340 ms at 4B, 31% and 16% less than the Large encoder. Each request has one 448-pixel image (1,024 vision tokens) and 256 text tokens, served on one H200. Budgets describe the encoder's vision-only parameters, which the in-fusion modules match. TTFT change relative to the scratch encoder is shown in parentheses.

| | TTFT with 16 requests (ms) ↓ | | |
|---|---|---|---|
| Budget | ● Scratch encoder | ▲ In-fusion (FFN-only) | ▼ In-fusion |
| *1.7B backbone* | | | |
| Small (46M) | 158 | 152 (−4%) | 154 (−3%) |
| Base (152M) | 176 | 157 (−11%) | 169 (−4%) |
| Large (401M) | 210 | 171 (−18%) | 182 (−13%) |
| Huge (924M) | 268 | 196 (−27%) | 206 (−23%) |
| Wide (1.26B) | 273 | 212 (−22%) | 219 (−19%) |
| *4B backbone* | | | |
| Base (152M) | 374 | 354 (−5%) | 361 (−3%) |
| Large (401M) | 405 | 365 (−10%) | 381 (−6%) |
| Huge (924M) | 464 | 392 (−16%) | 405 (−13%) |

# I Per-benchmark results

This section reports every VQA and text benchmark of Tables 13 and 14 for the runs that each main-text exhibit shows, one table per exhibit. Avg. VQA and Avg. Text are computed as in Appendix B, and FLOPs is lifetime compute. Cap. is the held-out caption loss on regenerated captions of MetaCLIP 1.2 images never used in training, and Text the held-out text loss on unseen Nemotron-CC text weighted as in the training mixture (Appendix B.4); both are in nats per token and measured before VQA instruction tuning.

**Column headers.**

| Header | VQA benchmark |
|---|---|
| *General (Gen)* | |
| MME-P | MME-P |
| MMB | MMBench-EN |
| SEED | SEED-Bench |
| GQA | GQA |
| *Knowledge (Knw)* | |
| SQA | ScienceQA (image) |
| MMMU | MMMU |
| MathV | MathVista |
| AI2D | AI2D |
| *OCR and chart (OCR)* | |
| Chart | ChartQA |
| OCRB | OCRBench |
| TextV | TextVQA |
| DocV | DocVQA |
| *Vision-centric (Vis)* | |
| MMVP | MMVP |
| RWQA | RealWorldQA |
| CV2D | CV-Bench-2D |
| CV3D | CV-Bench-3D |

| Header | Text benchmark |
|---|---|
| *Commonsense reasoning (Com)* | |
| COPA | COPA |
| HS | HellaSwag |
| PIQA | PIQA |
| SIQA | Social IQa |
| WG | WinoGrande |
| *Science question answering (Sci)* | |
| ARC-E | ARC-Easy |
| ARC-C | ARC-Challenge |
| OBQA | OpenBookQA |
| *Broad knowledge (Knw)* | |
| MMLU | MMLU (57 subjects) |
| *Reading comprehension (Rdg)* | |
| BoolQ | BoolQ |
| RACEm | RACE-middle |
| RACEh | RACE-high |

**Tables.**


| Table | Main-text exhibit | Comparison | Page |
|---|---|---|---|
| 37 | Figure 2a | Encoder pretraining shares of a fixed lifetime budget | 48 |
| 38 | Figure 2b | From-scratch versus MetaCLIP-pretrained encoders across lifetime compute | 48 |
| 39 | Figure 3 | The from-scratch encoder versus encoder-free across lifetime compute | 49 |
| 40 | Table 2 | Encoder-free backbones enlarged by the encoder's parameters | 49 |
| 41 | Table 3 | Placements of added in-fusion capacity | 50 |
| 42 | Figure 5 | In-fusion versus in-fusion (FFN-only) across lifetime compute | 50 |
| 43 | Figure 6 | The from-scratch encoder, encoder-free, and In-fusion across lifetime compute | 51 |
| 44 | Figure 7 | Recipes across backbone sizes at 64 TPP | 52 |
| 45 | Figure 8 | Free MetaCLIP pretraining budgets before a fixed multimodal stage | 53 |
| 46 | Figure 9 | Recipes midtrained from a 1.7B text-only model | 53 |

**Table 37 Per-benchmark results for Figure 2a.** All runs use a 1.7B backbone and a $1.04 \times 10^{22}$-FLOP lifetime budget, of which Share is spent on encoder pretraining.

| Setup | | VQA (%) ↑ | | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Encoder | Share (%) | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| • Scratch encoder | 0 | 49.0 | 63.1 | 65.4 | 59.9 | 52.7 | 58.0 | 27.7 | 30.2 | 49.7 | 49.1 | 25.7 | 55.3 | 43.5 | 57.0 | 46.8 | 49.4 | 51.2 | 54.3 | 80.0 | 67.3 | 77.9 | 43.2 | 62.2 | 69.2 | 42.8 | 39.2 | 47.3 | 68.4 | 55.5 | 39.5 | 0.898 | 1.940 |
| • MetaCLIP | 7.4 | 47.3 | 57.3 | 64.2 | 58.3 | 52.2 | 56.2 | 29.2 | 29.5 | 45.5 | 46.0 | 25.4 | 52.6 | 39.6 | 56.3 | 49.3 | 35.9 | 59.5 | 54.4 | 80.0 | 66.5 | 77.6 | 44.6 | 63.4 | 70.3 | 43.8 | 37.4 | 47.5 | 69.0 | 54.6 | 39.4 | 0.908 | 1.938 |
| | 15 | 47.5 | 61.0 | 65.5 | 59.9 | 51.3 | 55.5 | 29.8 | 27.9 | 47.9 | 46.3 | 24.6 | 50.3 | 39.8 | 60.0 | 44.8 | 47.6 | 48.3 | 54.7 | 79.0 | 66.5 | 77.7 | 43.4 | 62.9 | 70.3 | 44.9 | 39.0 | 48.0 | 69.0 | 54.8 | 39.5 | 0.912 | 1.945 |
| | 30 | 46.4 | 60.7 | 62.8 | 56.9 | 51.9 | 52.9 | 27.7 | 28.8 | 44.5 | 45.2 | 23.2 | 52.3 | 39.6 | 55.3 | 49.7 | 39.9 | 50.3 | 53.5 | 80.0 | 65.8 | 78.2 | 42.3 | 62.4 | 70.1 | 42.0 | 38.2 | 45.7 | 67.6 | 54.3 | 38.3 | 0.916 | 1.951 |
| | 60 | 44.4 | 58.1 | 56.9 | 53.7 | 50.6 | 50.7 | 26.7 | 28.3 | 43.4 | 42.8 | 22.3 | 48.8 | 35.9 | 55.3 | 46.9 | 38.5 | 50.7 | 52.2 | 76.0 | 65.1 | 77.2 | 43.3 | 62.2 | 68.2 | 41.6 | 36.2 | 43.5 | 66.4 | 55.0 | 39.3 | 0.943 | 1.970 |
| + WebSSL | 7.4 | 48.3 | 63.4 | 66.2 | 61.1 | 52.6 | 57.8 | 27.4 | 30.7 | 48.3 | 47.4 | 24.5 | 54.0 | 42.0 | 60.0 | 46.1 | 42.4 | 48.3 | 55.4 | 81.0 | 66.9 | 77.3 | 43.4 | 60.7 | 71.2 | 45.1 | 37.0 | 48.5 | 74.7 | 54.7 | 40.4 | 0.909 | 1.936 |
| | 15 | 49.3 | 63.1 | 66.8 | 60.1 | 52.5 | 58.2 | 29.0 | 29.4 | 48.8 | 44.6 | 24.2 | 51.7 | 38.6 | 62.3 | 50.6 | 44.1 | 65.2 | 54.6 | 80.0 | 66.9 | 77.4 | 42.8 | 63.2 | 67.2 | 43.3 | 37.8 | 48.5 | 70.2 | 55.0 | 40.0 | 0.914 | 1.944 |
| | 30 | 47.4 | 59.0 | 64.7 | 58.0 | 51.8 | 57.4 | 26.6 | 31.2 | 45.6 | 43.4 | 24.7 | 51.7 | 41.1 | 58.3 | 53.2 | 42.4 | 49.8 | 53.5 | 81.0 | 66.7 | 76.9 | 44.3 | 61.0 | 68.2 | 42.4 | 37.2 | 47.2 | 66.2 | 54.0 | 38.6 | 0.909 | 1.951 |
| | 60 | 44.8 | 57.9 | 62.4 | 54.7 | 51.2 | 54.5 | 26.6 | 29.3 | 43.6 | 44.0 | 22.8 | 50.8 | 38.9 | 53.7 | 43.8 | 33.0 | 50.2 | 52.2 | 79.0 | 64.8 | 77.7 | 41.7 | 61.4 | 68.7 | 41.5 | 37.2 | 43.2 | 66.3 | 53.8 | 38.7 | 0.928 | 1.970 |
| • Cap | 7.4 | 48.5 | 59.2 | 64.6 | 60.4 | 52.0 | 56.7 | 30.9 | 29.5 | 47.7 | 46.4 | 25.3 | 53.2 | 40.5 | 57.0 | 49.9 | 45.5 | 57.8 | 53.9 | 79.0 | 66.8 | 78.3 | 43.7 | 62.2 | 68.5 | 42.7 | 38.2 | 46.8 | 68.4 | 53.3 | 39.1 | 0.907 | 1.941 |
| | 15 | 48.3 | 60.8 | 63.4 | 58.0 | 52.3 | 56.8 | 30.4 | 29.3 | 48.4 | 44.6 | 24.6 | 50.0 | 40.3 | 59.3 | 52.0 | 45.2 | 56.7 | 54.4 | 79.0 | 67.0 | 78.4 | 42.5 | 63.4 | 69.3 | 41.5 | 40.0 | 46.2 | 71.1 | 54.9 | 39.4 | 0.912 | 1.945 |
| | 30 | 46.4 | 55.0 | 63.5 | 57.9 | 51.7 | 56.2 | 30.0 | 28.0 | 43.4 | 45.9 | 24.9 | 52.3 | 40.0 | 59.7 | 43.3 | 39.9 | 50.2 | 53.0 | 83.0 | 66.0 | 78.3 | 42.4 | 61.8 | 69.0 | 43.3 | 40.2 | 44.5 | 62.5 | 54.5 | 39.3 | 0.918 | 1.950 |
| | 60 | 44.4 | 60.2 | 59.6 | 51.8 | 50.5 | 51.3 | 26.6 | 28.5 | 38.5 | 42.6 | 22.7 | 49.0 | 37.9 | 52.7 | 47.3 | 42.5 | 49.3 | 51.5 | 81.0 | 65.5 | 77.9 | 43.9 | 60.1 | 68.2 | 41.1 | 36.2 | 41.5 | 64.3 | 53.4 | 38.5 | 0.942 | 1.971 |
| • CapPa | 7.4 | 47.0 | 63.2 | 60.7 | 58.9 | 52.2 | 56.6 | 27.3 | 29.9 | 48.0 | 45.2 | 24.1 | 50.9 | 41.2 | 58.7 | 49.5 | 41.4 | 44.6 | 55.0 | 78.0 | 66.4 | 77.9 | 42.9 | 62.7 | 71.0 | 41.7 | 40.8 | 48.1 | 72.2 | 54.5 | 39.2 | 0.907 | 1.939 |
| | 15 | 47.2 | 59.7 | 65.7 | 58.1 | 51.2 | 55.3 | 29.8 | 27.1 | 46.9 | 44.2 | 23.8 | 53.2 | 39.2 | 59.3 | 47.3 | 45.1 | 48.5 | 54.3 | 81.0 | 66.5 | 78.7 | 43.9 | 62.4 | 69.0 | 42.7 | 39.2 | 46.3 | 70.2 | 54.2 | 39.7 | 0.913 | 1.946 |
| | 30 | 46.5 | 58.8 | 63.2 | 59.0 | 51.8 | 56.9 | 26.7 | 27.9 | 47.7 | 45.3 | 23.6 | 51.4 | 39.1 | 60.0 | 48.2 | 38.4 | 45.6 | 53.2 | 79.0 | 66.3 | 78.1 | 42.1 | 62.7 | 70.6 | 41.3 | 36.6 | 45.8 | 67.0 | 53.6 | 39.3 | 0.924 | 1.951 |
| | 60 | 42.8 | 55.2 | 60.3 | 56.5 | 50.3 | 51.5 | 26.6 | 25.6 | 40.4 | 39.3 | 22.1 | 48.4 | 35.9 | 52.7 | 46.8 | 40.5 | 33.3 | 51.1 | 77.0 | 64.7 | 76.9 | 42.9 | 60.4 | 67.7 | 41.1 | 37.6 | 41.7 | 60.9 | 54.7 | 38.8 | 0.945 | 1.970 |

**Table 38 Per-benchmark results for Figure 2b.** MetaCLIP's lifetime FLOPs include its $1.54 \times 10^{21}$-FLOP encoder pretraining.

| Setup | | VQA (%) ↑ | | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Encoder | FLOPs | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| *1.7B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $6.51\times10^{20}$ | 37.3 | 51.7 | 48.0 | 42.7 | 47.0 | 42.5 | 26.2 | 24.4 | 32.0 | 35.7 | 12.8 | 34.5 | 18.3 | 51.3 | 40.3 | 36.7 | 52.5 | 45.3 | 69.0 | 56.5 | 75.5 | 41.4 | 58.5 | 61.9 | 36.8 | 36.4 | 30.4 | 55.8 | 49.3 | 36.0 | 1.169 | 2.118 |
| | $1.30\times10^{21}$ | 41.2 | 54.8 | 53.9 | 50.0 | 48.7 | 46.1 | 25.3 | 24.9 | 35.6 | 40.5 | 19.9 | 45.7 | 29.8 | 52.3 | 43.5 | 34.0 | 54.2 | 47.3 | 73.0 | 60.6 | 76.7 | 42.0 | 59.0 | 63.4 | 38.5 | 36.4 | 32.4 | 60.8 | 51.3 | 36.9 | 1.029 | 2.052 |
| | $2.60\times10^{21}$ | 44.4 | 61.3 | 60.7 | 56.0 | 51.0 | 49.7 | 26.0 | 27.0 | 40.2 | 43.9 | 23.0 | 50.4 | 35.6 | 55.3 | 45.6 | 35.3 | 49.6 | 49.7 | 75.0 | 63.3 | 76.9 | 42.9 | 61.2 | 66.8 | 40.6 | 36.2 | 36.7 | 64.5 | 53.3 | 37.7 | 0.960 | 2.003 |
| | $5.21\times10^{21}$ | 46.5 | 60.3 | 63.8 | 57.5 | 52.4 | 55.2 | 28.8 | 28.8 | 42.8 | 44.4 | 23.7 | 50.7 | 38.8 | 57.7 | 47.3 | 43.2 | 49.4 | 52.5 | 77.0 | 65.4 | 78.8 | 43.2 | 60.9 | 68.0 | 42.1 | 39.4 | 41.5 | 69.2 | 55.4 | 38.8 | 0.922 | 1.967 |
| | $7.81\times10^{21}$ | 47.5 | 63.3 | 66.5 | 60.0 | 51.8 | 55.2 | 29.3 | 29.0 | 47.2 | 45.4 | 23.8 | 53.5 | 40.7 | 57.3 | 49.0 | 44.2 | 44.4 | 54.4 | 80.0 | 66.3 | 77.6 | 43.6 | 61.2 | 69.3 | 43.6 | 41.4 | 46.1 | 68.9 | 56.1 | 40.4 | 0.906 | 1.951 |
| | $1.04\times10^{22}$ | 49.0 | 63.1 | 65.4 | 59.9 | 52.7 | 58.0 | 27.7 | 30.2 | 49.7 | 49.1 | 25.7 | 55.3 | 43.5 | 57.0 | 46.8 | 49.4 | 51.2 | 54.3 | 80.0 | 67.3 | 77.9 | 43.2 | 62.2 | 69.2 | 42.8 | 39.2 | 47.3 | 68.4 | 55.5 | 39.5 | 0.898 | 1.940 |
| • MetaCLIP | $2.60\times10^{21}$ | 38.8 | 51.9 | 52.0 | 48.9 | 46.9 | 44.3 | 25.8 | 25.9 | 37.2 | 34.6 | 15.1 | 36.9 | 22.3 | 52.7 | 39.7 | 35.5 | 50.3 | 47.5 | 72.0 | 59.4 | 75.6 | 41.5 | 57.9 | 63.8 | 37.2 | 35.6 | 32.7 | 64.9 | 51.2 | 37.5 | 1.097 | 2.062 |
| | $5.21\times10^{21}$ | 44.5 | 56.9 | 62.2 | 55.8 | 50.9 | 50.1 | 27.2 | 27.4 | 43.0 | 42.0 | 21.5 | 48.2 | 35.0 | 52.7 | 46.4 | 43.6 | 49.8 | 52.2 | 76.0 | 64.1 | 76.8 | 44.3 | 60.4 | 67.2 | 40.5 | 38.6 | 42.5 | 69.1 | 53.1 | 38.1 | 0.955 | 1.981 |
| | $1.04\times10^{22}$ | 47.5 | 61.0 | 65.5 | 59.9 | 51.3 | 55.5 | 29.8 | 27.9 | 47.9 | 46.3 | 24.6 | 50.3 | 39.8 | 60.0 | 44.8 | 47.6 | 48.3 | 54.7 | 79.0 | 66.5 | 77.7 | 43.4 | 62.9 | 70.3 | 44.9 | 39.0 | 48.0 | 69.0 | 54.8 | 39.5 | 0.912 | 1.945 |
| *4B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $2.90\times10^{21}$ | 47.8 | 61.3 | 63.8 | 59.1 | 51.9 | 57.1 | 29.1 | 31.6 | 45.9 | 48.2 | 24.0 | 50.7 | 36.1 | 56.7 | 45.1 | 50.6 | 54.3 | 52.0 | 78.0 | 65.9 | 77.4 | 43.0 | 63.4 | 69.0 | 42.1 | 39.0 | 41.1 | 65.6 | 52.6 | 38.7 | 0.944 | 1.959 |
| | $5.79\times10^{21}$ | 51.2 | 65.4 | 69.1 | 63.2 | 53.0 | 64.0 | 30.3 | 33.1 | 50.8 | 50.4 | 26.9 | 54.0 | 42.6 | 63.0 | 49.5 | 50.0 | 53.4 | 56.4 | 83.0 | 68.3 | 78.2 | 44.1 | 64.2 | 72.7 | 45.2 | 40.6 | 49.3 | 72.5 | 56.3 | 40.4 | 0.891 | 1.905 |
| | $1.16\times10^{22}$ | 51.9 | 64.8 | 71.0 | 62.5 | 53.7 | 65.9 | 29.1 | 34.0 | 55.2 | 53.0 | 27.8 | 55.4 | 45.8 | 63.0 | 49.3 | 47.3 | 52.9 | 57.9 | 83.0 | 70.6 | 79.0 | 45.3 | 65.0 | 73.1 | 47.7 | 40.6 | 52.3 | 75.2 | 56.8 | 40.5 | 0.864 | 1.867 |
| • MetaCLIP | $2.90\times10^{21}$ | 41.4 | 57.0 | 54.2 | 52.8 | 48.8 | 48.1 | 25.8 | 28.8 | 38.7 | 40.8 | 17.7 | 39.1 | 25.8 | 53.3 | 42.2 | 38.2 | 51.8 | 48.3 | 71.0 | 61.5 | 74.6 | 41.9 | 59.6 | 64.7 | 39.1 | 35.8 | 34.7 | 64.4 | 51.8 | 37.7 | 1.098 | 2.034 |
| | $5.79\times10^{21}$ | 46.5 | 60.3 | 62.0 | 58.8 | 52.4 | 55.8 | 23.7 | 32.5 | 44.7 | 46.8 | 23.6 | 51.0 | 39.6 | 57.0 | 45.2 | 46.4 | 43.8 | 53.2 | 78.0 | 67.7 | 78.8 | 43.0 | 62.9 | 69.3 | 44.5 | 38.8 | 43.6 | 66.6 | 55.7 | 39.8 | 0.925 | 1.923 |
| | $1.16\times10^{22}$ | 50.1 | 60.3 | 68.2 | 63.1 | 53.2 | 62.0 | 30.9 | 31.0 | 52.9 | 47.2 | 24.6 | 53.1 | 42.3 | 60.3 | 51.4 | 50.5 | 51.1 | 57.0 | 81.0 | 70.5 | 79.2 | 44.1 | 63.2 | 72.3 | 47.1 | 41.0 | 51.3 | 71.9 | 57.1 | 40.9 | 0.879 | 1.877 |

**Table 39 Per-benchmark results for Figure 3.**

| Setup | | VQA (%) ↑ | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Recipe | FLOPs | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| *1.7B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $6.51\times10^{20}$ | 37.3 | 51.7 | 48.0 | 42.7 | 47.0 | 42.5 | 26.2 | 24.4 | 32.0 | 35.7 | 12.8 | 34.5 | 18.3 | 51.3 | 40.3 | 36.7 | 52.5 | 45.3 | 69.0 | 56.5 | 75.5 | 41.4 | 58.5 | 61.9 | 36.8 | 36.4 | 30.4 | 55.8 | 49.3 | 36.0 | 1.169 | 2.118 |
| | $1.30\times10^{21}$ | 41.2 | 54.8 | 53.9 | 50.0 | 48.7 | 46.1 | 25.3 | 24.9 | 35.6 | 40.5 | 19.9 | 45.7 | 29.8 | 52.3 | 43.5 | 34.0 | 54.2 | 47.3 | 73.0 | 60.6 | 76.7 | 42.0 | 59.0 | 63.4 | 38.5 | 36.4 | 32.4 | 60.8 | 51.3 | 36.9 | 1.029 | 2.052 |
| | $2.60\times10^{21}$ | 44.4 | 61.3 | 60.7 | 56.0 | 51.0 | 49.7 | 26.0 | 27.0 | 40.2 | 43.9 | 23.0 | 50.4 | 35.6 | 55.3 | 45.6 | 35.3 | 49.6 | 49.7 | 75.0 | 63.3 | 76.9 | 42.9 | 61.2 | 66.8 | 40.6 | 36.2 | 36.7 | 64.5 | 53.3 | 37.7 | 0.960 | 2.003 |
| | $5.21\times10^{21}$ | 46.5 | 60.3 | 63.8 | 57.5 | 52.4 | 55.2 | 28.8 | 28.8 | 42.8 | 44.4 | 23.7 | 50.7 | 38.8 | 57.7 | 47.3 | 43.2 | 49.4 | 52.5 | 77.0 | 65.4 | 78.8 | 43.2 | 60.9 | 68.0 | 42.1 | 39.4 | 41.5 | 69.2 | 55.4 | 38.8 | 0.922 | 1.967 |
| | $7.81\times10^{21}$ | 47.5 | 63.3 | 66.5 | 60.0 | 51.8 | 55.2 | 29.3 | 29.0 | 47.2 | 45.4 | 23.8 | 53.5 | 40.7 | 57.3 | 49.0 | 44.2 | 44.4 | 54.4 | 80.0 | 66.3 | 77.6 | 43.6 | 61.2 | 69.3 | 43.6 | 41.4 | 46.1 | 68.9 | 56.1 | 40.4 | 0.906 | 1.951 |
| | $1.04\times10^{22}$ | 49.0 | 63.1 | 65.4 | 59.9 | 52.7 | 58.0 | 27.7 | 30.2 | 49.7 | 49.1 | 25.7 | 55.3 | 43.5 | 57.0 | 46.8 | 49.4 | 51.2 | 54.3 | 80.0 | 67.3 | 77.9 | 43.2 | 62.2 | 69.2 | 42.8 | 39.2 | 47.3 | 68.4 | 55.5 | 39.5 | 0.898 | 1.940 |
| | $1.30\times10^{22}$ | 48.9 | 64.1 | 66.5 | 58.5 | 53.4 | 56.7 | 31.4 | 29.2 | 47.7 | 46.2 | 25.3 | 54.9 | 43.5 | 58.0 | 45.8 | 42.2 | 58.8 | 55.3 | 80.0 | 67.7 | 77.5 | 43.8 | 61.9 | 69.7 | 44.0 | 39.4 | 49.3 | 70.3 | 56.7 | 40.1 | 0.890 | 1.930 |
| ▪ Encoder-free | $6.51\times10^{20}$ | 32.0 | 47.0 | 40.4 | 37.9 | 44.5 | 39.2 | 26.9 | 23.2 | 33.3 | 18.6 | 5.0 | 16.1 | 4.0 | 48.7 | 39.5 | 36.8 | 50.8 | 46.4 | 70.0 | 57.0 | 75.1 | 42.2 | 57.1 | 61.5 | 35.7 | 36.0 | 30.6 | 64.4 | 49.7 | 36.4 | 1.307 | 2.114 |
| | $1.30\times10^{21}$ | 34.8 | 48.6 | 43.3 | 42.1 | 45.4 | 43.0 | 25.2 | 23.4 | 32.4 | 27.4 | 11.4 | 29.4 | 11.7 | 52.0 | 38.4 | 32.2 | 50.5 | 48.3 | 72.0 | 59.8 | 75.7 | 42.5 | 59.7 | 63.4 | 39.7 | 34.6 | 34.1 | 66.5 | 51.4 | 37.9 | 1.146 | 2.050 |
| | $2.60\times10^{21}$ | 39.0 | 52.5 | 54.3 | 48.3 | 47.8 | 48.4 | 28.0 | 25.3 | 38.3 | 33.0 | 13.8 | 36.2 | 18.0 | 51.7 | 40.5 | 35.0 | 52.8 | 50.7 | 78.0 | 63.2 | 75.8 | 42.3 | 60.9 | 67.6 | 41.7 | 37.0 | 38.2 | 67.2 | 51.9 | 38.7 | 1.061 | 2.003 |
| | $5.21\times10^{21}$ | 41.9 | 58.9 | 57.6 | 53.0 | 49.3 | 52.2 | 26.2 | 24.9 | 41.7 | 35.8 | 17.2 | 39.7 | 23.3 | 53.3 | 46.1 | 39.2 | 51.6 | 52.6 | 79.0 | 65.1 | 77.7 | 43.1 | 61.7 | 68.2 | 43.0 | 37.6 | 42.9 | 67.4 | 54.9 | 39.2 | 1.014 | 1.964 |
| | $7.81\times10^{21}$ | 43.0 | 58.3 | 58.5 | 53.4 | 50.6 | 52.4 | 26.4 | 27.0 | 41.1 | 38.3 | 18.3 | 42.6 | 27.5 | 53.0 | 45.2 | 41.0 | 54.5 | 53.7 | 81.0 | 65.9 | 77.3 | 43.1 | 61.9 | 68.3 | 43.7 | 38.0 | 45.4 | 68.8 | 55.0 | 40.2 | 0.987 | 1.952 |
| | $1.04\times10^{22}$ | 43.8 | 55.1 | 59.9 | 54.6 | 49.9 | 52.8 | 28.0 | 26.3 | 42.5 | 40.8 | 20.4 | 45.0 | 31.4 | 54.3 | 45.8 | 39.2 | 55.3 | 55.3 | 82.0 | 67.1 | 78.1 | 44.0 | 63.1 | 70.7 | 43.3 | 37.8 | 48.3 | 72.5 | 55.5 | 40.4 | 0.974 | 1.941 |
| | $1.30\times10^{22}$ | 44.8 | 56.9 | 62.5 | 56.4 | 49.8 | 53.5 | 29.0 | 26.9 | 45.9 | 41.7 | 19.9 | 45.2 | 31.9 | 56.7 | 45.1 | 44.7 | 50.8 | 55.7 | 81.0 | 67.5 | 77.5 | 44.4 | 62.1 | 70.7 | 42.7 | 39.6 | 48.6 | 73.7 | 56.7 | 40.4 | 0.964 | 1.930 |
| *4B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $2.90\times10^{21}$ | 47.8 | 61.3 | 63.8 | 59.1 | 51.9 | 57.1 | 29.1 | 31.6 | 45.9 | 48.2 | 24.0 | 50.7 | 36.1 | 56.7 | 45.1 | 50.6 | 54.3 | 52.0 | 78.0 | 65.9 | 77.4 | 43.0 | 63.4 | 69.0 | 42.1 | 39.0 | 41.1 | 65.6 | 52.6 | 38.7 | 0.944 | 1.959 |
| | $5.79\times10^{21}$ | 51.2 | 65.4 | 69.1 | 63.2 | 53.0 | 64.0 | 30.3 | 33.1 | 50.8 | 50.4 | 26.9 | 54.0 | 42.6 | 63.0 | 49.5 | 50.0 | 53.4 | 56.4 | 83.0 | 68.3 | 78.2 | 44.1 | 64.2 | 72.7 | 45.2 | 40.6 | 49.3 | 72.5 | 56.3 | 40.4 | 0.891 | 1.905 |
| | $1.16\times10^{22}$ | 51.9 | 64.8 | 71.0 | 62.5 | 53.7 | 65.9 | 29.1 | 34.0 | 55.2 | 53.0 | 27.8 | 55.4 | 45.8 | 63.0 | 49.3 | 47.3 | 52.9 | 57.9 | 83.0 | 70.6 | 79.0 | 45.3 | 65.0 | 73.1 | 47.7 | 40.6 | 52.3 | 75.2 | 56.8 | 40.5 | 0.864 | 1.867 |
| | $1.74\times10^{22}$ | 54.2 | 70.1 | 71.0 | 65.9 | 53.8 | 68.2 | 35.4 | 33.9 | 58.5 | 51.6 | 28.0 | 56.3 | 45.0 | 67.7 | 48.6 | 51.0 | 61.4 | 58.9 | 80.0 | 72.0 | 79.5 | 44.9 | 65.2 | 73.9 | 49.2 | 39.6 | 54.7 | 79.1 | 56.9 | 40.4 | 0.863 | 1.846 |
| | $2.90\times10^{22}$ | 54.5 | 71.2 | 73.3 | 67.6 | 53.5 | 68.5 | 33.7 | 34.6 | 59.7 | 52.6 | 26.9 | 56.3 | 46.2 | 63.0 | 54.1 | 55.6 | 55.2 | 60.4 | 81.0 | 73.1 | 80.1 | 46.3 | 66.9 | 74.7 | 49.9 | 40.8 | 57.5 | 79.6 | 58.5 | 41.3 | 0.850 | 1.819 |
| ▪ Encoder-free | $2.90\times10^{21}$ | 41.0 | 56.6 | 57.1 | 53.7 | 48.6 | 51.7 | 27.8 | 30.3 | 43.0 | 37.9 | 13.4 | 35.9 | 16.8 | 50.3 | 42.5 | 37.3 | 53.1 | 51.5 | 74.0 | 65.8 | 77.9 | 42.1 | 60.8 | 68.0 | 41.7 | 38.0 | 42.2 | 63.2 | 53.4 | 39.2 | 1.057 | 1.958 |
| | $5.79\times10^{21}$ | 44.9 | 57.4 | 61.6 | 58.0 | 50.8 | 56.7 | 29.2 | 27.7 | 48.8 | 40.4 | 19.1 | 43.2 | 27.6 | 55.3 | 46.4 | 41.6 | 53.8 | 56.0 | 82.0 | 68.5 | 78.2 | 43.4 | 63.4 | 70.2 | 45.6 | 41.4 | 49.5 | 70.0 | 56.8 | 40.7 | 0.970 | 1.905 |
| | $1.16\times10^{22}$ | 48.4 | 58.1 | 68.4 | 62.9 | 52.5 | 61.4 | 29.4 | 31.1 | 53.1 | 43.0 | 22.3 | 49.2 | 35.4 | 58.7 | 46.1 | 45.1 | 58.3 | 57.0 | 84.0 | 70.2 | 79.1 | 43.2 | 64.9 | 72.9 | 47.0 | 40.0 | 52.3 | 68.6 | 57.4 | 41.3 | 0.921 | 1.870 |
| | $1.74\times10^{22}$ | 50.2 | 65.9 | 68.0 | 62.5 | 53.5 | 63.6 | 30.8 | 31.2 | 54.2 | 45.9 | 23.4 | 50.9 | 38.0 | 61.3 | 46.8 | 52.6 | 54.5 | 59.2 | 84.0 | 71.4 | 78.7 | 44.6 | 64.2 | 73.4 | 50.4 | 40.2 | 55.3 | 77.0 | 58.8 | 41.9 | 0.894 | 1.845 |
| | $2.90\times10^{22}$ | 51.7 | 65.7 | 69.8 | 65.4 | 54.6 | 63.0 | 31.9 | 31.1 | 57.3 | 48.1 | 24.3 | 53.5 | 40.9 | 60.7 | 51.6 | 51.9 | 58.1 | 61.0 | 84.0 | 73.1 | 78.7 | 44.6 | 67.2 | 74.8 | 49.6 | 43.6 | 58.3 | 79.6 | 58.4 | 42.3 | 0.870 | 1.821 |

**Table 40 Per-benchmark results for Table 2.** All rows use a 1.7B backbone and $2.6\times10^{21}$ lifetime FLOPs.

| Architecture | VQA (%) ↑ | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Variant | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| • Scratch encoder | 44.4 | 61.3 | 60.7 | 56.0 | 51.0 | 49.7 | 26.0 | 27.0 | 40.2 | 43.9 | 23.0 | 50.4 | 35.6 | 55.3 | 45.6 | 35.3 | 49.6 | 49.7 | 75.0 | 63.3 | 76.9 | 42.9 | 61.2 | 66.8 | 40.6 | 36.2 | 36.7 | 64.5 | 53.3 | 37.7 | 0.960 | 2.003 |
| ▪ Encoder-free | 39.0 | 52.5 | 54.3 | 48.3 | 47.8 | 48.4 | 28.0 | 25.3 | 38.3 | 33.0 | 13.8 | 36.2 | 18.0 | 51.7 | 40.5 | 35.0 | 52.8 | 50.7 | 78.0 | 63.2 | 75.8 | 42.3 | 60.9 | 67.6 | 41.7 | 37.0 | 38.2 | 67.2 | 51.9 | 38.7 | 1.061 | 2.003 |
| + wider | 38.6 | 50.4 | 47.6 | 48.2 | 47.9 | 49.1 | 26.1 | 25.9 | 35.7 | 37.2 | 16.2 | 39.6 | 24.5 | 53.0 | 42.4 | 31.7 | 41.8 | 51.3 | 78.0 | 64.0 | 76.1 | 42.5 | 59.2 | 68.4 | 40.9 | 38.4 | 38.0 | 69.8 | 53.1 | 39.4 | 1.028 | 1.986 |
| + deeper | 38.7 | 53.8 | 49.5 | 48.8 | 47.7 | 49.7 | 26.0 | 23.0 | 34.2 | 35.1 | 14.4 | 37.9 | 17.8 | 50.7 | 44.1 | 36.5 | 50.3 | 51.2 | 76.0 | 64.3 | 77.4 | 42.8 | 60.6 | 67.9 | 43.1 | 36.0 | 39.1 | 68.9 | 52.9 | 38.8 | 1.047 | 1.982 |
| + wider and deeper | 38.8 | 55.1 | 52.6 | 50.4 | 48.1 | 48.7 | 28.7 | 23.9 | 38.1 | 33.5 | 12.6 | 36.6 | 17.8 | 51.3 | 40.7 | 34.3 | 48.3 | 51.4 | 77.0 | 63.9 | 77.0 | 41.2 | 60.7 | 66.9 | 40.9 | 37.4 | 40.3 | 68.2 | 54.5 | 39.1 | 1.045 | 1.984 |
| *Richer patch embedding* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Two-layer projection | 40.5 | 50.9 | 55.5 | 48.2 | 48.7 | 47.7 | 24.9 | 25.6 | 37.2 | 36.1 | 18.1 | 40.2 | 27.9 | 54.7 | 40.3 | 37.4 | 54.5 | 49.7 | 74.0 | 63.2 | 76.8 | 42.5 | 59.6 | 66.4 | 41.2 | 37.4 | 36.3 | 64.3 | 53.6 | 38.1 | – | 2.002 |
| Hierarchical MLP | 35.9 | 53.5 | 53.1 | 48.5 | 48.1 | 46.3 | 23.7 | 25.0 | 37.3 | 18.7 | 8.3 | 30.7 | 3.4 | 57.7 | 42.9 | 36.0 | 41.4 | 50.3 | 72.0 | 63.3 | 77.2 | 42.6 | 62.6 | 67.4 | 39.1 | 37.0 | 39.3 | 64.6 | 52.8 | 37.9 | 1.132 | 2.001 |

**Table 41 Per-benchmark results for Table 3.** All rows use a 1.7B backbone and $2.6 \times 10^{21}$ lifetime FLOPs.

| Architecture | VQA (%) ↑ | | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Variant | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| • Scratch encoder | 44.4 | 61.3 | 60.7 | 56.0 | 51.0 | 49.7 | 26.0 | 27.0 | 40.2 | 43.9 | 23.0 | 50.4 | 35.6 | 55.3 | 45.6 | 35.3 | 49.6 | 49.7 | 75.0 | 63.3 | 76.9 | 42.9 | 61.2 | 66.8 | 40.6 | 36.2 | 36.7 | 64.5 | 53.3 | 37.7 | 0.960 | 2.003 |
| ▪ Encoder-free | 39.0 | 52.5 | 54.3 | 48.3 | 47.8 | 48.4 | 28.0 | 25.3 | 38.3 | 33.0 | 13.8 | 36.2 | 18.0 | 51.7 | 40.5 | 35.0 | 52.8 | 50.7 | 78.0 | 63.2 | 75.8 | 42.3 | 60.9 | 67.6 | 41.7 | 37.0 | 38.2 | 67.2 | 51.9 | 38.7 | 1.061 | 2.003 |
| *In-fusion (FFN-only): where to add (vision tokens)* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Every layer (narrow) | 39.4 | 47.4 | 45.4 | 43.1 | 49.7 | 47.0 | 25.8 | 22.6 | 38.4 | 35.5 | 19.2 | 44.3 | 26.3 | 54.3 | 41.0 | 41.4 | 49.5 | 50.7 | 80.0 | 62.8 | 76.2 | 42.2 | 61.1 | 66.7 | 40.2 | 37.8 | 39.8 | 64.3 | 51.7 | 37.8 | 0.974 | 2.003 |
| Spread (8 of 28) | 37.6 | 48.5 | 43.6 | 46.4 | 48.5 | 44.0 | 27.3 | 23.5 | 37.6 | 32.8 | 15.6 | 39.9 | 24.1 | 49.7 | 37.8 | 33.2 | 49.0 | 50.0 | 80.0 | 63.1 | 76.7 | 41.3 | 58.5 | 67.1 | 41.4 | 37.0 | 40.0 | 59.4 | 50.9 | 37.7 | 1.007 | 2.004 |
| Layers 21–28 | 35.1 | 47.6 | 39.2 | 36.8 | 46.8 | 41.2 | 26.7 | 22.5 | 35.4 | 29.1 | 12.5 | 35.3 | 16.3 | 51.7 | 41.7 | 33.7 | 46.1 | 49.8 | 77.0 | 63.8 | 76.7 | 42.2 | 60.8 | 66.2 | 41.0 | 35.6 | 38.5 | 61.3 | 53.6 | 38.6 | 1.061 | 2.004 |
| ▲ Layers 1–8 | 45.0 | 58.4 | 63.2 | 56.7 | 49.8 | 51.2 | 27.4 | 25.5 | 42.7 | 40.8 | 21.7 | 46.6 | 35.0 | 59.0 | 43.5 | 42.3 | 56.8 | 50.4 | 76.0 | 63.3 | 76.2 | 40.5 | 58.7 | 66.6 | 40.0 | 37.2 | 40.6 | 63.8 | 53.1 | 37.4 | 0.969 | 2.003 |
| *All-token controls (layers 1–8)* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Shared FFN widening | 40.1 | 60.3 | 50.3 | 47.3 | 48.2 | 47.2 | 25.2 | 26.8 | 36.9 | 36.6 | 18.1 | 38.3 | 25.5 | 52.0 | 40.5 | 38.0 | 49.7 | 51.2 | 76.0 | 64.2 | 77.6 | 43.1 | 61.6 | 66.9 | 41.8 | 36.8 | 40.6 | 66.2 | 52.2 | 38.1 | 1.050 | 1.988 |
| Separate FFN branch | 42.3 | 54.6 | 57.4 | 55.1 | 49.4 | 49.8 | 29.9 | 27.4 | 42.6 | 38.8 | 17.5 | 41.1 | 25.9 | 58.0 | 41.3 | 34.9 | 52.8 | 51.2 | 80.0 | 63.9 | 76.6 | 43.2 | 60.2 | 67.6 | 41.5 | 38.0 | 41.6 | 61.8 | 51.1 | 38.6 | 1.032 | 1.987 |

**Table 42 Per-benchmark results for Figure 5.**

| Setup | | VQA (%) ↑ | | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Recipe | FLOPs | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| *1.7B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| ▲ In-fusion (FFN-only) | $6.51\times10^{20}$ | 36.3 | 54.6 | 43.8 | 43.3 | 47.3 | 39.5 | 24.2 | 22.3 | 30.3 | 34.8 | 14.2 | 35.3 | 19.4 | 50.0 | 42.4 | 33.6 | 46.7 | 45.9 | 73.0 | 56.8 | 74.9 | 40.9 | 57.9 | 61.4 | 35.1 | 34.4 | 29.6 | 64.3 | 49.4 | 36.1 | 1.138 | 2.119 |
| | $1.30\times10^{21}$ | 41.4 | 53.2 | 53.1 | 49.5 | 49.0 | 44.6 | 28.2 | 26.3 | 33.7 | 39.5 | 19.6 | 43.0 | 28.8 | 52.3 | 46.3 | 44.0 | 51.0 | 48.4 | 74.0 | 60.3 | 75.0 | 41.6 | 59.9 | 63.8 | 38.7 | 37.4 | 35.2 | 62.5 | 50.6 | 37.5 | 1.034 | 2.052 |
| | $2.60\times10^{21}$ | 45.0 | 58.4 | 63.2 | 56.7 | 49.8 | 51.2 | 27.4 | 25.5 | 42.7 | 40.8 | 21.7 | 46.6 | 35.0 | 59.0 | 43.5 | 42.3 | 56.8 | 50.4 | 76.0 | 63.3 | 76.2 | 40.5 | 58.7 | 66.6 | 40.0 | 37.2 | 40.6 | 63.8 | 53.1 | 37.4 | 0.969 | 2.003 |
| | $5.21\times10^{21}$ | 46.7 | 62.5 | 64.4 | 58.8 | 51.6 | 55.8 | 28.0 | 27.1 | 44.5 | 42.6 | 21.4 | 50.1 | 38.2 | 58.7 | 47.2 | 42.8 | 53.5 | 52.7 | 78.0 | 65.2 | 77.4 | 42.6 | 60.8 | 68.9 | 43.1 | 38.4 | 46.3 | 62.0 | 54.4 | 38.8 | 0.927 | 1.966 |
| | $1.30\times10^{22}$ | 48.6 | 60.6 | 65.7 | 61.2 | 52.9 | 59.2 | 28.6 | 29.3 | 46.3 | 44.0 | 24.8 | 53.0 | 40.8 | 58.7 | 50.6 | 47.8 | 54.2 | 55.6 | 78.0 | 67.2 | 78.1 | 43.3 | 64.2 | 70.8 | 44.0 | 39.6 | 50.5 | 70.3 | 56.7 | 39.1 | 0.889 | 1.930 |
| ▼ In-fusion | $6.51\times10^{20}$ | 37.4 | 48.8 | 50.4 | 48.9 | 48.0 | 39.0 | 24.0 | 26.8 | 31.8 | 35.2 | 14.2 | 36.1 | 18.4 | 52.7 | 41.8 | 33.7 | 48.7 | 45.7 | 70.0 | 57.1 | 73.8 | 41.5 | 55.1 | 62.1 | 35.9 | 35.2 | 30.0 | 63.4 | 48.3 | 35.7 | 1.129 | 2.117 |
| | $1.30\times10^{21}$ | 40.4 | 56.6 | 56.2 | 51.8 | 47.4 | 46.2 | 24.8 | 27.4 | 33.9 | 37.4 | 16.4 | 42.8 | 26.8 | 54.7 | 44.8 | 29.8 | 49.5 | 48.1 | 79.0 | 60.9 | 75.7 | 41.4 | 58.7 | 63.1 | 38.1 | 35.8 | 33.8 | 63.5 | 50.9 | 36.8 | 1.029 | 2.051 |
| | $2.60\times10^{21}$ | 42.6 | 57.2 | 57.9 | 53.6 | 50.1 | 48.5 | 24.9 | 27.1 | 39.4 | 40.0 | 21.2 | 45.6 | 32.5 | 54.0 | 44.6 | 36.2 | 49.1 | 50.7 | 77.0 | 63.1 | 77.4 | 42.5 | 60.9 | 65.8 | 40.9 | 37.2 | 39.5 | 65.4 | 53.4 | 38.3 | 0.964 | 2.001 |
| | $5.21\times10^{21}$ | 46.5 | 62.5 | 62.7 | 57.9 | 51.1 | 57.3 | 27.7 | 27.4 | 46.3 | 43.8 | 23.1 | 48.0 | 37.2 | 58.3 | 49.5 | 40.5 | 50.2 | 53.4 | 80.0 | 65.4 | 76.8 | 42.6 | 62.0 | 68.5 | 42.8 | 39.8 | 46.6 | 64.3 | 54.7 | 39.1 | 0.922 | 1.965 |
| | $1.30\times10^{22}$ | 49.2 | 61.8 | 65.8 | 60.0 | 52.5 | 58.3 | 30.8 | 30.8 | 52.2 | 46.8 | 25.2 | 53.2 | 41.3 | 61.3 | 46.3 | 49.5 | 51.4 | 55.9 | 83.0 | 67.6 | 77.8 | 44.3 | 61.6 | 71.4 | 44.4 | 39.2 | 49.3 | 73.6 | 54.5 | 40.3 | 0.886 | 1.927 |
| *4B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| ▲ In-fusion (FFN-only) | $2.90\times10^{21}$ | 44.8 | 57.9 | 60.6 | 56.5 | 50.9 | 54.4 | 26.3 | 29.3 | 46.0 | 45.2 | 21.1 | 47.3 | 34.2 | 53.3 | 46.5 | 35.1 | 52.8 | 50.8 | 81.0 | 65.7 | 77.9 | 41.5 | 62.7 | 68.1 | 41.5 | 36.8 | 42.2 | 55.7 | 54.5 | 39.1 | 0.980 | 1.959 |
| | $5.79\times10^{21}$ | 48.4 | 61.3 | 66.2 | 59.6 | 52.0 | 60.5 | 30.8 | 30.0 | 50.4 | 47.2 | 22.7 | 50.0 | 39.6 | 57.3 | 47.2 | 40.9 | 58.7 | 55.8 | 84.0 | 68.5 | 78.0 | 43.3 | 62.0 | 70.0 | 46.5 | 39.0 | 49.7 | 70.7 | 54.9 | 40.5 | 0.917 | 1.905 |
| | $1.16\times10^{22}$ | 49.8 | 62.4 | 68.4 | 63.3 | 53.0 | 59.1 | 28.2 | 31.0 | 51.5 | 49.8 | 24.9 | 51.6 | 41.3 | 60.0 | 49.9 | 48.5 | 53.5 | 57.1 | 84.0 | 70.3 | 78.2 | 43.4 | 63.0 | 72.4 | 47.6 | 38.6 | 52.6 | 72.0 | 57.5 | 40.7 | 0.886 | 1.874 |
| | $2.90\times10^{22}$ | 53.4 | 68.2 | 71.6 | 66.2 | 54.2 | 64.7 | 32.1 | 32.5 | 58.3 | 53.5 | 25.8 | 55.4 | 46.7 | 62.7 | 51.2 | 54.7 | 57.4 | 60.2 | 87.0 | 73.1 | 79.8 | 45.3 | 64.7 | 75.8 | 49.7 | 41.6 | 56.4 | 78.3 | 58.6 | 41.5 | 0.846 | 1.827 |
| ▼ In-fusion | $2.90\times10^{21}$ | 45.5 | 59.3 | 62.3 | 60.1 | 51.3 | 55.4 | 26.0 | 31.2 | 45.7 | 45.0 | 19.1 | 45.7 | 31.9 | 58.0 | 47.5 | 40.4 | 49.5 | 51.9 | 77.0 | 66.1 | 76.7 | 41.6 | 62.7 | 69.4 | 43.0 | 38.2 | 42.2 | 63.2 | 54.2 | 39.1 | 0.977 | 1.958 |
| | $5.79\times10^{21}$ | 49.0 | 60.8 | 67.0 | 63.1 | 53.0 | 62.9 | 27.1 | 31.3 | 50.7 | 48.9 | 23.5 | 49.1 | 38.0 | 60.7 | 48.8 | 45.3 | 54.3 | 55.9 | 81.0 | 68.5 | 77.9 | 43.3 | 62.4 | 71.9 | 44.9 | 39.4 | 48.9 | 73.4 | 55.6 | 40.4 | 0.908 | 1.904 |
| | $1.16\times10^{22}$ | 51.8 | 66.6 | 70.5 | 66.3 | 53.2 | 64.3 | 31.6 | 32.5 | 56.4 | 51.0 | 25.3 | 51.6 | 41.7 | 59.0 | 49.9 | 56.5 | 53.2 | 57.7 | 82.0 | 70.3 | 79.2 | 43.9 | 64.2 | 72.2 | 47.5 | 41.4 | 52.6 | 74.2 | 57.4 | 40.6 | 0.871 | 1.870 |
| | $2.90\times10^{22}$ | 54.3 | 70.1 | 71.1 | 66.4 | 55.0 | 68.2 | 31.9 | 33.5 | 57.9 | 54.3 | 26.3 | 55.3 | 47.1 | 65.7 | 51.6 | 54.7 | 59.7 | 60.7 | 82.0 | 73.1 | 79.4 | 44.7 | 67.0 | 75.0 | 50.6 | 42.4 | 57.8 | 79.7 | 59.4 | 41.3 | 0.828 | 1.821 |

**Table 43 Per-benchmark results for Figure 6.**

| Setup | | VQA (%) ↑ | | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Recipe | FLOPs | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| *1.7B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $6.51\times10^{20}$ | 37.3 | 51.7 | 48.0 | 42.7 | 47.0 | 42.5 | 26.2 | 24.4 | 32.0 | 35.7 | 12.8 | 34.5 | 18.3 | 51.3 | 40.3 | 36.7 | 52.5 | 45.3 | 69.0 | 56.5 | 75.5 | 41.4 | 58.5 | 61.9 | 36.8 | 36.4 | 30.4 | 55.8 | 49.3 | 36.0 | 1.169 | 2.118 |
| | $1.30\times10^{21}$ | 41.2 | 54.8 | 53.9 | 50.0 | 48.7 | 46.1 | 25.3 | 24.9 | 35.6 | 40.5 | 19.9 | 45.7 | 29.8 | 52.3 | 43.5 | 34.0 | 54.2 | 47.3 | 73.0 | 60.6 | 76.7 | 42.0 | 59.0 | 63.4 | 38.5 | 36.4 | 32.4 | 60.8 | 51.3 | 36.9 | 1.029 | 2.052 |
| | $2.60\times10^{21}$ | 44.4 | 61.3 | 60.7 | 56.0 | 51.0 | 49.7 | 26.0 | 27.0 | 40.2 | 43.9 | 23.0 | 50.4 | 35.6 | 55.3 | 45.6 | 35.3 | 49.6 | 49.7 | 75.0 | 63.3 | 76.9 | 42.9 | 61.2 | 66.8 | 40.6 | 36.2 | 36.7 | 64.5 | 53.3 | 37.7 | 0.960 | 2.003 |
| | $5.21\times10^{21}$ | 46.5 | 60.3 | 63.8 | 57.5 | 52.4 | 55.2 | 28.8 | 28.8 | 42.8 | 44.4 | 23.7 | 50.7 | 38.8 | 57.7 | 47.3 | 43.2 | 49.4 | 52.5 | 77.0 | 65.4 | 78.8 | 43.2 | 60.9 | 68.0 | 42.1 | 39.4 | 41.5 | 69.2 | 55.4 | 38.8 | 0.922 | 1.967 |
| | $1.30\times10^{22}$ | 48.9 | 64.1 | 66.5 | 58.5 | 53.4 | 56.7 | 31.4 | 29.2 | 47.7 | 46.2 | 25.3 | 54.9 | 43.5 | 58.0 | 45.8 | 42.2 | 58.8 | 55.3 | 80.0 | 67.7 | 77.5 | 43.8 | 61.9 | 69.7 | 44.0 | 39.4 | 49.3 | 70.3 | 56.7 | 40.1 | 0.890 | 1.930 |
| ▪ Encoder-free | $6.51\times10^{20}$ | 32.0 | 47.0 | 40.4 | 37.9 | 44.5 | 39.2 | 26.9 | 23.2 | 33.3 | 18.6 | 5.0 | 16.1 | 4.0 | 48.7 | 39.5 | 36.8 | 50.8 | 46.4 | 70.0 | 57.0 | 75.1 | 42.2 | 57.1 | 61.5 | 35.7 | 36.0 | 30.6 | 64.4 | 49.7 | 36.4 | 1.307 | 2.114 |
| | $1.30\times10^{21}$ | 34.8 | 48.6 | 43.3 | 42.1 | 45.4 | 43.0 | 25.2 | 23.4 | 32.4 | 27.4 | 11.4 | 29.4 | 11.7 | 52.0 | 38.4 | 32.2 | 50.5 | 48.3 | 72.0 | 59.8 | 75.7 | 42.5 | 59.7 | 63.4 | 39.7 | 34.6 | 34.1 | 66.5 | 51.4 | 37.9 | 1.146 | 2.050 |
| | $2.60\times10^{21}$ | 39.0 | 52.5 | 54.3 | 48.3 | 47.8 | 48.4 | 28.0 | 25.3 | 38.3 | 33.0 | 13.8 | 36.2 | 18.0 | 51.7 | 40.5 | 35.0 | 52.8 | 50.7 | 78.0 | 63.2 | 75.8 | 42.3 | 60.9 | 67.6 | 41.7 | 37.0 | 38.2 | 67.2 | 51.9 | 38.7 | 1.061 | 2.003 |
| | $5.21\times10^{21}$ | 41.9 | 58.9 | 57.6 | 53.0 | 49.3 | 52.2 | 26.2 | 24.9 | 41.7 | 35.8 | 17.2 | 39.7 | 23.3 | 53.3 | 46.1 | 39.2 | 51.6 | 52.6 | 79.0 | 65.1 | 77.7 | 43.1 | 61.7 | 68.2 | 43.0 | 37.6 | 42.9 | 67.4 | 54.9 | 39.2 | 1.014 | 1.964 |
| | $1.30\times10^{22}$ | 44.8 | 56.9 | 62.5 | 56.4 | 49.8 | 53.5 | 29.0 | 26.9 | 45.9 | 41.7 | 19.9 | 45.2 | 31.9 | 56.7 | 45.1 | 44.7 | 50.8 | 55.7 | 81.0 | 67.5 | 77.5 | 44.4 | 62.1 | 70.7 | 42.7 | 39.6 | 48.6 | 73.7 | 56.7 | 40.4 | 0.964 | 1.930 |
| ▾ In-fusion | $6.51\times10^{20}$ | 37.4 | 48.8 | 50.4 | 48.9 | 48.0 | 39.0 | 24.0 | 26.8 | 31.8 | 35.2 | 14.2 | 36.1 | 18.4 | 52.7 | 41.8 | 33.7 | 48.7 | 45.7 | 70.0 | 57.1 | 73.8 | 41.5 | 55.1 | 62.1 | 35.9 | 35.2 | 30.0 | 63.4 | 48.3 | 35.7 | 1.129 | 2.117 |
| | $1.30\times10^{21}$ | 40.4 | 56.6 | 56.2 | 51.8 | 47.4 | 46.2 | 24.8 | 27.4 | 33.9 | 37.4 | 16.4 | 42.8 | 26.8 | 54.7 | 44.8 | 29.8 | 49.5 | 48.1 | 79.0 | 60.9 | 75.7 | 41.4 | 58.7 | 63.1 | 38.1 | 35.8 | 33.8 | 63.5 | 50.9 | 36.8 | 1.029 | 2.051 |
| | $2.60\times10^{21}$ | 42.6 | 57.2 | 57.9 | 53.6 | 50.1 | 48.5 | 24.9 | 27.1 | 39.4 | 40.0 | 21.2 | 45.6 | 32.5 | 54.0 | 44.6 | 36.2 | 49.1 | 50.7 | 77.0 | 63.1 | 77.4 | 42.5 | 60.9 | 65.8 | 40.9 | 37.2 | 39.5 | 65.4 | 53.4 | 38.3 | 0.964 | 2.001 |
| | $5.21\times10^{21}$ | 46.5 | 62.5 | 62.7 | 57.9 | 51.1 | 57.3 | 27.7 | 27.4 | 46.3 | 43.8 | 23.1 | 48.0 | 37.2 | 58.3 | 49.5 | 40.5 | 50.2 | 53.4 | 80.0 | 65.4 | 76.8 | 42.6 | 62.0 | 68.5 | 42.8 | 39.8 | 46.6 | 64.3 | 54.7 | 39.1 | 0.922 | 1.965 |
| | $1.30\times10^{22}$ | 49.2 | 61.8 | 65.8 | 60.0 | 52.5 | 58.3 | 30.8 | 30.8 | 52.2 | 46.8 | 25.2 | 53.2 | 41.3 | 61.3 | 46.3 | 49.5 | 51.4 | 55.9 | 83.0 | 67.6 | 77.8 | 44.3 | 61.6 | 71.4 | 44.4 | 39.2 | 49.3 | 73.6 | 54.5 | 40.3 | 0.886 | 1.927 |
| *4B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $2.90\times10^{21}$ | 47.8 | 61.3 | 63.8 | 59.1 | 51.9 | 57.1 | 29.1 | 31.6 | 45.9 | 48.2 | 24.0 | 50.7 | 36.1 | 56.7 | 45.1 | 50.6 | 54.3 | 52.0 | 78.0 | 65.9 | 77.4 | 43.0 | 63.4 | 69.0 | 42.1 | 39.0 | 41.1 | 65.6 | 52.6 | 38.7 | 0.944 | 1.959 |
| | $5.79\times10^{21}$ | 51.2 | 65.4 | 69.1 | 63.2 | 53.0 | 64.0 | 30.3 | 33.1 | 50.8 | 50.4 | 26.9 | 54.0 | 42.6 | 63.0 | 49.5 | 50.0 | 53.4 | 56.4 | 83.0 | 68.3 | 78.2 | 44.1 | 64.2 | 72.7 | 45.2 | 40.6 | 49.3 | 72.5 | 56.3 | 40.4 | 0.891 | 1.905 |
| | $1.16\times10^{22}$ | 51.9 | 64.8 | 71.0 | 62.5 | 53.7 | 65.9 | 29.1 | 34.0 | 55.2 | 53.0 | 27.8 | 55.4 | 45.8 | 63.0 | 49.3 | 47.3 | 52.9 | 57.9 | 83.0 | 70.6 | 79.0 | 45.3 | 65.0 | 73.1 | 47.7 | 40.6 | 52.3 | 75.2 | 56.8 | 40.5 | 0.864 | 1.867 |
| | $2.90\times10^{22}$ | 54.5 | 71.2 | 73.3 | 67.6 | 53.5 | 68.5 | 33.7 | 34.6 | 59.7 | 52.6 | 26.9 | 56.3 | 46.2 | 63.0 | 54.1 | 55.6 | 55.2 | 60.4 | 81.0 | 73.1 | 80.1 | 46.3 | 66.9 | 74.7 | 49.9 | 40.8 | 57.5 | 79.6 | 58.5 | 41.3 | 0.850 | 1.819 |
| ▪ Encoder-free | $2.90\times10^{21}$ | 41.0 | 56.6 | 57.1 | 53.7 | 48.6 | 51.7 | 27.8 | 30.3 | 43.0 | 37.9 | 13.4 | 35.9 | 16.8 | 50.3 | 42.5 | 37.3 | 53.1 | 51.5 | 74.0 | 65.8 | 77.9 | 42.1 | 60.8 | 68.0 | 41.7 | 38.0 | 42.2 | 63.2 | 53.4 | 39.2 | 1.057 | 1.958 |
| | $5.79\times10^{21}$ | 44.9 | 57.4 | 61.6 | 58.0 | 50.8 | 56.7 | 29.2 | 27.7 | 48.8 | 40.4 | 19.1 | 43.2 | 27.6 | 55.3 | 46.4 | 41.6 | 53.8 | 56.0 | 82.0 | 68.5 | 78.2 | 43.4 | 63.4 | 70.2 | 45.6 | 41.4 | 49.5 | 70.0 | 56.8 | 40.7 | 0.970 | 1.905 |
| | $1.16\times10^{22}$ | 48.4 | 58.1 | 68.4 | 62.9 | 52.5 | 61.4 | 29.4 | 31.1 | 53.1 | 43.0 | 22.3 | 49.2 | 35.4 | 58.7 | 46.1 | 45.1 | 58.3 | 57.0 | 84.0 | 70.2 | 79.1 | 43.2 | 64.9 | 72.9 | 47.0 | 40.0 | 52.3 | 68.6 | 57.4 | 41.3 | 0.921 | 1.870 |
| | $2.90\times10^{22}$ | 51.7 | 65.7 | 69.8 | 65.4 | 54.6 | 63.0 | 31.9 | 31.1 | 57.3 | 48.1 | 24.3 | 53.5 | 40.9 | 60.7 | 51.6 | 51.9 | 58.1 | 61.0 | 84.0 | 73.1 | 78.7 | 44.6 | 67.2 | 74.8 | 49.6 | 43.6 | 58.3 | 79.6 | 58.4 | 42.3 | 0.870 | 1.821 |
| ▾ In-fusion | $2.90\times10^{21}$ | 45.5 | 59.3 | 62.3 | 60.1 | 51.3 | 55.4 | 26.0 | 31.2 | 45.7 | 45.0 | 19.1 | 45.7 | 31.9 | 58.0 | 47.5 | 40.4 | 49.5 | 51.9 | 77.0 | 66.1 | 76.7 | 41.6 | 62.7 | 69.4 | 43.0 | 38.2 | 42.2 | 63.2 | 54.2 | 39.1 | 0.977 | 1.958 |
| | $5.79\times10^{21}$ | 49.0 | 60.8 | 67.0 | 63.1 | 53.0 | 62.9 | 27.1 | 31.3 | 50.7 | 48.9 | 23.5 | 49.1 | 38.0 | 60.7 | 48.8 | 45.3 | 54.3 | 55.9 | 81.0 | 68.5 | 77.9 | 43.3 | 62.4 | 71.9 | 44.9 | 39.4 | 48.9 | 73.4 | 55.6 | 40.4 | 0.908 | 1.904 |
| | $1.16\times10^{22}$ | 51.8 | 66.6 | 70.5 | 66.3 | 53.2 | 64.3 | 31.6 | 32.5 | 56.4 | 51.0 | 25.3 | 51.6 | 41.7 | 59.0 | 49.9 | 56.5 | 53.2 | 57.7 | 82.0 | 70.3 | 79.2 | 43.9 | 64.2 | 72.2 | 47.5 | 41.4 | 52.6 | 74.2 | 57.4 | 40.6 | 0.871 | 1.870 |
| | $2.90\times10^{22}$ | 54.3 | 70.1 | 71.1 | 66.4 | 55.0 | 68.2 | 31.9 | 33.5 | 57.9 | 54.3 | 26.3 | 55.3 | 47.1 | 65.7 | 51.6 | 54.7 | 59.7 | 60.7 | 82.0 | 73.1 | 79.4 | 44.7 | 67.0 | 75.0 | 50.6 | 42.4 | 57.8 | 79.7 | 59.4 | 41.3 | 0.828 | 1.821 |

**Table 44 Per-benchmark results for Figure 7.** All runs train to 64 TPP.

| Setup | | VQA (%) ↑ | | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Recipe | FLOPs | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| *1B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $3.66\times10^{20}$ | 32.9 | 45.4 | 37.0 | 37.0 | 44.7 | 36.7 | 24.9 | 23.0 | 26.9 | 31.8 | 10.0 | 28.5 | 11.1 | 49.7 | 38.2 | 31.0 | 51.3 | 43.3 | 70.0 | 54.4 | 73.5 | 41.0 | 57.1 | 58.7 | 36.8 | 35.4 | 26.2 | 53.3 | 48.4 | 35.0 | 1.229 | 2.161 |
| ▪ Encoder-free | $3.66\times10^{20}$ | 29.1 | 46.9 | 31.0 | 30.5 | 41.9 | 37.4 | 22.3 | 20.4 | 26.9 | 15.0 | 3.2 | 12.6 | 2.9 | 49.3 | 37.4 | 38.9 | 49.6 | 43.5 | 76.0 | 55.1 | 74.5 | 40.0 | 56.5 | 60.5 | 33.9 | 34.8 | 25.2 | 55.8 | 48.2 | 35.4 | 1.388 | 2.158 |
| ▾ In-fusion | $3.66\times10^{20}$ | 33.6 | 52.4 | 36.5 | 35.6 | 45.4 | 38.3 | 22.7 | 24.3 | 27.4 | 30.3 | 10.7 | 28.9 | 10.0 | 50.3 | 41.7 | 32.5 | 50.4 | 44.0 | 68.0 | 54.3 | 74.4 | 39.3 | 56.4 | 58.5 | 36.2 | 34.6 | 27.3 | 59.2 | 48.5 | 35.6 | 1.196 | 2.160 |
| *1.7B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $1.06\times10^{21}$ | 39.9 | 53.9 | 50.6 | 47.8 | 48.0 | 44.6 | 27.6 | 25.1 | 33.1 | 39.3 | 17.7 | 40.2 | 22.7 | 53.0 | 44.1 | 34.6 | 56.8 | 48.2 | 78.0 | 60.6 | 76.4 | 42.3 | 58.4 | 65.3 | 38.2 | 36.8 | 32.2 | 66.0 | 50.1 | 36.8 | 1.080 | 2.054 |
| ▪ Encoder-free | $1.06\times10^{21}$ | 34.9 | 47.5 | 45.0 | 42.2 | 46.3 | 47.2 | 25.0 | 23.0 | 35.2 | 25.6 | 8.2 | 25.0 | 7.4 | 51.3 | 41.0 | 37.8 | 50.4 | 48.3 | 75.0 | 60.3 | 75.8 | 40.5 | 60.9 | 63.4 | 40.4 | 35.6 | 34.4 | 64.5 | 49.7 | 36.5 | 1.198 | 2.050 |
| ▾ In-fusion | $1.06\times10^{21}$ | 38.4 | 51.6 | 55.4 | 50.1 | 48.8 | 46.5 | 23.4 | 25.7 | 32.6 | 35.0 | 13.4 | 38.5 | 20.0 | 53.7 | 39.0 | 33.4 | 46.6 | 48.1 | 77.0 | 60.2 | 75.7 | 41.2 | 58.6 | 63.9 | 38.1 | 36.4 | 32.4 | 66.7 | 51.0 | 36.4 | 1.066 | 2.052 |
| *3.2B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $3.91\times10^{21}$ | 46.4 | 60.5 | 61.0 | 58.9 | 52.2 | 54.1 | 28.2 | 29.4 | 43.6 | 45.1 | 21.7 | 50.3 | 37.8 | 57.7 | 48.9 | 42.4 | 51.0 | 53.0 | 83.0 | 67.1 | 78.2 | 42.3 | 62.3 | 71.0 | 44.7 | 39.2 | 42.2 | 66.3 | 54.5 | 38.9 | 0.935 | 1.931 |
| ▪ Encoder-free | $3.91\times10^{21}$ | 39.9 | 55.0 | 54.5 | 52.2 | 47.9 | 51.9 | 27.0 | 25.5 | 44.3 | 32.7 | 11.0 | 33.9 | 14.7 | 54.0 | 43.1 | 41.7 | 49.3 | 53.7 | 79.0 | 67.0 | 77.3 | 43.2 | 62.1 | 70.2 | 46.4 | 38.2 | 44.0 | 69.2 | 54.6 | 39.9 | 1.064 | 1.930 |
| ▾ In-fusion | $3.91\times10^{21}$ | 45.9 | 59.2 | 62.7 | 58.8 | 52.7 | 54.9 | 26.1 | 28.1 | 40.1 | 45.0 | 22.6 | 49.2 | 35.0 | 57.3 | 50.2 | 41.1 | 50.9 | 52.7 | 78.0 | 66.9 | 77.6 | 42.7 | 62.6 | 68.0 | 44.5 | 38.0 | 41.6 | 68.8 | 55.8 | 39.8 | 0.934 | 1.929 |
| *6.4B backbone* | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| • Scratch encoder | $1.56\times10^{22}$ | 52.8 | 64.1 | 70.0 | 65.5 | 53.8 | 63.2 | 32.0 | 33.7 | 56.4 | 52.5 | 28.2 | 57.6 | 47.0 | 63.3 | 52.8 | 52.3 | 52.6 | 59.1 | 80.0 | 72.8 | 79.4 | 44.8 | 64.7 | 75.5 | 51.8 | 41.4 | 53.9 | 76.8 | 58.4 | 41.7 | 0.842 | 1.826 |
| ▪ Encoder-free | $1.56\times10^{22}$ | 47.5 | 61.8 | 65.6 | 61.3 | 51.9 | 58.3 | 26.8 | 30.3 | 52.2 | 42.4 | 20.0 | 46.6 | 29.1 | 57.3 | 49.9 | 50.9 | 55.4 | 58.9 | 85.0 | 72.6 | 79.5 | 43.9 | 66.6 | 73.5 | 51.8 | 41.8 | 54.3 | 73.2 | 57.0 | 41.3 | 0.923 | 1.823 |
| ▾ In-fusion | $1.56\times10^{22}$ | 52.0 | 64.6 | 71.0 | 64.2 | 54.0 | 65.3 | 29.8 | 34.5 | 57.2 | 51.8 | 24.1 | 55.1 | 42.6 | 65.3 | 49.4 | 49.7 | 52.8 | 59.1 | 87.0 | 72.8 | 79.3 | 45.0 | 64.9 | 72.5 | 50.2 | 40.8 | 54.8 | 75.7 | 57.2 | 40.9 | 0.843 | 1.824 |

**Table 45 Per-benchmark results for Figure 8.** All runs use a 1.7B backbone and the same $2.7 \times 10^{21}$-FLOP multimodal stage; Pretraining is the encoder's pretraining compute in FLOPs.

| Setup | | VQA (%) ↑ | | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Encoder | Pretraining | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| MetaCLIP | $8.98\times10^{21}$ | 41.6 | 58.1 | 53.3 | 52.0 | 48.7 | 49.7 | 24.0 | 28.1 | 37.8 | 40.6 | 20.3 | 45.0 | 33.8 | 52.3 | 44.3 | 29.6 | 48.3 | 50.7 | 77.0 | 63.2 | 77.6 | 42.7 | 60.2 | 66.7 | 39.7 | 37.6 | 37.9 | 68.6 | 53.2 | 38.0 | 0.987 | 2.001 |
| | $5.52\times10^{21}$ | 42.7 | 55.2 | 58.1 | 53.6 | 48.5 | 50.1 | 24.7 | 27.7 | 40.7 | 42.3 | 21.3 | 46.0 | 33.0 | 50.3 | 41.7 | 40.1 | 50.1 | 51.0 | 79.0 | 63.5 | 76.7 | 43.0 | 60.4 | 66.6 | 40.7 | 37.0 | 39.2 | 67.1 | 53.1 | 38.4 | 0.986 | 2.001 |
| | $2.76\times10^{21}$ | 42.2 | 56.7 | 54.0 | 51.3 | 49.5 | 46.8 | 24.7 | 27.7 | 38.3 | 41.8 | 21.4 | 47.5 | 34.2 | 54.0 | 45.9 | 33.0 | 48.8 | 51.0 | 80.0 | 62.9 | 76.3 | 43.4 | 60.9 | 66.4 | 40.2 | 37.6 | 38.6 | 68.8 | 51.7 | 37.9 | 0.981 | 2.001 |
| | $1.38\times10^{21}$ | 43.2 | 60.2 | 57.9 | 53.8 | 49.0 | 46.5 | 27.1 | 27.8 | 38.0 | 42.0 | 21.5 | 48.2 | 34.6 | 52.7 | 43.7 | 37.6 | 50.2 | 50.5 | 77.0 | 63.4 | 76.5 | 42.3 | 60.3 | 66.3 | 40.2 | 37.2 | 39.9 | 63.7 | 52.4 | 37.9 | 0.981 | 2.001 |
| Scratch encoder | 0 | 43.5 | 60.1 | 59.3 | 54.9 | 50.9 | 50.0 | 24.0 | 27.4 | 40.8 | 42.2 | 21.6 | 49.6 | 35.6 | 58.0 | 42.1 | 32.1 | 47.9 | 50.6 | 76.0 | 63.4 | 76.6 | 43.0 | 62.4 | 67.7 | 39.9 | 38.6 | 36.3 | 68.6 | 53.4 | 37.9 | 0.964 | 2.001 |

**Table 46 Per-benchmark results for Figure 9.** Each run continues a 1.7B text-only model from 1T to 1.33T token-equivalents.

| Setup | VQA (%) ↑ | | | | | | | | | | | | | | | | | Text (%) ↑ | | | | | | | | | | | | | Held-out loss ↓ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Gen | | | | Knw | | | | OCR | | | | Vis | | | | | Com | | | | | Sci | | | Knw | Rdg | | | | |
| Recipe | Avg | MME-P | MMB | SEED | GQA | SQA | MMMU | MathV | AI2D | Chart | OCRB | TextV | DocV | MMVP | RWQA | CV2D | CV3D | Avg | COPA | HS | PIQA | SIQA | WG | ARC-E | ARC-C | OBQA | MMLU | BoolQ | RACEm | RACEh | Cap. | Text |
| In-fusion | 50.1 | 62.3 | 68.1 | 62.0 | 51.7 | 60.6 | 30.3 | 30.0 | 51.3 | 45.4 | 25.3 | 52.1 | 41.4 | 60.3 | 49.8 | 48.2 | 62.3 | 56.4 | 80.0 | 68.0 | 78.5 | 45.6 | 61.4 | 71.3 | 42.7 | 38.6 | 51.0 | 75.5 | 56.2 | 40.4 | 0.903 | 1.913 |
| Scratch encoder | 48.6 | 54.7 | 65.6 | 62.6 | 52.4 | 60.4 | 29.2 | 29.4 | 49.0 | 45.8 | 23.8 | 51.2 | 34.9 | 57.7 | 51.1 | 52.0 | 57.1 | 56.7 | 82.0 | 67.9 | 78.4 | 45.0 | 64.2 | 70.9 | 43.9 | 40.4 | 50.5 | 75.0 | 56.3 | 40.4 | 0.930 | 1.914 |
| MoT, layers 1–8 | 48.5 | 61.8 | 64.9 | 60.9 | 52.7 | 58.1 | 29.9 | 29.7 | 49.5 | 46.6 | 22.0 | 48.9 | 40.2 | 58.7 | 51.9 | 48.3 | 51.3 | 56.7 | 79.0 | 68.1 | 78.7 | 44.9 | 63.5 | 72.0 | 44.7 | 39.2 | 51.6 | 74.0 | 56.6 | 40.0 | 0.909 | 1.912 |
| MetaCLIP ("free") | 43.5 | 57.2 | 58.4 | 55.6 | 49.7 | 57.0 | 28.8 | 27.6 | 45.5 | 36.0 | 17.2 | 43.0 | 21.0 | 52.7 | 47.1 | 43.4 | 56.2 | 56.5 | 83.0 | 67.8 | 78.1 | 44.4 | 63.2 | 71.0 | 43.1 | 39.0 | 50.9 | 74.2 | 56.5 | 40.4 | 0.992 | 1.915 |
| MoT, all layers | 43.3 | 49.1 | 50.9 | 51.8 | 51.6 | 52.6 | 31.1 | 29.9 | 43.8 | 42.1 | 24.2 | 51.7 | 34.6 | 52.0 | 44.8 | 34.0 | 48.3 | 56.6 | 82.0 | 68.0 | 78.3 | 44.9 | 63.5 | 70.9 | 43.3 | 39.0 | 50.9 | 75.3 | 55.1 | 40.3 | 0.872 | 1.911 |
| Encoder-free | 39.8 | 50.6 | 46.6 | 50.9 | 48.4 | 54.3 | 25.4 | 26.8 | 48.3 | 33.0 | 14.4 | 36.9 | 18.3 | 53.0 | 44.6 | 45.1 | 40.0 | 56.6 | 84.0 | 67.7 | 78.4 | 44.5 | 63.6 | 69.4 | 43.0 | 41.0 | 51.4 | 73.2 | 55.6 | 40.0 | 1.038 | 1.916 |
| Text-only control | – | – | – | – | – | – | – | – | – | – | – | – | – | – | – | – | – | 57.3 | 85.0 | 68.8 | 78.6 | 44.3 | 63.3 | 71.6 | 45.9 | 39.0 | 51.6 | 76.0 | 56.4 | 40.5 | – | 1.909 |